质量属性：

• 质量属性，也称为非功能性需求，用于衡量一个软件系统如何运作，而非他具体做什么的指标。他们定义了系统的品质和外部可见特性，是系统功能之外的约束和要求

• 功能性需求回答的是“系统需要做什么”（如系统允许用户下单购买商品）

• 质量属性回答的是“系统需要做得多好”（如下单操作必须在 2 秒内完成）

质量属性分类 1：

• 开发期：易理解性、可扩展性、可重用性、可测试性、可维护性、可移植性

• 运行期：性能、安全性、可伸缩性、互操作性、可靠性、可用性、鲁棒性(健壮性、容错性)、易用性

质量属性分类 2：

• 面向架构评估时质量属性：性能、可靠性、可用性、安全性、可修改性、功能性、可变性、互操作性

可靠性和可用性对比：

特征	可靠性	可用性
核心问题	系统会失败吗	系统可用吗
关注点	**故障间隔，多久坏一次**	**正常运行时间比例**
关注指标	平均无故障时间	可用性百分比
时间视角	**两次故障之间的时长**	**总时间内的可用时长**
比喻	汽车不容易抛锚	想开车时，车总在那里
提升手段	**提高代码质量、容错设计、彻底测试**	**冗余、自动故障转移、快速恢复机制**

可修改性和可变性对比：

特征	可修改性	可变性
核心问题	**修改代码有多难**	**不修改代码能做多少事**
关注点	**变更的成本、风险和工作量**	**内置的灵活性、配置和扩展能力**
变化类型	未计划的、未知的变化	已计划的、预期的变化
实现方式	通过架构风格和设计原则（模块化、低耦合）	通过具体机制（配置文件、插件接口、功能开关）
主要受益者	开发者、维护人员	管理员、用户、实施人员
比喻	房子的墙体结构是否易于改造	房子是否预装了一键切换的灯光模式

质量属性描述、关注点及其提升方法：

• 性能：系统在给定时间间隔内处理特定任务或操作的效率和速度

  • 关注点：响应时间、吞吐量（单位时间内处理请求的数量）、资源利用率（CPU、内存、网络）

  • 提升方法：优先级队列、增加计算资源、减少计算开销、引入并发机制、采用资源调度

    • 优先级队列：在系统的任务调度或消息中间件中，使用具有优先级排序功能的队列数据结构（如堆）。每个任务在进入队列时都会被赋予一个优先级标识

    • 引入并发机制：将一个大的任务拆分成多个可以同时执行的小任务，利用多核 CPU 的计算能力

    • 多线程：在单个进程内创建多个线程来并行处理任务。适用于 I/O 密集型（线程在等待 I/O 时，CPU 可以执行其他线程）和计算密集型任务（在多核 CPU 上真正并行）

    • 多进程：创建多个独立的进程。进程间资源隔离，稳定性更高（一个进程崩溃不影响其他进程），但通信开销（IPC）比线程大

    • 异步编程：一种非阻塞的编程模型。单个线程通过事件循环处理多个任务，在遇到 I/O 等待时不会阻塞，而是去处理其他任务。非常适合高并发的 I/O 密集型应用（如 Node.js）

• 采用资源调度：

• 动态资源分配：在虚拟化或容器化环境中（如 Kubernetes），根据 Pod 的实际负载动态调整其可使用的 CPU 和内存上限

• 负载均衡：将新到的请求智能地分发给当前最空闲或能力最强的服务器

• 数据分片/分区：将大数据集水平拆分到不同的数据库实例或节点上，使读写操作可以并行进行，这是一种对数据资源的调度

• 可靠性：系统在特定环境和时间周期内无故障运行的能力和稳定性。【连续提供服务】

• 关注点：【平均无故障时间（MTBF）】、容错能力、系统在出现部分故障时能否继续运行或优雅降级

• 提升方法：冗余设计（硬件冗余、数据冗余）、错误检测与恢复机制（如奇偶校验、循环冗余校验）、异常处理（捕获并处理程序运行中的异常）、备份与恢复策略（定期备份数据，在故障时可恢复）、负载均衡（避免单个组件过载）、恢复块设计、N 版本程序设计

	恢复块设计	N 版本程序设计
基本思想	主块+备用块串行执行，出错切换	多版本独立开发，并行运行+多数表决
执行方式	串行执行	并行执行
冗余方式	软件模块冗余	多版本冗余+时间冗余
错误恢复	前向恢复	后向恢复
关键部件	验收测试	表决器
开发要求	同一设计，不同实现	独立设计、独立开发、独立验证
共因失效	无法避免	可有效降低
资源开销	小	大
实时性	较好	较差
实现成本	较低	很高
适用场景	资源有限、实时性要求较高的系统	高可靠、高安全关键系统

• 可用性：系统在需要时能够正常提供服务的时间和比例，通常用几个 9 来衡量。【正常提供服务的时间占比】【故障恢复得快】

• 关注点：系统正常运行时间、宕机频率和时长、可维护性（热升级不中断服务）

• 提升方法：心跳机制（检测系统组件的运行状态）、冗余系统（如双机热备、集群）、快速恢复机制（使用备用设备或系统）、内置监控器监控与预警（实时监控系统状态，提前发现潜在问题）、故障转移（自动将工作负载转移到备用组件）

• 心跳机制： 被监控节点主动、周期性地向监控方或其它节点发送一个简单的信号（即“心跳”），宣告自己“存活”。如果监控方在一段时间内没有收到心跳，就认为该节点已经宕机或发生网络分区

• Ping-Echo：监控方主动地向被监控节点发送一个探测请求（Ping），并等待其回复（Echo）。如果收到有效回复则认为服务健康，否则认为故障

• 内置监控器与预警：在系统内部植入一个专门的、功能更强大的监控代理，它不仅检查“是否存活”，还收集各种深度指标，并在异常发生前或发生时及时发出预警

	心跳机制	Ping-Echo	内置监控器与预警
主动性	被动接收（由被监控方主动推）	主动探测（由监控方主动拉）	主动采集与分析
监控粒度	粗粒度：只关心“死活”	可粗可细：从 TCP 端口通断到完整的业务健康检查	细粒度：涵盖系统、应用、业务等全方位指标
监控深度	浅：仅存活状态	中等：可达性，有时包括服务可用性	深：性能、资源、健康度、趋势
主要目的	快速发现故障，实现自动故障转移	服务健康检查，用于负载均衡和故障隔离	全面洞察系统状态，预警潜在风险，辅助定位问题
典型场景	集群主从切换（如 Redis Sentinel, Kafka）	负载均衡器健康检查（如 Nginx, K8s Liveness Probe）	全链路监控、APM（应用性能管理）、运维预警（如 Prometheus+Grafana+Alertmanager）

• 双机热备：主从切换—通常由一主一备两台服务器组成。主节点对外提供服务，备节点处于“热”状态（数据与主节点实时同步，但通常不处理业务）。当主节点故障时，由高可用软件控制备节点接管服务其目标是保证服务不中断。被动备用，备机资源在正常情况下是闲置的，只有当故障发生时才会被启用

• 集群：主动分担，所有节点的资源在正常情况下都被充分利用

特性/维度	双机热备	集群
核心目标	高可用性，保证业务连续性	高可用性 + 高可扩展性 + 高性能
资源利用率	低。备机处于闲置状态，造成资源浪费	高。所有节点都参与工作，负载均衡
架构复杂度	相对简单。主从角色清晰，数据同步和切换逻辑相对直接	复杂。涉及负载均衡、任务调度、数据分布式一致性、状态同步等复杂问题
故障切换时间	较短，但通常有秒级中断（需要检测故障、切换 VIP、启动服务等）	极短或无感知。对于用户来说，只是某个节点失效，请求被自动转发到其他健康节点
数据一致性	通过主从数据同步（如数据库日志复制、磁盘镜像）来保证	更具挑战性，需要使用分布式一致性协议（如 Paxos、Raft）或分布式事务
成本	硬件和软件成本相对较低	硬件、软件（如负载均衡器、集群管理软件）和运维成本通常更高
典型场景	数据库（如 MySQL 主从切换）、关键业务应用服务器、文件服务器	Web 服务器农场、分布式缓存（如 Redis Cluster）、大数据计算（如 Hadoop）、云计算基础设施

- 安全性：系统保护自身数据和资源免受未经授权的访问、修改或破坏的能力
  - 关注点：机密性、完整性、可用性、身份认证和授权
  - 提升方法：用户认证（如用户名密码认证、多因素认证）、授权管理（基于角色的访问控制）、数据加密（对敏感数据进行加密存储和传输）、防火墙（防止外部非法访问）、入侵检测与防范系统（检测并阻止网络攻击）、审计与日志记录（记录系统操作，便于事后审计）

- 可修改性：系统能够以较低的成本和风险进行变更的难易程度
  - 关注点：模块化、耦合度、变更的影响分析、扩展性功能的成本
  - 提升方法：模块化设计（降低模块间耦合度）、接口标准化（便于替换和扩展组件）、抽象与封装（隐藏实现细节，提高可维护性）、配置管理（使用配置文件管理系统参数）、版本控制（管理代码和文档的变更）

- 互操作性：系统与其他系统进行数据交换和协同工作的能力
  - 关注点：标准协议的支持、数据格式的兼容性、api 设计的清晰度
  - 提升方法：使用标准协议和接口（如 RESTfulAPI、SOAP）、数据格式转换（将不同系统的数据格式进行转换）、中间件（作为不同系统间的桥梁）、服务注册与发现（便于系统间相互发现和调用服务）、跨语言开发工具（支持不同语言间的交互）

- 可移植性：将软件系统从一个运行环境转移到另一个不同的运行环境的难易程度
  - 关注点：软件或硬件适应不同运行环境的能力
  - 提升方法：遵循相关标准协议并依赖标准 API，避免特定平台专有 API 以确保兼容性和减少平台依赖；抽象底层差异，通过封装底层代码和借助中间件或虚拟机隔离软件与底层的差异；进行兼容性测试，利用多种硬件平台、操作系统、浏览器等进行多平台测试

- 可测试性：软件测试以证明其满足需求规范的难易程度
  - 关注点：件易于发现缺陷、验证功能正确性和性能指标达标性的特性
  - 提升方法：设计分层与模块化：将系统拆分为独立模块，减少模块间耦合度，便于针对单个模块编写测试用例，降低测试复杂度；提供测试接口：在代码中预留专用的测试接口，允许测试人员直接访问内部状态或调用特定功能，辅助进行白盒测试

- 易用性：系统让用户易用的程度
  - 关注点：“好学、好找、好操作、少犯错”的程度，包括学习成本、操作效率、可理解性与可访问性
  - 提升方法：用户中心设计、一致的交互模型与抽象

- 功能性：系统是否能够满足用户明确和隐含的需求，提供所承诺的功能和行为。最根本的属性
  - 关注点：需求的完备性、正确性、适用性
  - 提升方法：分层架构与模块化、对外接口与系统交互

• 可变性：系统能够被轻松的配置、定制或扩展以适应不同用户、市场或环境需求的能力。他与修改行相关，但更强调【适配】而非修改

• 关注点：可适配性、可定制性、支持多租户

• 提升方法：高内聚、低耦合与模块化、元数据与配置化

质量属性场景的六要素：

• 刺激源：某个生成刺激的实体

• 刺激：当刺激达到系统时可能产生的影响

• 环境：该刺激在哪些条件下发生

• 制品：系统中受刺激的部分

• 响应：刺激达到后所采取的行动

• 响应度量：当相应发生时，用某种方式进行度量。

举例：

当大量并发用户（刺激源）在系统正常运行的高峰期（环境），同时发起大量查询与业务请求（刺激）时，应用服务与数据库等系统制品（制品）应能够正常处理请求并正确返回结果（响应），确保 95% 请求响应时间不超过 300ms，吞吐量不低于 1000 TPS（响应度量）。

如何解决传统数据库性能瓶颈？

磁盘 I/O 瓶颈，高并发导致读写性能下降

复杂查询的响应延迟

解决方法：在业务服务器和数据库之间增加一层缓存（Redis）

redis：key-value 结构

key：

只支持 string

value：

数据结构	存储内容与特性	典型应用场景
String	可以是文本、数字（整数/浮点数）或二进制数据一个键最大能存储 512MB	缓存对象、计数器、分布式锁、Session 存储
List	一个有序的字符串列表，元素可以重复支持从两端插入或弹出	消息队列、时间轴列表（如微博关注人动态）
Hash	键值对集合，类似于编程中的 Map，适合存储对象	存储用户信息、商品详情等结构化数据
Set	无序的字符串集合，元素不可重复，支持交集、并集、差集等运算	标签系统、共同好友、抽奖去重
Zset	与 Set 类似，但每个元素都会关联一个分数（score），元素按分数排序	排行榜、带权重的队列
Bitmap	通过 String 类型的位操作实现，本质上是一个位数组	用户签到统计、活跃用户分析
HyperLogLog	用于基数统计（估算不重复元素的数量），占用空间小，但有误差	大规模 UV（独立访客）统计
Geospatial	存储地理位置信息（经纬度），支持范围查询和距离计算	附近的人/店铺
Stream	一个支持持久化的消息队列，按时间顺序存储消息，支持消费者组	日志流、事件流

String 存储的是序列化后的整个对象数据，相比 Hash 更节省内存，缓存相同量的对象数据内存消耗约为 Hash 的一半，存储具有多层嵌套的复杂对象更加方便

但是 Hash 可以单独存储和获取对象的部分字段信息，可以方便地修改或添加部分字段，节省网络流量，适合于对象中某些字段经常变动或需要单独查询的场景

过期键删除策略：（为 key 设置了过期时间）

1. 惰性删除：只会在取出 key 的时候才对数据进行过期检查。这样对 CPU 最友好，但是可能会造成太多过期 key 没有被删除，存在内存泄露风险

2. 定期删除：每隔一段时间抽取一批 key 执行删除过期 key 操作。Redis 底层会通过限制删除操作执行的时长和频率来减少删除操作对 CPU 时间的影响。执行时会影响性能

内存淘汰策略：（redis 占用内存达到阈值，新的写入触发删除）

淘汰策略	策略描述	适用场景
noeviction 默认策略	当内存使用达到上限时，拒绝所有写入操作	需要确保数据完整性，对可用性要求相对较低的场景
volatile-lru 最近最少使用	从已设置过期时间的键中，淘汰最近最少使用的数据	缓存场景，特别是希望优先淘汰不常用缓存数据的情况
allkeys-lru 全局最近最少使用	从所有键中，淘汰最近最少使用的数据	通用的缓存场景
volatile-random 随机淘汰	从已设置过期时间的键中，随机淘汰数据	对数据淘汰顺序无严格要求的缓存场景
allkeys-random 全局随机淘汰	从所有键中，随机淘汰数据	对数据淘汰顺序无严格要求，希望均匀淘汰数据的场景
volatile-ttl 最小 TTL 优先	从已设置过期时间的键中，淘汰剩余 TTL（存活时间）最小的数据	缓存即将过期的数据，希望优先处理快过期数据的场景

对比维度	过期键删除策略	内存淘汰策略
核心目的	保证数据时效性，删除已过期的键	控制内存占用，防止 Redis 内存溢出
触发条件	键过期时间到达 / 访问时检查	Redis 内存达到 maxmemory 上限
优点	1. 保证过期数据及时清理 2. 不影响未过期键的正常使用	1. 保障 Redis 服务稳定不宕机 2. 灵活控制内存上限 3. 支持多种淘汰算法适配业务
缺点	1. 纯定时删除：CPU 消耗大 2. 纯惰性删除：内存占用可能偏高 3. 需要平衡 CPU 与内存	1. 可能删除热点 key，影响业务 2. 策略选择不当会导致缓存命中率下降 3. 额外占用少量 CPU
对业务影响	影响数据准确性，过期不删会导致脏数据	影响缓存命中率，删错 key 会击穿缓存
使用场景	所有带过期时间的缓存、会话、限时活动	高并发、内存有限、缓存容量不可控的场景

缓存失效：

1. **穿透**：请求数据库中不存在的数据，给 redis 穿透了

解决方法：

① 布隆过滤器：

在 redis 前加布隆过滤器，通过布隆过滤器检查 key。如果 key 存在查 redis/数据库，如果 key 不存在直接返回空

② 在 API 网关或业务层对请求检验：

对请求参数进行强校验，拦截明显的非法请求

③ 缓存空对象：

当数据库查询结果是 null 时，将一个短生命周期的空值写入缓存（实现简单，对应用透明，但占用缓存空间，需要设置合理的 TTL）

2. **击穿**：热点 key 过期瞬间，大量并发请求该 key。击穿了 redis 打到数据库

解决方法：

① 设置热点 key 永不过期：

TTL=-1，注意需保证数据最终的一致性

② 加互斥锁：

在 redis 和 db 之间加锁，在缓存失效后，查数据库前，获取一个分布式锁，只有获取锁的请求才能查 db。没有锁的请求可以短暂休眠再读缓存或者等待锁释放并监听缓存更新通知

3. **雪崩**：大量缓存同一时间过期或者 redis 实例宕机，请求如雪崩般打到数据库

同时过期解决方法：

① 给每个 key 的 TTL 加一个随机值，需注意随机值要合理设置

② 缓存永不过期 + 后台更新

redis 宕机解决方法：

① 熔断：当数据库访问出错率或慢查比达到阈值后，自动拒绝请求

② 降级：当数据库访问出错率或慢查比达到阈值后，返回预设值或兜底数据

③ 限流：当数据库访问出错率或慢查比达到阈值后，限制并发数量

④ 构建高可用集群：主备模式、哨兵模式、集群模式

⑤ redis 重启后，缓存预热：启动时预热，提前加载热点数据。定时任务预热，在缓存批量过期前，异步触发预热即将过期的热点数据

如何预热缓存数据？

redis 持久化

1. RDB
2. AOF
3. RDB-AOF

1. RDB：快照持久化

在特定时间点，对当前内存中整个数据库（全量数据）生成一个二进制的快照文件（.rdb 文件）

缺点：比如 3 点发出 RDB 持久化命令，下一次是 3 点半发出 RDB 持久化命令，如果在 3 点到 3 点半之间 redis 宕机这中间的数据会丢失

触发机制：

手动命令触发：

- SAVE：阻塞主进程，直到快照完成。注意生产环境一般禁用
- BGSAVE：后台异步执行（fork 子进程负责快照，主进程继续提供服务）

自动触发：在 redis.conf 中配置：

- save 900 1 #900 秒内有至少 1 次写操作则触发

工作流程：

- 父进程 fork 一个子进程（子进程拥有父进程数据副本）
- 子进程遍历内存数据，将其序列化写入临时 RDB 文件
- 子进程完成后，替换旧的 RDB 文件

优点：

- 紧凑高效：单文件压缩存储，恢复速度快
- 对性能影响小：fork 子进程操作，父进程阻塞时间短
- 适合冷备：可将 RDB 文件拷贝到异地作为灾难恢复备份

缺点：

- 可能会丢失数据：两次快照间的数据修改会丢失
- fork 操作可能耗时：数据集大时，fork 子进程可能阻塞主进程（父进程在创建子进程时会阻塞）
- 非实时持久化：宕机时可能丢失秒级甚至分钟级数据

2. AOF：日志追加持久化

记录每条写命令（如 SET，LPUSH 等），以追加方式写入文本日志文件（.aof 文件）。重启时重放命令重建数据

① 日志记录：将每个会修改 redis 数据的写命令，在命令被成功执行后，以 redis 协议格式（RESP）追加（append）到 AOF 文件（缓冲区）的末尾

② 文件同步（Fsync）：redis 会根据配置的策略（同步频率），定期或不定期的将写入到内存缓冲区的 AOF 内容真正同步到磁盘上的 AOF 文件。这一步决定了数据持久化的安全级别

③ 恢复数据：当 redis 服务重启时，redis 会读取 AOF 文件，按顺序重新执行文件中记录的每条写命令，以此来重建服务器关闭前的数据状态。这就像“回放”操作日志来恢复数据

AOF 文件写入和同步策略：

- appendfsync always：每条命令都同步（最安全，性能最低）
- appendfsync everysec：每秒同步一次（推荐值，安全性与性能平衡）
- appendfsync no：由操作系统决定（性能最佳，安全性最低）

- WRITE 命令把写 redis 命令写到 AOF 缓冲区
- FSYNC 命令把 AOF 缓冲区数据写入磁盘
- AOF 重写（BGREWRITEAOF）：AOF 文件会无限增长，重写机制创建精简版新 AOF 文件

手动命令：BGREWRITEAOF

自动配置：在 redis.conf 中配置：

- auto-aof-rewrite-percentage 100 # 当前 AOF 超过上次重写大小 100%
- auto-aof-rewrite-min-size 64mb # 且 AOF 文件超过 64MB

重写原理：不分析旧的 AOF 文件，而是 fork 一个子进程执行重写任务，子进程会遍历当前数据做快照，用最精简命令以 RESP 格式写入一个临时的更新 AOF 文件

重写期间的处理：在子进程重写的过程中，主进程会继续处理命令并将新的写命令追加到内存缓冲区（同时也会写入到旧的 AOF 文件中，以保持其最新）。当子进程完成重写，它会向主进程发送信号。主进程收到信号后，会将重写期间累计在缓冲区的新命令追加到这个新的临时 AOF 文件中，最后原子地替换旧的文件，完成重写过程

优点：

- 数据安全性高：always 策略下数据基本不丢失
- 实时性好：everysec 策略下最多丢失 1 秒数据
- 文件可读性强：通过文本直接分析操作记录

缺点：

- 文件体积大：相同数据量 AOF 文件通常大于 RDB
- 恢复速度慢：需要逐条重放命令，尤其是数据量大时
- 性能影响略大：频繁写入对 IO 有压力（可通过 fsync 策略调整）

3. RDB-AOF 混合持久化：结合 AOF 和 RDB 优势

① 正常写 AOF 文件：

Ⅰ 日志记录：将每个会修改 redis 数据的写命令，在命令被成功执行后，以 redis 协议格式（RESP）追加（append）到 AOF 文件（缓冲区）的末尾

Ⅱ 文件同步（Fsync）：redis 会根据配置的策略（同步频率），定期或不定期的将写入到内存缓冲区的 AOF 内容真正同步到磁盘上的 AOF 文件

② AOF 重写触发时：

Ⅰ fork 子进程生成一份当前内存数据的 RDB 快照(先把当前整个内存数据，以 RDB 格式写入新 AOF 文件头部)

Ⅱ 主进程接收到的所有新的写命令，不仅会写到内存中，还会同时写到 AOF 重写缓冲区中，当子进程完成 RDB 快照的写入后，子进程将 AOF 重写缓冲区中累计的所有命令以 AOF 格式追加写入到刚才生成的 RDB 数据之后的同一个临时文件中。这个临时文件包含两部分，一部分是 RDB 格式的全量数据快照，一部分是 AOF 格式的增量写命令日志

③ 文件替换：最终 AOF 文件结构：[RDB 全量数据] + [AOF 增量命令]，临时文件写入完成后，redis 会用这个新的、包含混合内容的文件原子的替换掉旧的 AOF 文件

④ 后续写入：替换完成后，所有新的写命令继续以标准的 AOF 格式追加到这个新的混合文件末尾

优点：

- 数据安全性与 RDB 恢复速度兼得
- 文件体积比纯 AOF 小，且定期清理历史操作记录
- 已成为生产环境推荐的持久化方案

如何选择合适的策略？

场景	推荐策略	原因
允许少量数据丢失，需要快速重启恢复	RDB	恢复速度最快，归档方便
数据重要性高，可接受较慢的恢复速度	AOF	减少丢失风险
追求安全性与恢复速度的平衡	RDB-AOF	redis4.0+最佳实践
停机维护时的手动备份	手动执行 SAVE 或 BGSAVE	用于灾备或迁移

如何解决单点故障？

redis 的高可用

- 核心基础：主从复制
- 故障自动转移：redis sentinel，哨兵模式
- 分区高可用与扩展：redis cluster，redis 集群

一、主从复制：

基本原理：部署一个主节点和至少一个从节点

- 主节点负责处理写操作，并将写命令异步同步给从节点，（也可以处理读请求，但不推荐）
- 从节点复制主节点的数据副本，处理读请求

作用：

- 数据冗余：提供了数据备份，主节点故障时，从节点有完整的数据（可能稍有延迟）
- 读扩展：将读请求分发到从节点，分担主节点压力
- 关键点：复制是异步的，主从间存在数据延迟，主节点故障不会自动切换

优点：

- 数据备份与冗余
- 读写分离，提升读性能
- 架构简单，部署容易缺点：

缺点：

- 无自动故障转移（高可用性差）
- 主节点写能力瓶颈
- 存储能力受单机限制

适用于数据备份、读写分离和在线升级场景。

主从同步：

1. 启动从节点时，通过在配置文件中的 replicaof 或启动后执行 REPLICAOF 命令，指定要复制的主节点地址和端口
2. 连接：从节点向主节点发起 TCP 连接，包括密码
3. 同步数据：全量同步、增量同步

连接建立并认证成功后，从节点向主节点发送 PSYNC 命令（带有 runid 和 offset）。主节点通过 runid 和 offset 判断是全量同步（参数是？0 或者 offset 参数在复制积压缓冲区中找不到）还是增量同步

全量同步：

1. 主节点回复：FULLRESYNC，通知从节点将开始全量同步
2. 主节点生成 RDB 快照：主节点启动一个后台进程（BGSAVE），将当前内存中所有数据的快照写入到一个 RDB 文件
3. 生成 RDB 期间有写操作，则把写命令写入复制缓冲区（每个从节点都有对应的单独的复制缓冲区）和复制积压缓冲区（全局）
4. 发送 RDB 文件：建立 TCP 连接传输
5. 从节点加载 RDB：从节点收到 RDB 文件后，会将其数据库清空，然后加载 RDB 文件到内存，将自身数据状态更新至生成 RDB 时主节点的状态
6. 发送复制缓冲区的写命令：主节点将复制缓冲区所有写命令发送给从节点，从节点依次执行这些命令将自身数据状态更新至生成 RDB 时主节点的状态
7. 完成全量同步：此时从节点与主节点在完成 RDB 传输时刻的数据完全一致。主节点的角色状态转变为“在线从节点”，开始后续的增量同步过程

增量同步：

1. 主节点记录写命令：将写命令按协议格式追加到所有从节点连接的复制缓冲区中，并发送给所有已连接的从节点。同时将该命令写入复制积压缓冲区（所有从节点共享），并保存其对应的偏移量（offset）
2. 从节点执行命令：
   - 从节点接收主节点传播过来的写命令
   - 从节点在自身的数据集上顺序执行这些命令
   - 从节点也会更新自身的复制偏移量（offset），记录已处理的数据量

断线重连：时间很短，不必要进行全量同步。只需要进行部分重同步

1. 从节点发送 PSYNC 请求：runid，offset
2. 主节点判断：
   - runid 是否匹配当前主节点的 runid
   - 从节点的 offset 是否在主节点的复制积压缓冲区内
3. 执行部分重同步：

   主节点把从节点 offset 之后在复制积压缓冲区中的命令发送给从节点。从节点执行命令，同步到主节点当前状态
4. 退化到全量同步：

   如果 runid 不匹配，或者 offset 不在复制积压缓冲区内，则主节点会触发一次全量同步

- runid：每个 redis 实例启动的唯一 ID，用于标识特定的主节点。主节点重启后 runid 会改变
- offset（复制偏移量）：记录发送了多少数据和接受处理了多少数据，用于判断数据一致性成都和部分重同步

二、哨兵模式：

基本原理：Sentinel 是一个分布式系统，由多个 sentinel 进程组成（奇数个，3 或 5），sentinel 进程独立于 redis 服务运行

优点：

- 自动故障转移（高可用）
- 主从模式的无缝升级
- 监控、通知、配置提供者

缺点：

- 写能力和存储能力未解决
- 配置稍复杂
- 故障转移期间数据可能丢失

故障转移流程：

1. 监控阶段：持续健康检查
2. 故障检测：主观下线与客观下线
3. 领导者选举：谁来执行故障转移
4. 故障转移：选择新主节点并切换
5. 配置传播：通知集群与客户端

1. 监控阶段：持续健康检查

周期性 PING 检测：

- 每个哨兵节点每隔 1 秒向所有被检控的主节点、从节点及其他哨兵节点发送 PING 命令
- 有效响应（如 PING、LOADING 等）表示节点在线
- 无效响应或无响应表示节点可能故障

2. 故障检测：主观下线与客观下线

① 主观下线：

- 当一个哨兵节点在配置的时间内，没有收到主节点的有效响应，则将其标记为主观下线

② 客观下线：

- 该哨兵节点向其他哨兵节点发起命令 SENTINEL is -master-down-by-addr 询问主节点状态
- 当超过 quarum 数量（可设置，一般为 N/2 + 1）的哨兵节点均认为主节点主观下线时，主节点被标记为客观下线
- 关键机制：避免因网络抖动导致误判，需多个哨兵达成共识

3. 领导者选举：谁来执行故障转移

① 选题条件：

- 发起选举的哨兵节点需满足：最先发现客观下线，与多数节点保持连接

② raft 算法实现：

- 每个哨兵会为当前纪元（epoch）发起投票请求
- 采用先到先得原则：最先收到投票请求的哨兵会同意该请求
- 只有获得超过半数投票的哨兵节点会成为领导者

4. 故障转移：选择新主节点并切换

① 筛选新主节点：

- 根据从节点优先级→复制偏移量→运行时长等标准选择最优从节点

② 执行晋升操作：

- 向选中的从节点发送 SLAVEOF NO ONE 命令，将其提升为主节点
- 等待新主节点完成角色切换

③ 更新从节点配置：

- 向其他所有从节点发送 SLAVEOF new_master_ip new_master_port，使他们开始复制新主节点

④ 旧主节点处理：

- 若旧主节点恢复，会将其设置为新主节点的从节点（通过 SLAVEOF 命令）

5. 配置传播：通知集群与客户端

① 更新自身配置：

- 哨兵节点将新的主节点信息持久化到本地配置文件

② 同步其他哨兵节点：

- 通过发布订阅机制通知其他哨兵节点更新配置

③ 客户端服务发现：

- 客户端通过 sentinel api 获取主节点信息
- 主节点变更时，哨兵节点通过 pub/sub 频道通知所有订阅的客户端

补充：

Raft 算法核心：

节点三种角色

- Follower：追随者，被动接受请求（Sentinel 节点）
- Candidate：候选人，参与竞选
- Leader：领导者，处理写请求、同步日志（主节点）

选举触发

- Follower 一段时间没收到 Leader 心跳（最先没有收到的那个） → 变成 Candidate

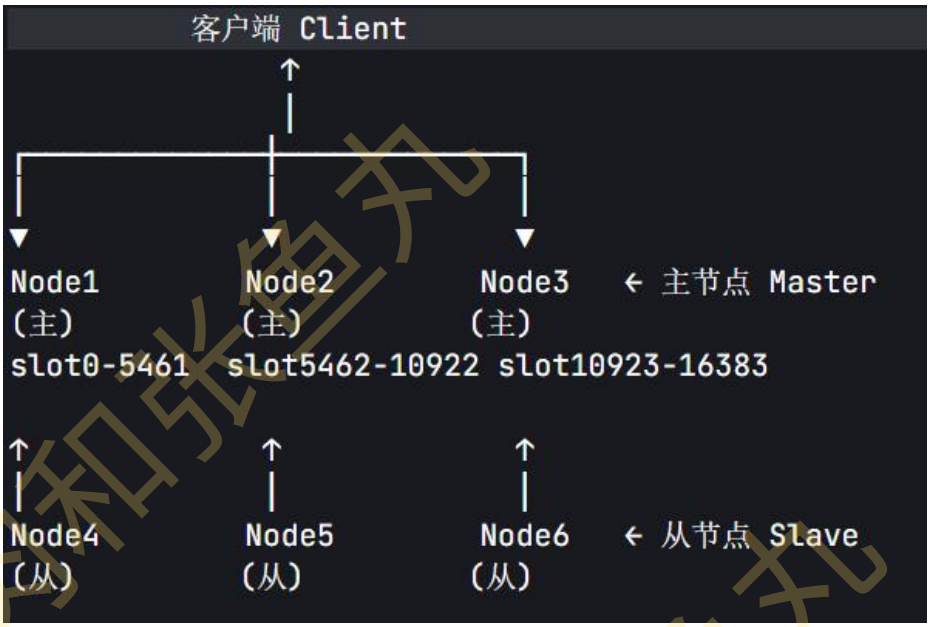
选举规则（最关键）

- Candidate 先投自己一票
- 让其他 Follower 投我（Candidate）一票，谁申请的就投谁
- 谁先拿到 N/2 + 1 票，谁直接当 Leader
- 同一任期内只有一个 Leader

一句话记：先超时先参选，谁过半谁当 Leader，同一任期只认一个

三、集群模式

redis 集群是 redis 官方提供的分布式解决方案，允许你将数据自动分片存储到多个 redis 节点上，并提供高可用性。他的实际目标时保证一定程度高性能的同时，满足大规模数据存储的需求和高可用性需求



优点：

- 数据分片，解决写压力和存储瓶颈
- 高可用性（内置哨兵功能）
- 可线性扩展

缺点：

- 架构复杂，运维要求高
- 客户端实现更复杂
- 不支持跨节点事务和多键操作

工作流程：

1. 初始化集群：
   - 多个 redis 节点启动，每个节点以单机模式运行
   - 使用 CLUSTER MEET ip port 命令，让节点互相认识别组成集群网络
   - 初始创建时，通常使用 redis-cli --cluster create 工具自动完成节点加入，槽分配和主从关系绑定
2. 槽分配：新来的数据从哪台机器取？如果是简单的 hash 然后取模，在增加新节点后，需要所有节点都重新分配，所以引入了槽的概念
   - 集群自动将 16384 个槽平均分配给所有主节点，比如 0～5460，5461～10922，10923～16383
   - 每个节点记录一份完整的槽映射表（slot map，槽 X 由节点 Y 负责）
   - 槽映射表通过集群总线在节点间用 gossip 协议传播和同步
3. 主从关系建立：
   - 从节点使用 CLUSTER REPLICATE master-node-ip 绑定到主节点
   - 主节点开始向从节点异步复制数据

客户端请求处理与路由：

1. 客户端本地缓存 slot map：
   - 客户端启动时，会向任一集群节点发送 CLUSTER SLOTS 命令，获取集群的槽映射关系，并缓存到本地
2. 计算 key 的槽：
   - 客户端使用 CRC16（key）mod/16384 计算该键属于哪个槽
3. 根据 slot map 连接目标节点：
   - 客户端根据本地缓存的 slot map，直接连接负责该槽的主节点发起请求
4. 节点处理请求：

① 请求落在正确节点

- 节点正常执行命令并返回结果

② 请求落在错误节点 - MOVED 重定向（因为客户端的 slot map 是被动更新，落在错误节点后才会更新）

- 节点发现槽不是自己负责
- 节点向客户端返回 -MOVED slot node-ip:node-port 错误响应
- 客户端更新本地 slot map，根据返回的节点信息重新连接目标，重新发送请求

故障检测与转移（高可用核心流程）

当一个主节点出现故障时：

1. 主观下线（PFAIL）：
   - 节点 A 通过集群总线定期向节点 B 发送 PING
   - 如果节点 A 在配置的超时时间内未收到节点 B 的 PONG 响应，则节点 A 在本地将节点 B 标记为 PFAIL
2. gossip 传播和客观下线（FAIL）：
   - 节点 A 在与其他节点交换 gossip 消息中，包含他认为主观下线的节点列表
   - 其他节点收到消息后，在自己的视角下更新节点 B 的状态
   - 如果一个主节点被集群超过（N/2 + 1）标记为主观下线，则该节点状态被升级为客观下线
   - 集群会广播一条客观下线消息，所有收到消息的节点立即将节点 B 标记为客观下线

故障转移（Failover）：

- 当 B 节点被判定为客观下线：
  - 他的从节点检测到主节点客观下线状态并开始竞争成为新主
  - 从节点增加当前纪元（currentEpoch）并广播

主节点投票：

- 发起请求的从节点如果在（N/2 + 1）张有效投票中赢得多数票，他就会赢得选举

角色切换：

- 赢得选举的从节点执行 SLAVEOF no one，将自己提升为新主节点
- 他接管其旧主节点负责的所有哈希槽
- 他会增加配置纪元（configEpoch）并广播一条 PONG 消息，宣布其新角色和槽信息
- 集群中的其他节点接收到该信息，会更新他们的 slot map 和节点状态

槽分配：

- 一共 16384 个固定槽，一个槽就是一个放数据的抽屉，一个主节点管一些抽屉
- 节点 A：槽 0～5460、节点 B：槽 5461～10922、节点 C：槽 10923～16383
- 数据来了，slot = CRC16(key) & 16383，比如得 1000，那么就放到 A 管的抽屉

槽带来的好处：

- 定位超级快算一下槽号 → 直接找到节点不用遍历、不用猜
- 扩缩容非常方便加节点 = 从老节点挪一部分槽过去删节点 = 把槽挪给其他节点整个过程可以在线、不停机
- 不会出现数据乱飘槽和节点的对应关系，整个集群统一知道

主节点扩容（增加主节点）：

- 假设原来有 3 主节点：A：0～5460、B：5461～10922、C：10923～16383
- 加了一个主节点 D，从 A、B、C 中各给 D 一些槽，依旧是平均分的
- A：1365～5460、B：6827～10922、C：12289～16383、D：0～1364、5461～6826、10923～12288

主节点缩容（减少主节点）：

- 假设原来有 3 主节点：A：0～5460、B：5461～10922、C：10923～16383
- C 要下线，把 C 的所有槽全部迁移走，平均分给 A、B
- A：0～5460 + 一部分 C 的槽、B：5461～10922 + 剩下 C 的槽、C：空（然后被踢出集群）

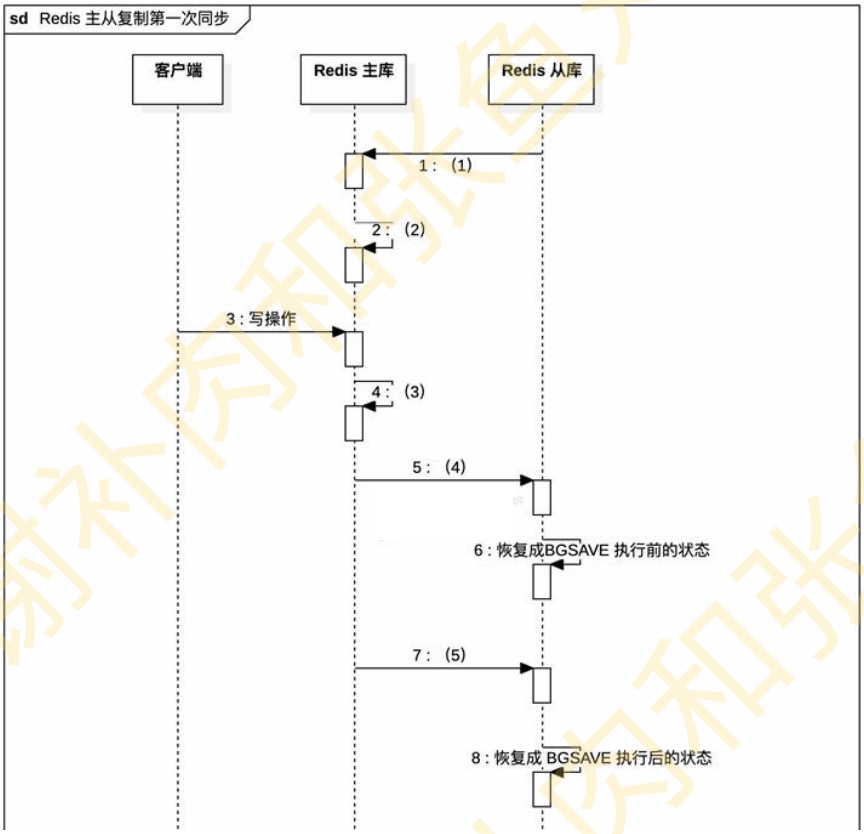
redis 指令：

集合（Sets）	**SADD**keymember[member..]：向集合添加一个或多个成员 **SREM**keymember[member..]：从集合中删除一个或多个成员 **SPOP**key：从集合中随机移除并返回一个成员 **SCARD**key：获取集合的成员数
有序集合（SortedSets）	**ZADD**keyscoremember[scoremember..]：向有序集合添加一个或多个成员，每个成员关联一个分数 **ZREM**keymember[member..]：从有序集合中删除一个或多个成员 **ZRANGE**keystartstop[WITHSCORES]：获取有序集合中指定分数范围的成员
散列（Hashes）	**HSET**keyfieldvalue：在散列中设置字段的值 **HGET**keyfield：从散列中获取字段的值 **HGETALL**key：获取散列中的所有字段和值
键管理	**KEYS**pattern：查找所有匹配给定模式的键 **DEL**key[key..]：删除一个或多个键 **EXPIRE**keyseconds：为键设置生存时间
事务	**MULTI**：开始一个事务块 **EXEC**：执行事务块中的所有命令 **WATCH**key[key..]：监视一个或多个键，如果在执行事务期间这些键被修改，则事务将失败
持久化	**SAVE**：将数据同步到磁盘 **BGSAVE**：在后台异步保存数据到磁盘

真题：

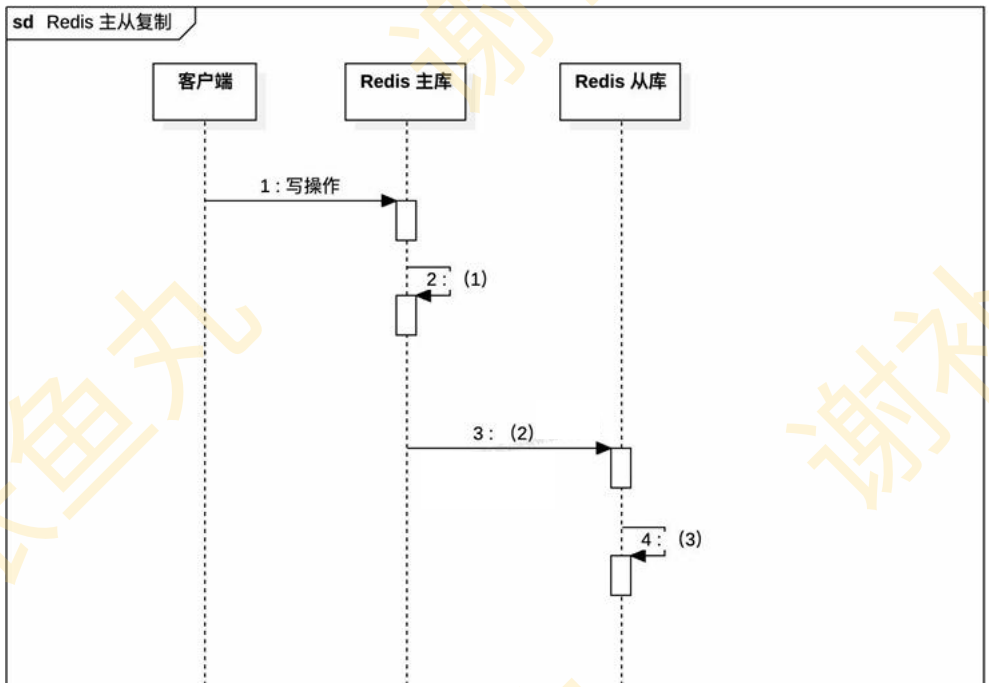
（2025 年 5 月）某互联网企业开发并部署了一套高并发电商系统，系统中的商品信息、用户购物车、热销榜单等高频数据均存储在 Redis 中以提高访问效率。随着业务规模扩大，单实例 Redis 出现了性能瓶颈和单点故障风险。为提升系统的可用性、扩展性和数据容灾能力，技术团队决定采用 Redis 的主从复制架构，将写操作集中在主节点（Master），将读操作分散到多个从节点（Slave）中。同时，系统还需支持主从自动同步机制，确保在主库故障后可以快速切换至从库，保障业务连续性。

1. 请根据 Redis 的主从架构原理和下图的同步流程，在图示中填入正确的关键步骤和指令。



1. 从库向主库发送PSYNC？0进行首次同步
2. 主库执行 BGSAVE 命令，后台生成快照 RDB 文件
3. 生成 RDB 快照期间的写命令，主库将写命令序列化后存在复制缓冲区（replication buffer）和复制积压缓冲区（replication backlog）
4. 主库将生成的 RDB 文件发送给从库
5. 主库将复制缓冲区内的写操作发送给从库，此时即完成全量同步

2. 主从库第一次同步完成之后，后面是如何同步的，请在图示中填入正确的关键步骤和指令



1. 主库会将写命令序列化后存到复制缓冲区和复制积压缓冲区
2. 主库将复制缓冲区中的写命令发送给从库
3. 从库按顺序执行主库发来的写命令，使其与主库数据保持一致

3.Redis 的数据持久化机制在生产环境中至关重要，当系统遭遇故障时需依赖持久化数据进行快速恢复。请详细阐述 Redis 的两种主要持久化方式，并对比分析其优缺点

Redis 主要有两种数据持久化方式：RDB（快照持久化）和 AOF（追加日志）

RDB 是定时将内存中的全量数据快照写入磁盘的二进制 RDB 文件。其优点是占用内存小，恢复数据快；缺点是实时性差、两次快照之间的数据有丢失风险，数据量大时候 fork 进程会阻塞。适合冷启动和灾备。

AOF 是记录每个写命令 到磁盘日志文件，重启时重新执行日志恢复数据。其优点是数据可靠性高，实时性高；缺点是文件体积大，恢复速度慢，写入性能开销大。适用于对数据安全性要求高的场景。

补充知识：

特性	复制缓冲区 (Replication Buffer)	复制积压缓冲区 (Replication Backlog)
作用范围	每个从节点私有（独占）	所有从节点共享（全局）
主要目的	暂存待发送给特定从节点的命令，支持异步网络传输	暂存最近传播过的命令历史，支持部分重同步
数据结构	线性缓冲区（本质是网络发送缓冲区）	**固定大小的环形缓冲区（FIFO，覆盖旧数据）**
关键风险	**溢出导致连接断开 -> 可能触发全量同步**	**历史数据被覆盖 -> 部分重同步失败 -> 触发全量同步**
生命周期	随主从连接建立创建，随连接断开释放	随复制启用（首个从连接）创建，无连接且超时后释放
配置相关	`client-output-buffer-limit replica ...`	`repl-backlog-size`, `repl-backlog-ttl`
核心价值	**保证异步传输的吞吐量和连接平滑性**	**保障高可用的关键（避免因短暂中断导致全量同步）**

简单来说：

- 复制缓冲区：全量同步 + 正常在线实时增量同步
  特点：从节点专属、临时存在、连接断开就销毁
- 复制积压缓冲区：从节点断线 → 短时间重连后的增量恢复同步
  特点：全局共用、永久存在、环形覆盖

（2024 年 11 月）、某电商平台使用缓存来存储热点商品数据。旁路缓存模式是常见的一种缓存模式，其核心是将缓存模块设置为 I/O 路径的一个非强制性组成部分。这意味着缓存模块可以根据需要灵活地加入或移除，而不会影响原有的 I/O 路径的完整性。

补充知识：

缓存更新策略，两大类：更新、删除

- 更新：
  1. 先更新缓存，后更新数据库
  2. 先更新数据库，后更新缓存
- 删除：
  1. 先删除缓存，后删除数据库
  2. 先删除数据库，后删除缓存

- **旁路缓存模式：先更新数据库，再让缓存失效。数据不一致时间窗口更短**

三大核心原则：

1. 懒加载：仅缓存未命中时从 db 加载
2. 写优先 db：先更新数据源再处理缓存
3. 删除缓存非更新：写操作后删除缓存而非更新

- 读流程：

1. 应用先读缓存
2. 缓存命中 → 直接返回
3. 缓存不命中 → 去读数据库 → 写入缓存 → 返回给应用

- 写流程：

1. 应用写数据库
2. 删除缓存

优点	缺点
性能较好，适合读多写少场景	业务代码需要维护缓存逻辑，侵入性较强
缓存与数据库解耦，灵活性高	并发场景下可能出现数据不一致
成熟稳定，业界应用最广泛	需要自行处理：缓存穿透、击穿、雪崩

- **读写穿透模式：应用只访问缓存，缓存自己同步读写数据库，应用完全不碰 DB**

- 读流程：

1. 应用读数据 → 只找缓存
2. 缓存命中 → 直接返回
3. 缓存不命中 → 缓存自己去 DB 加载数据 → 写入缓存 → 返回给应用

- 写流程：

1. 应用写数据 → 只写缓存
2. 缓存同步、立即把数据写到 DB
3. 缓存和 DB 都写完，才返回成功给应用

优点	缺点
强一致性：缓存与 DB 始终一致	写性能差：每次写都要同步落库
同步写：写操作必须等 DB 写完才返回	高并发下吞吐量低
应用代码最简单：只操作缓存	不适合写多读少场景

• **异步写回模式：写操作只写缓存就返回，数据异步、批量、延迟写到 DB**

• 读流程：

1. 应用读数据 → 只找缓存
2. 缓存命中 → 直接返回
3. 缓存不命中 → 缓存自己去 DB 加载数据 → 写入缓存 → 返回给应用

• 写流程（重点）：

1. 应用写数据 → 只写缓存
2. 立即返回成功（不用等 DB）
3. 后台线程异步、批量、定时把缓存数据刷到 DB
4. DB 感知不到前端压力

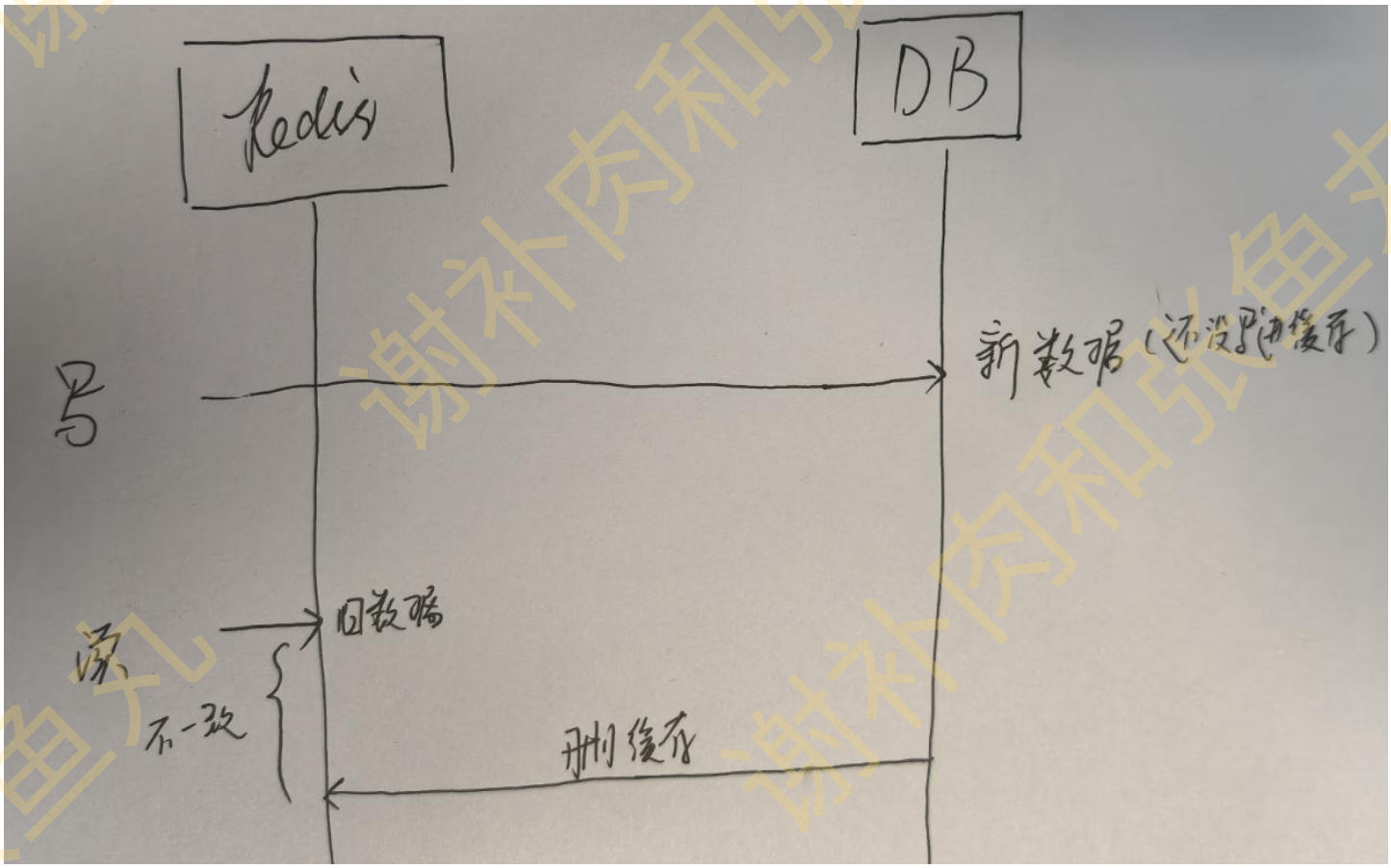
优点	缺点
写性能极高，抗高并发	数据不是强一致
合并请求，降低 DB 压力	缓存宕机可能丢数据
秒杀、日志、埋点，高写入场景	实现复杂，需要异步、重试、兜底

对比维度	旁路缓存	读写穿透	异步写回
优点	1. 实现简单、轻量 2. 性能良好、灵活 3. 业界最成熟通用	1. 业务代码简单，只操作缓存 2. 缓存与 DB 强一致 3. 不用关心同步逻辑	1. 写性能极高 2. 异步刷库，DB 压力小 3. 抗并发峰值强
缺点	1. 代码侵入，需手动维护 2. 并发下有不一致风险 3. 要处理缓存三大问题	1. 同步写库，写性能差 2. 依赖中间件，复杂度高 3. 实际场景少	1. 数据暂时不一致 2. 缓存宕机有丢数据风险 3. 实现最复杂
一致性	最终一致性	强一致性	最终一致性（有窗口期）
适用场景	读多写少、互联网业务电商、后台、Redis 主流	强一致性需求内部系统、简化业务层	高并发写、写多读少日志、统计、计数

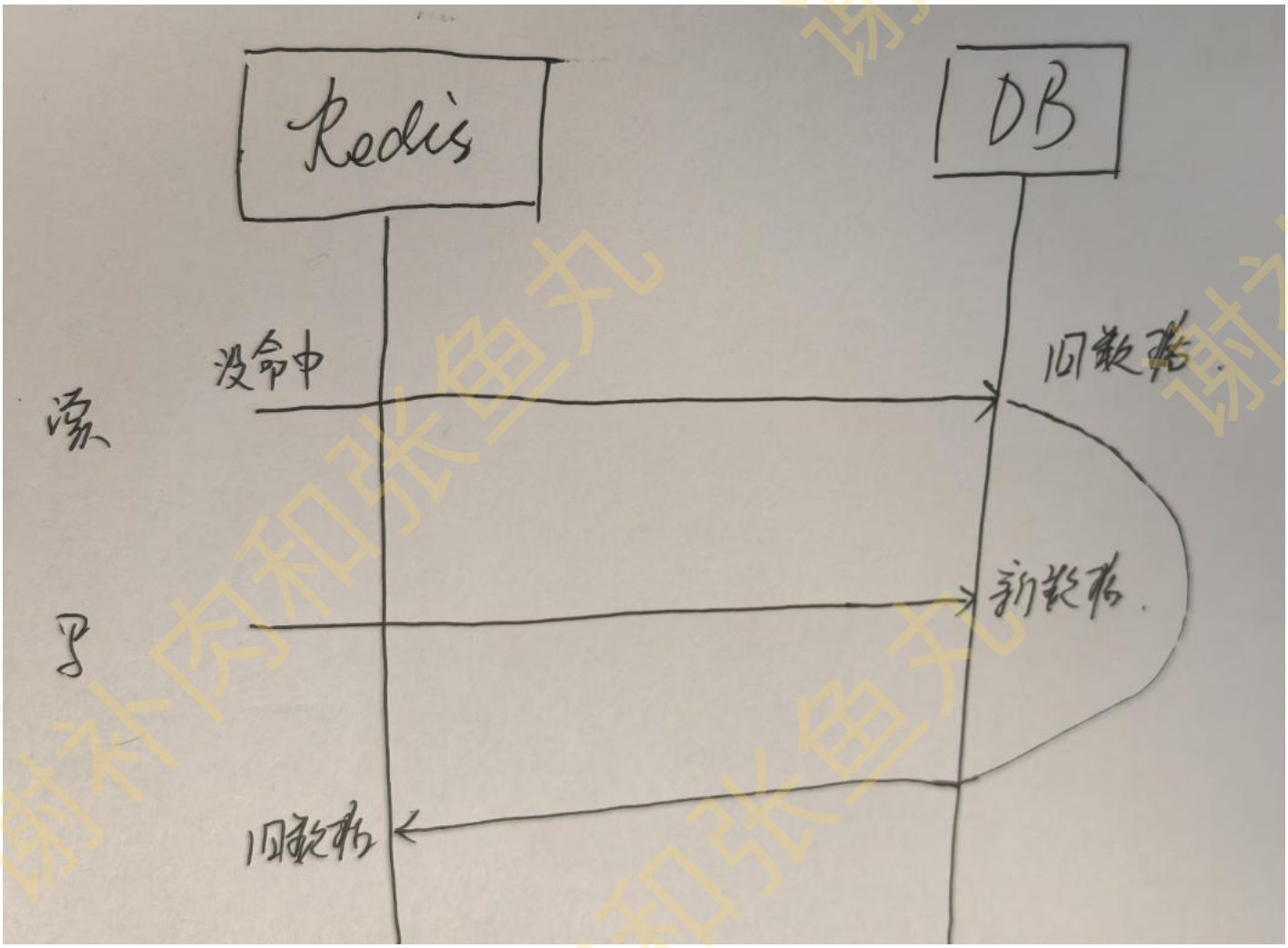
旁路缓存数据不一致的原因及解决方案：

不一致原因	具体场景描述	解决方案	适用性与说明
1. 并发读写 - 脏读旧缓存	写线程更新DB后，**删除缓存前**，读线程命中旧缓存。	**最终一致性兜底。** **串行化**：对同一Key的读写加锁（牺牲性能）。	简单有效，基础必备方案。不一致窗口期短（最终一致）。
2. 并发读写 - 先删后写旧值回填	读线程未命中缓存后，写线程**更新DB + 删除缓存**，读线程再将**旧值写入缓存**。	**缓存TTL**：兜底旧数据自动过期。 **延迟双删**：写线程更新DB并删除缓存后，等待一段时间（如500ms）**再次删除缓存。** **串行化**：对同一Key的读写加锁（牺牲性能）。	TTL必选；延迟双删需评估业务耗时；串行化仅限强一致性场景。
3. 写操作失败	DB更新成功，缓存删除失败 → 缓存残留旧数据。 	**重试机制**：删除失败时，通过内存队列或消息队列（如Kafka）异步重试删除。 **事务消息**：将DB更新与缓存删除绑定为最终一致性事务（如RocketMQ事务消息）。 **监听Binlog**：通过Canal/Debezium监听DB变更，自动删除缓存（解耦应用）。	重试需设上限和退避策略；Binlog方案适合大型系统，但引入复杂度。

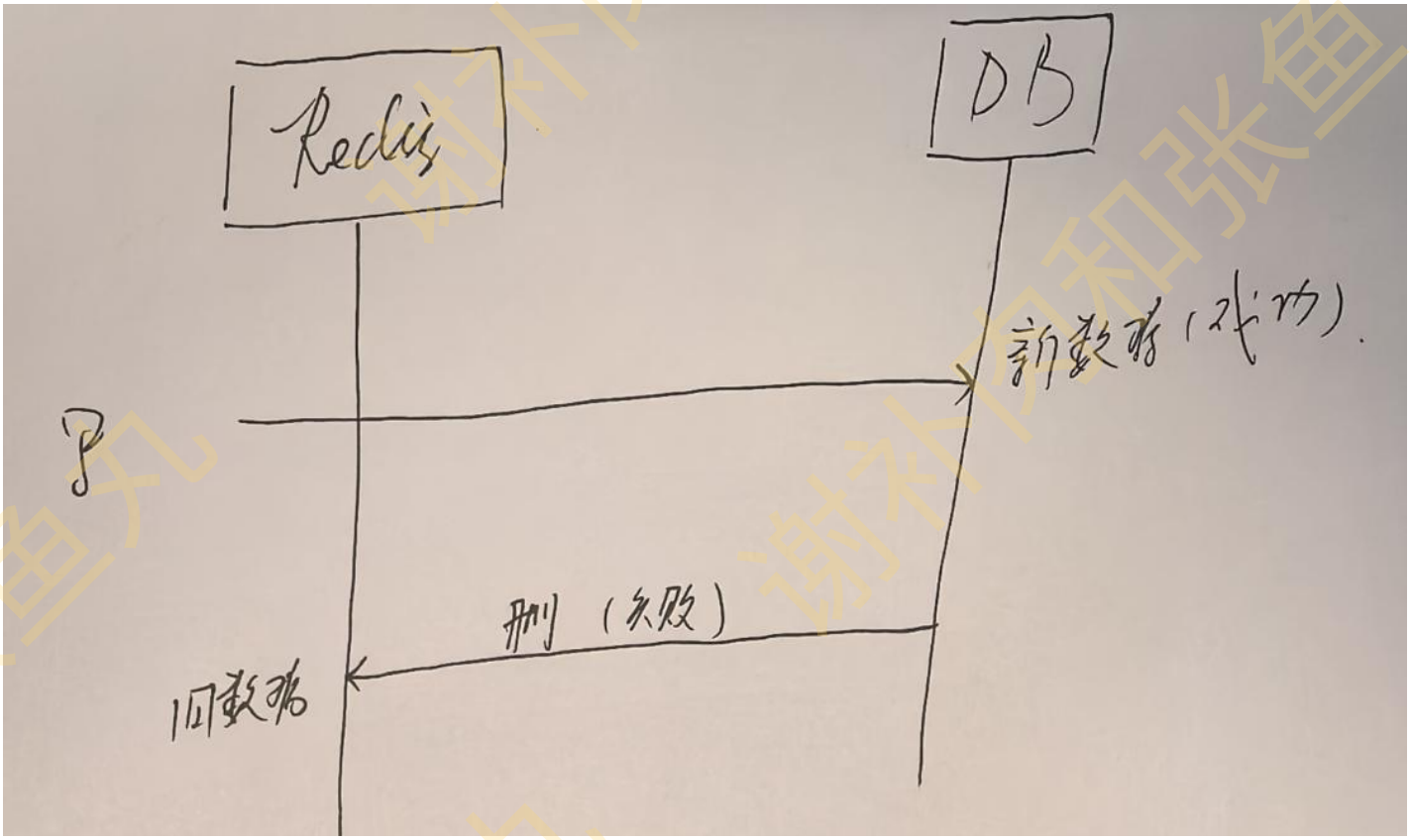
1. 并发读写，读旧数据：



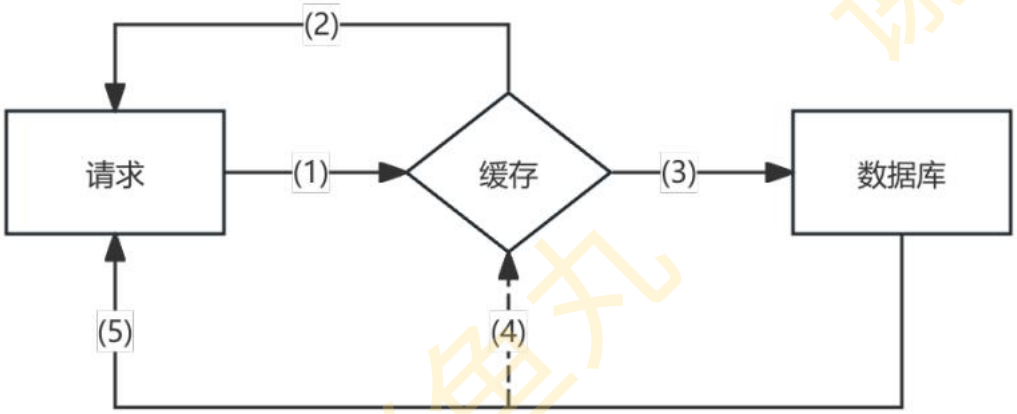
2. 并发读写，先删后写，数据回填：



3. 写操作失败：



1.根据旁路缓存模式的读数据原理，填写下图中（1）到（5）的空白。



1. 从缓存中读取商品数据
2. 命中缓存中的商品信息则直接返回数据
3. 没有命中则从数据库中查询商品数据
4. 将从数据库中查询得到的数据更新到缓存中
5. 将从数据库中查询到的数据返回给客户端

2.根据旁路缓存模式写数据原理，填写下图中（1）（2）的空白。



1. 更新数据库目标商品数据
2. 从缓存中删除目标商品数据

3.王工使用了多线程技术进行缓存处理，线程 1 负责写入，线程 2 负责读取，可能存在数据不一致性问题，请解释其原因，并给出 3 个以上的解决办法

①并发读写脏读旧缓存，在写进程更新数据库后删除缓存之前，读进程读取缓存命中，读到了旧数据。

- 解决方案：加锁保证读写进程保持串行。

②读进程读缓存未命中继续读取数据库命中后，在回写缓存前，写进程写如数据库新值，删除缓存。之后读进程将此前旧数据回写到缓存当中。

- 解决方案：1.合理设置缓存过期时间，保证最终一致。2.延迟双删，写进程在更新数据库删除缓存后，等待一段时间在删除一次缓存。3.加锁保证读写进程保持串行。

③写操作失败，更新数据库后删除缓存失败，或者更新数据库失败，缓存继续执行缓存空对象。

- 解决方案：重试机制，通过消息队列异步重试；事务消息，通过事务保证更新数据库和删除缓存是原子操作；监听 Binlog 变更自动删除缓存。

（2024 年 5 月）在分布式系统中，由于多个节点同时操作共享资源，容易产生竞争条件和资源冲突问题。为了保证数据一致性和系统的稳定性，需要一种有效的分布式锁机制来协调各节点对共享资源的访问。为了简化分布式锁的实现并提高性能，张工提出了一种利用数据库主键索引的方法来实现分布式锁。该方法的核心思想是利用数据库的唯一性约束和事务特性，通过对特定记录的插入和删除操作来实现锁的获取和释放。

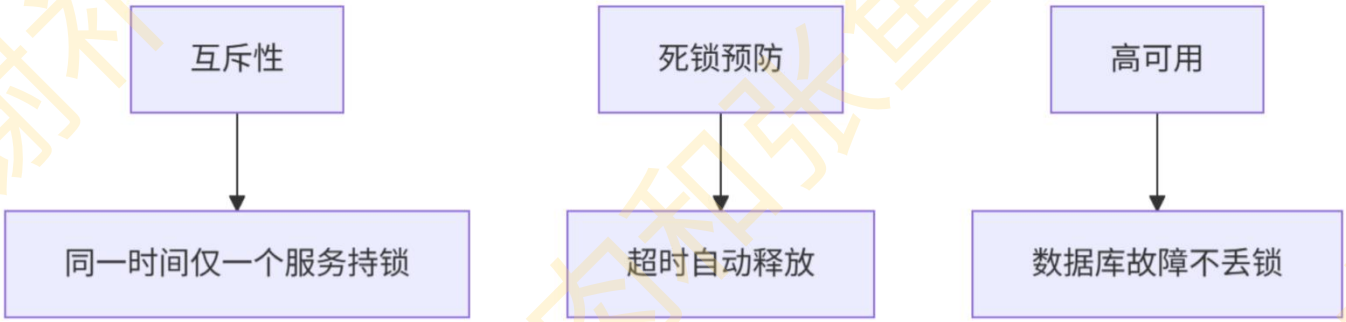
补充知识：

分布式锁是什么？

作用：解决分布式环境下的资源竞争问题

在传统的单机应用中，使用线程锁（如 Java 的 synchronized 或 ReentrantLock）就能轻松保证同一时刻只有一个线程操作共享变量。但在分布式系统中，应用程序部署在多台服务器上，这些进程运行在物理隔离的机器上，共享内存不再存在，传统锁就失效了。这时就需要分布式锁来协调分布在各个节点上的操作

分布式锁需要满足条件



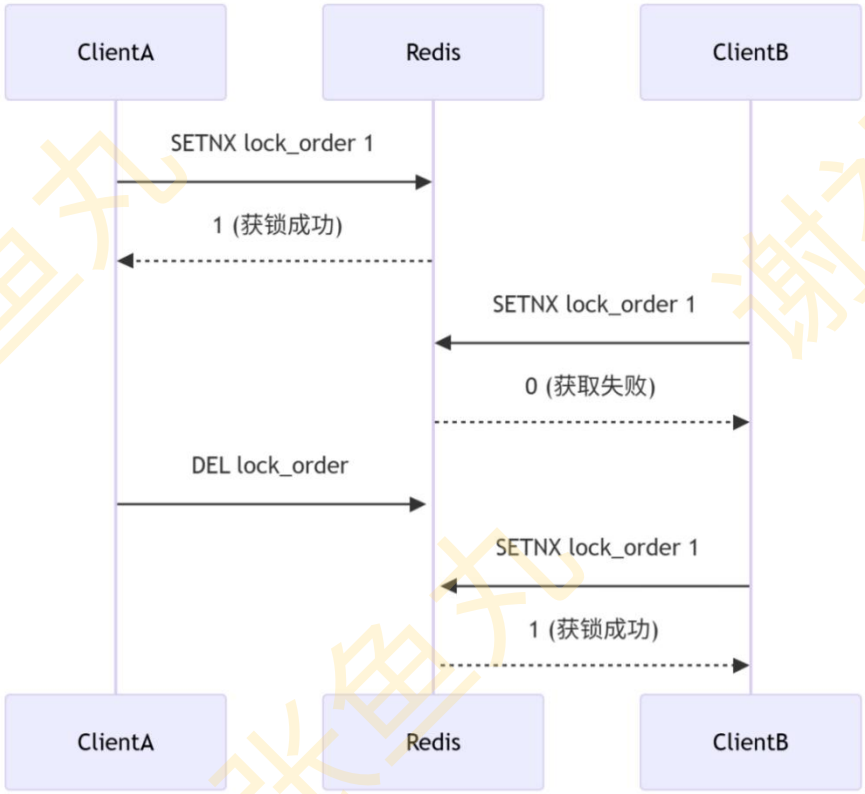
一、基于 Redis 实现分布式锁

阶段 1：基础 SETNX 锁

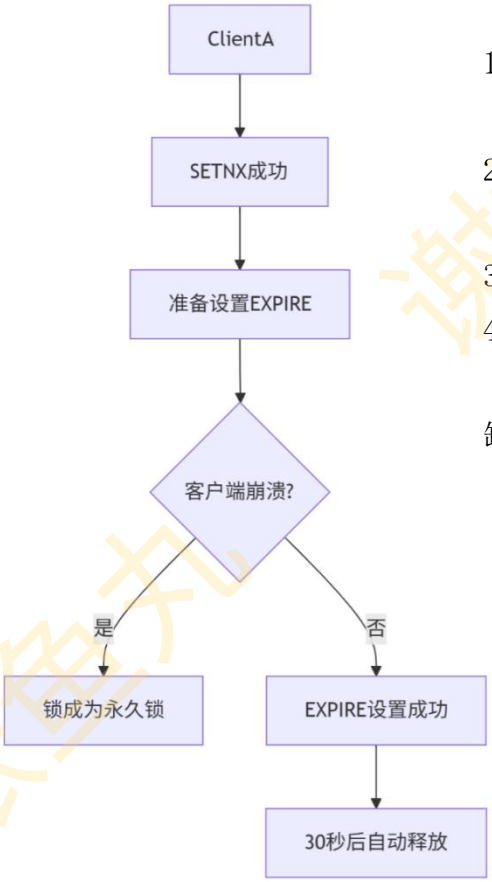
简单的互斥锁机制，SETNX 命令 - 只有当 set 的锁不存在时，才能 set 成功

1. 执行 SETNX lock_key client_id，
   若返回 1 则抢锁成功；
   返回 0 则抢锁失败
2. 成功持有锁的节点操作共享资源
   （如修改库存、写入数据）
3. 业务完成后，执行 DEL lock_key
   主动释放锁

缺点：clientA 崩溃时锁永不释放



阶段 2：SETNX+EXPIRE，给锁加过期时间



1. 执行 SETNX lock_key client_id，若返回 1 则抢锁成功；返回 0 则抢锁失败
2. 抢锁成功后，执行 EXPIRE lock_key expire_time（如 10s），避免死锁
3. 成功持有锁的节点操作共享资源（如修改库存、写入数据）
4. 业务完成后，执行 DEL lock_key 主动释放锁

缺点：在设置过期时间之前 clientA 崩溃，依旧永不释放

阶段 3：原子操作锁（SET NX EX）

1. 客户端发起命令：SET lock_key client_id NX EX 10

Redis 服务器原子性执行以下逻辑：

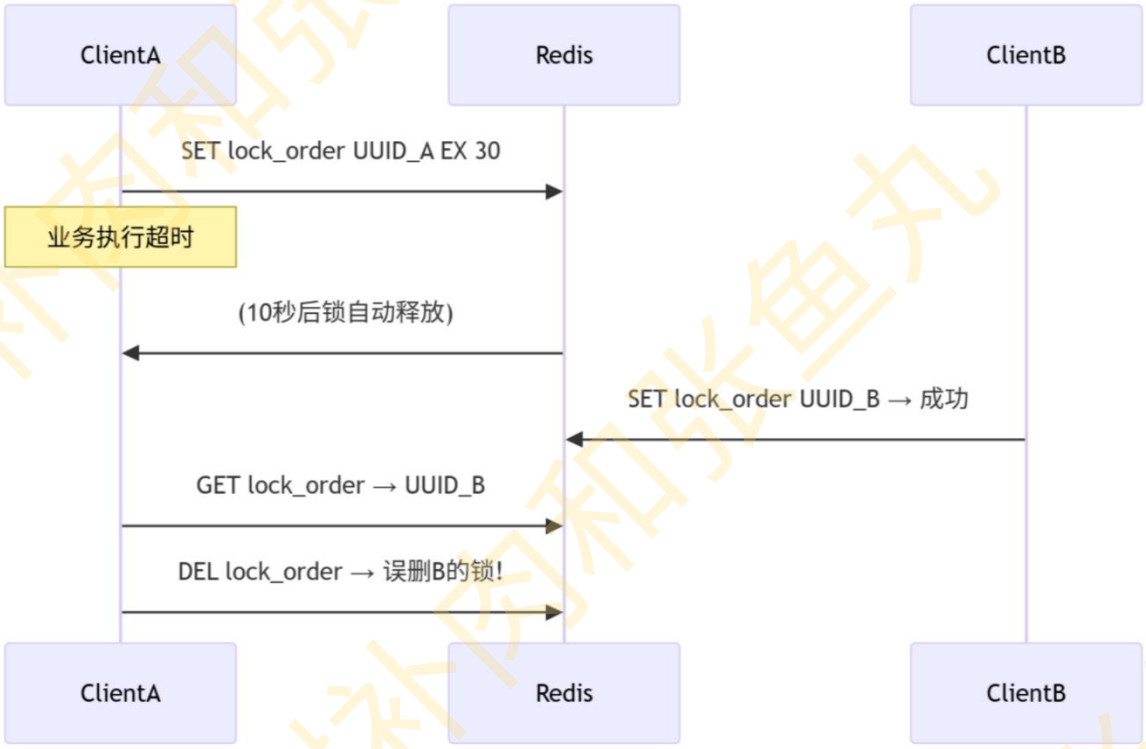
1）检查 lock_key 是否存在；

2）若不存在 → 设置 lock_key = client_id，并设置过期时间 10 秒 → 返回 OK（抢锁成功）；

3）若已存在 → 直接返回 nil（抢锁失败）；

2. 抢锁成功后：客户端执行业务逻辑（如操作共享资源、修改库存等）；此时其他客户端执行相同命令会返回 nil，无法获取锁，保证互斥性

3. 业务执行完成：客户端主动释放锁：先校验 lock_key 的值是否为自己的 client_id，再执行 DEL lock_key（避免误删其他客户端的锁）



存在问题：当 A 业务执行超时时

1. A 删 B 的锁
2. 业务流程没有执行完，锁就被释放

阶段 4：解决 A 删 B 的锁问题，防误删锁（唯一标识），通过 lua 脚本验证客户端唯一标识

1. lua 脚本执行 GET 命令，获取当前锁 key 对应的 value（即客户端唯一标识）
2. 判断这个 value 是否等于当前要释放锁的客户端 ID

   若相等：说明锁是自己的，执行 DEL 命令删除锁，返回「成功」

   若不相等：说明锁已经不属于自己（比如过期后被其他客户端抢占），直接返回「失败」，不执行任何删除操作
3. 客户端发送 Lua 脚本到 Redis

   客户端不是单独发 GET/DEL 命令，而是把整个 Lua 脚本作为「一个请求」发送给 Redis 服务器
4. Redis 原子执行脚本

   Redis 收到脚本请求后，会进入「原子执行模式」：

   锁定当前 Redis 实例的命令处理流程，不再处理其他任何客户端的命令

   按脚本内的逻辑，依次执行「查→判→删」

   整个过程中，锁的归属不会发生任何变化（因为其他命令都被暂停了）

   执行完脚本后，解锁并继续处理其他命令。
5. Redis 返回执行结果

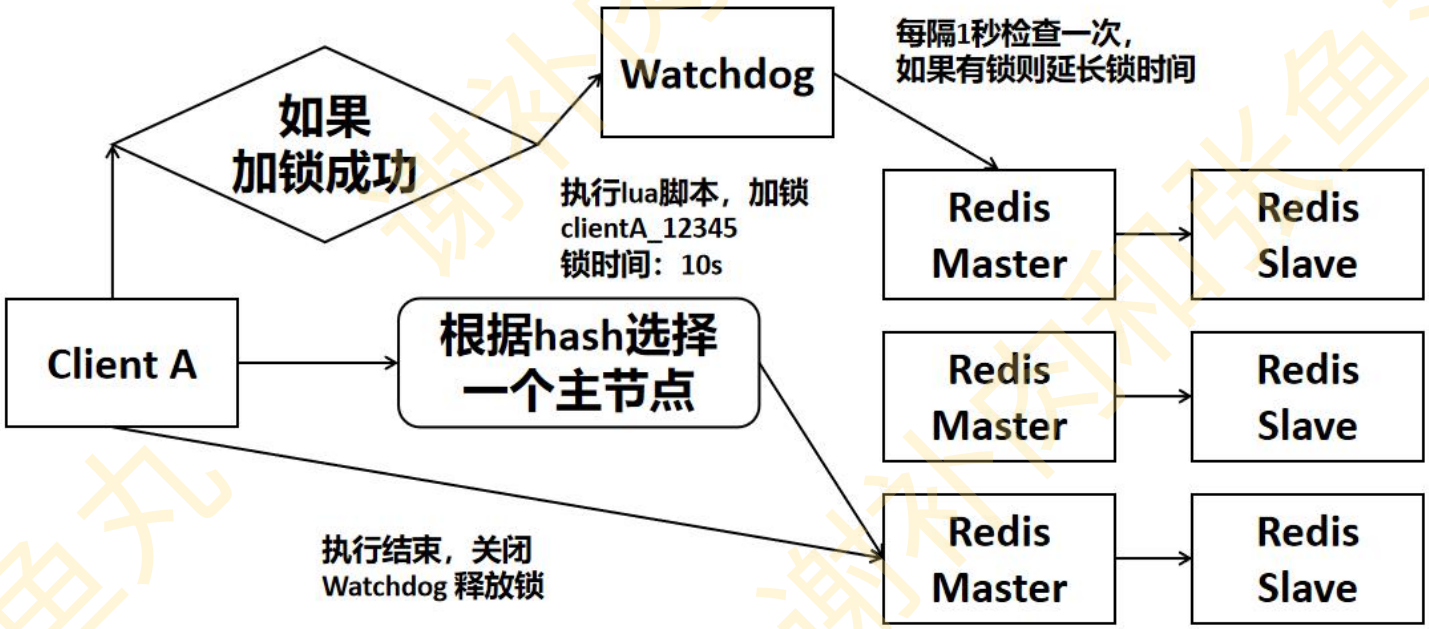
   客户端收到结果后，就知道自己是否成功释放了锁：

   若返回「成功」：说明锁是自己的，且已被删除

   若返回「失败」：说明锁已不属于自己，没有执行删除，避免了误删

阶段 5：解决业务流程没有执行完，锁就被释放：自动续期

Watchdog - Redisson - Java Redis 客户端



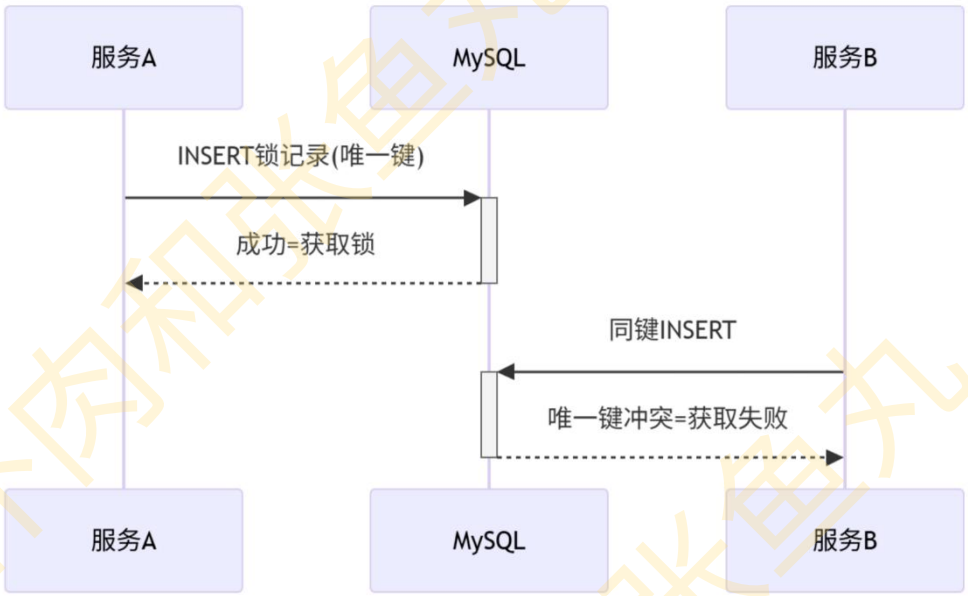
二、基于数据库分布式锁

方案 1：基于唯一键约束

1. 建一张“锁表”
2. 多个客户端同时想抢同一个锁（比如操作 “库存_1001”），就往这张表里插数据：

INSERT INTO 锁表(lock_key, client_id) VALUES('库存_1001', 'A')，lock_key 是唯一键

3. 释放锁，删除数据



唯一性索引：

优点	缺点
1. **实现简单**：仅需创建唯一约束字段	1. **无锁失效时间**：依赖业务释放，可能死锁
2. **天然互斥**：数据库保证唯一性	2. **非阻塞**：失败后需业务层重试
3. **无事务依赖**：降低连接占用	3. **锁释放风险**：进程崩溃可能锁泄漏
4. **轻量级**：仅依赖INSERT操作	4. **非公平锁**：可能线程饥饿

方案 2：基于悲观锁（SELECT ... FOR UPDATE）

SELECT ... FOR UPDATE （典型的悲观锁思路：先锁资源，再操作，从一开始就杜绝冲突）

1. 悲观锁可以直接用业务表

2. 抢锁 = 执行 SELECT ... FOR UPDATE（锁资源）

多个客户端要操作「商品 1001 的库存」，抢锁的核心操作就是执行这条 SQL：

SELECT * FROM 商品库存表 WHERE goods_id=1001 FOR UPDATE

客户端 A 先执行：数据库会给「goods_id=1001」这一行加「排他锁」—— 这一行被 A 独占，其他客户端不能对这行加锁 / 修改；A 抢锁成功，可以开始扣减库存

客户端 B 后执行：执行同样的 SQL 时，数据库会让 B「阻塞等待」（卡住）—— 要么等 A 释放锁，要么等超时；如果 A 一直不释放，B 会超时失败（抢锁失败）

3. 执行业务 + 释放锁（用完解锁）

客户端 A 在「事务」里执行业务（比如 UPDATE 商品库存表 SET stock=499 WHERE goods_id=1001）；

业务执行完后，执行 COMMIT（提交事务）或 ROLLBACK（回滚事务）—— 事务结束，数据库会自动释放这一行的排他锁

此时客户端 B 的阻塞会被唤醒：如果锁还在（没人抢），B 就能加锁成功；如果已经被其他客户端抢了，B 继续等待 / 超时

死锁场景：客户端 A 锁了商品 1001，想再锁 1002；客户端 B 锁了 1002，想再锁 1001，双方互相等对方释放，卡死



悲观锁（SELECT ... FOR UPDATE）：

优点	缺点
1. **强一致性**：保证独占资源	1. **性能瓶颈**：高并发下数据库连接易耗尽
2. **阻塞机制**：自动排队等待	2. **死锁风险**：事务嵌套可能循环阻塞
3. **自动释放**：事务结束锁即解除	3. **无超时控制**：依赖数据库锁超时参数
4. **简单直观**：语义明确	4. **扩展性差**：仅限单点数据库场景

方案 3：乐观锁（基于版本号）认为是没有并发情况的，先不加锁

乐观锁全程不锁任何东西，只靠（版本号核对）保证互斥

1. 给业务表加“版本号字段”，比如 version 字段初始值为 1，每次成功修改库存，版本号就 +1（比如 1→2→3），相当于 “记录本的更新次数”

2. 抢锁 = 先查版本号（看当前记录本）

客户端想操作「商品 1001 的库存」，第一步先查最新数据：

查：SELECT stock, version FROM 商品库存表 WHERE goods_id=1001

结果：stock=500，version=1（记下来这个版本号）

3. 执行业务 + 校验版本号（改记录本）

客户端要扣减 1 个库存，执行更新 SQL 时，必须把查到的版本号作为条件：

更：UPDATE 商品库存表 SET stock=499, version=version+1 WHERE goods_id=1001 AND version=1;

执行后看「影响行数」：

如果影响行数 =1：说明「version=1」的条件满足（没人改过），更新成功 → 相当于 “抢锁成功”，业务完成

如果影响行数 = 0：说明「version=1」的条件不满足（已经被其他客户端改成 2/3 了），更新失败 → 相当于 “抢锁失败”，需要重新查最新版本号再试

4. 抢锁失败就重试（重新看记录本）

如果更新失败（影响行数 = 0），客户端不用等锁释放，直接重新走「查版本号→更新」的流程：

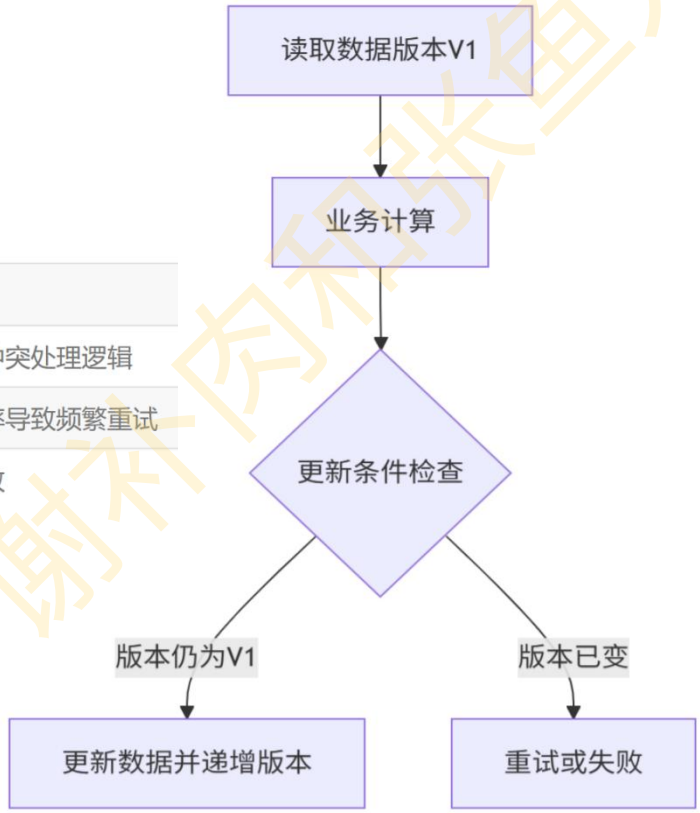
1. 重新查：SELECT stock, version FROM 商品库存表 WHERE goods_id=1001;（比如查到 version=2, stock=499）

2. 重新更：UPDATE ... WHERE version=2

3. 直到更新成功（抢锁成功），或达到重试次数上限（放弃）

乐观锁（基于版本号）：

优点	缺点
1. **高并发性能**：无阻塞，减少锁竞争	1. **实现复杂**：需自旋重试及冲突处理逻辑
2. **避免死锁**：无资源阻塞机制	2. **数据不一致风险**：高冲突率导致频繁重试
3. **轻量级**：仅需UPDATE操作	3. **非强一致性**：属于最终一致



对比维度	唯一键约束	SELECT...FORUPDATE 悲观锁	版本号乐观锁
实现思路	利用唯一键「不可重复插入」特性： • 插数据=抢锁（成功=抢到） • 插失败=抢锁失败 • 删数据=释放锁	利用数据库排他锁「先锁后操作」特性： • 事务内 SELECT...FOR UPDATE=加锁 • 事务提交/回滚=释放锁 • 其他客户端阻塞等待	利用版本号+UPDATE 原子性「无锁重试」特性： • 查版本号→校验版本 • 影响行数=1=抢锁成功 • 影响行数=0=抢锁失败，重试
锁类型本质	乐观锁(先操作,冲突再失败)	悲观锁（先锁资源假设必冲突）	乐观锁（先操作冲突就重试）
是否阻塞	无阻塞（插入失败直接返回，不等待）	阻塞(未抢到锁的客户端会卡住，直到锁释放/超时）	无阻塞(失败后直接重试，不等待）
死锁风险	无（仅插入冲突，无锁持有）	有(多资源加锁顺序不当易死锁）	无（全程无锁，仅重试）
性能	中（仅插入/删除，无事务阻塞）	低(事务阻塞+锁持有时间长，并发高时性能暴跌）	高（无锁无阻塞，仅 UPDATE 原子操作）
是否需单独建表	是（需专门的锁表，存储 lock_key+client_id）	否（直接复用业务表，锁业务数据行）	否(业务表加 version 字段即可）
关键注意点	1.需加过期时间+定时任务避免死锁 2.释放锁要校验 client_id 防误删	1.必须加索引（否则锁整张表） 2.必须在事务内执行 3.缩短事务时长减少阻塞	1.必须用独立 version 字段（不能用业务字段） 2.更新时必须带 version 条件 3.控制重试次数避免无限重试
数据一致性	最终一致(插入失败需业务重试）	强一致（锁持有期间独占资源，操作必成功）	最终一致(失败重试后保证一致）
适用场景	1.并发量中等 2.业务逻辑简单(如定时任务互斥） 3.不想修改原有业务表	1.并发量低 2.写多读少（如资金转账） 3.业务逻辑复杂,必须保证操作成功（不接受重试）	1.并发量高 2.读多写少（如商品库存扣减、秒杀） 3.能接受少量重试开销
典型缺点	1.高并发下大量插入失败请求打满数据库 2.锁表需额外维护	1.阻塞导致响应慢,并发高时系统卡顿 2.死锁风险需额外控制	1.高冲突场景下重试次数多，有性能开销 2.不适合必须一次成功的业务

三、基于 ZooKeeper 实现的分布式锁

• ZooKeeper 是分布式系统的“协调者”

• ZooKeeper 提供了一个高可用、高性能、顺序一致性的分布式协调服务，让分布式应用可以基于它来构建更高层次的同步原语（如锁、队列、选主）和服务（如配置管理、服务发现、命名服务）

• ZNode 数据节点：

ZooKeeper 的数据模型是一个类似文件系统的树形结构，称为 ZNode Tree

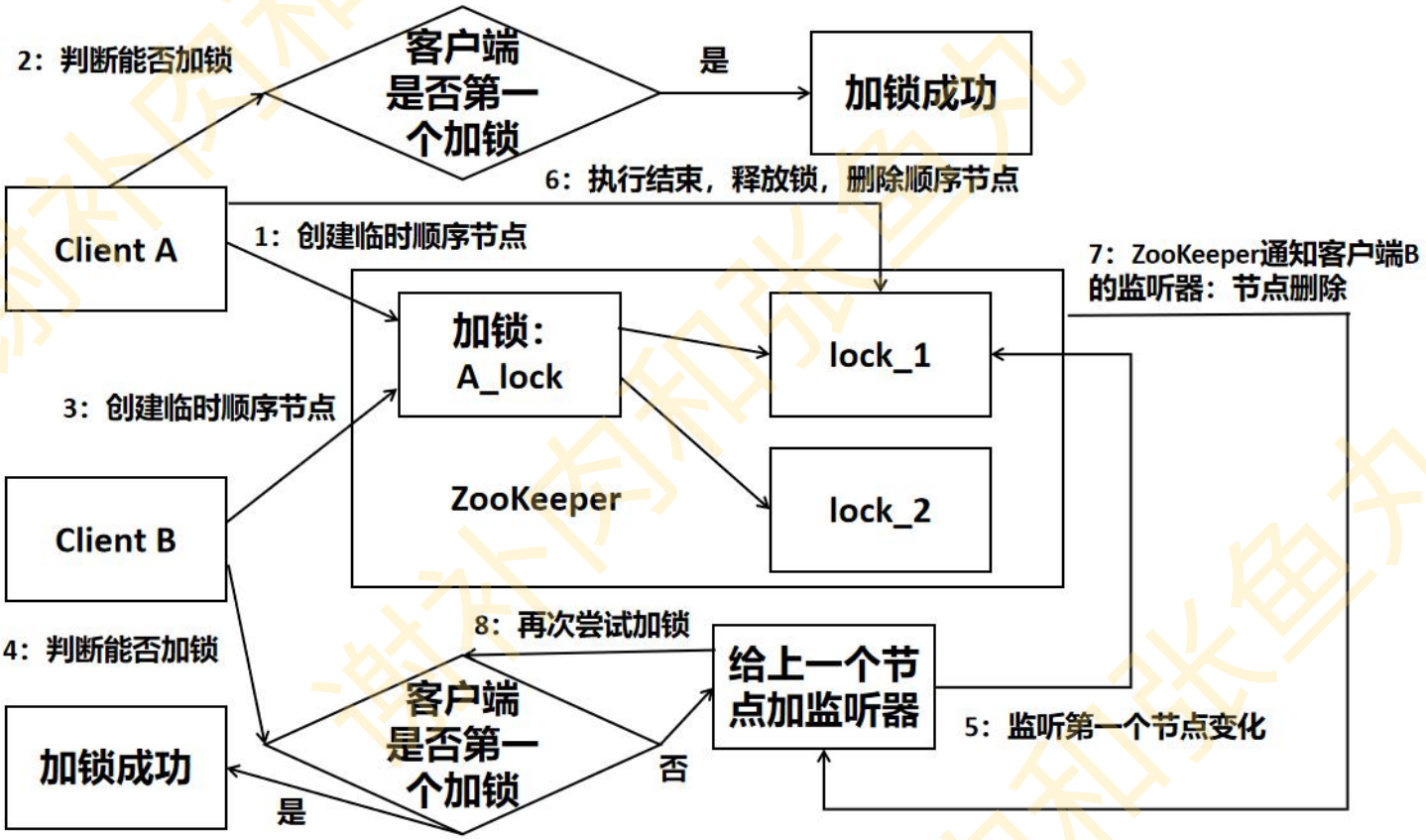
每个节点(/path/to/node)是一个 ZNode

ZNode 顺序节点会在节点名后追加一个单调递增的序列号，实现唯一 ID

• Watcher 监听器：

客户端可以在特定的 ZNode 上设置 Watcher

当被监听的 ZNode 或其子节点发生变化时，ZooKeeper 服务器会向设置该 Watcher 的客户端发送一个一次性通知



问题 1.基于数据库实现分布式锁的缺点

1.性能瓶颈。基于 select…for update 实现方案下，依赖事务，消耗数据库连接，cpu 和 IO 资源，在高并发场景下容易出现性能瓶颈。

2.死锁风险。在服务没有释放锁之前崩溃，会导致死锁。

3.可靠性问题。如果依赖单数据库实现分布式锁，数据库崩溃，锁将失效。

4.锁的公平性和可重入性支持弱。

问题 2.举一个产生 Redis 分布式锁死锁的场景

使用 setnx 命令实现分布式锁，A 客户端获取锁后，设置锁过期时间前，客户端 A 崩溃。因没有删除锁，且锁不会自动过期，导致死锁。

问题 3.在括号中填入对应的 zset 指令，存入秒杀的分数命令（1），获取分数范围的命令（2），获取分数（3）

1.ZADD

示例：ZADD seckill 1712345678999 user123，表示用户 user123 在时间戳 1712345678999 秒杀成功。

2.ZRANGEBYSCORE

示例： ZRANGEBYSCORE seckill 1712345678000 1712345678999，获取时间戳在 1712345678000 到 1712345678999 之间的所有用户。

3.ZSCORE

示例：ZSCORE seckill user123，返回用户 user123 的秒杀时间戳。

（2022 年 11 月）某大型电商平台建立了一个在线 B2B 商店系统，并在全国多地建设了货物仓储中心，通过提前备货的方式来提高货物的运送效率。但是在运营过程中，发现会出现很多跨仓储中心调货从而延误货物运送的情况。为此，该企业计划新建立一个全国仓储货物管理系统，在实现仓储中心常规管理功能之外，通过对在线 B2B 商店系统中订单信息进行及时的分析和挖掘，并通过大数据分析预测各地仓储中心中各类货物的配置数量，从而提高运送效率，降低成本。

当用户通过在线 B2B 商店系统选购货物时，全国仓储货物管理系统会通过该用户所在地址、商品类别以及仓储中心的货物信息和地址，实时为用户订单反馈货物起运地（某仓储中心）并预测送达时间。反馈送达时间的响应时间应小于 1 秒。

为满足反馈送达时间功能的性能要求，设计团队建议在全国仓储货物管理系统中采用数据缓存集群的方式，将仓储中心基本信息、商品类别以及库存数量放置在内存的缓存中，而仓储中心的其它商品信息则存储在数据库系统。

问题 1：设计团队在讨论缓存和数据库的数据一致性问题时，李工建议采取数据实时同步更新方案，而张工则建议采用数据异步准实时更新方案。请用 200 字以内的文字，简要介绍两种方案的基本思路，说明全国仓储货物管理系统应该采用哪种方案，并说明采取该方案的原因。

同步：客户端 —写请求→ DB → 缓存，返回结果。保证数据库和缓存强一致性，但响应时间较长
异步：客户端 —写请求→ DB → 返回结果，再异步同步缓存。响应快，但存在数据库和缓存不一致

- 实时同步更新，每次数据库更新时同步更新缓存。任何写操作（如库存扣减）都需在事务中同时修改数据库和缓存，确保二者数据强一致性，但响应延迟较高。
- 异步准实时更新，写操作仅优先更新数据库，通过消息队列或日志监听异步触发缓存更新。缓存与数据库存在短暂延迟 （毫秒到秒级），但写操作响应更快。
- 题中有对反馈时间要小于 1s 的要求，所以异步准实时更新更合适

问题 2：随着业务的发展，仓储中心以及商品的数量日益增加，需要对集群部署多个缓存节点，提高缓存的处理能力。李工建议采用缓存分片方法，把缓存的数据拆分到多个节点分别存储，减轻单个缓存节点的访问压力，达到分流效果。【缓存分片方法常用】的有【哈希算法】和【一致性哈希算法】，李工建议采用一致性哈希算法来进行分片。请用 200 字以内的文字简要说明两种算法的基本原理，并说明李工采用一致性哈希算法的原因。

通过计算数据的哈希值（如 hash(key) % N），将数据分配到固定的缓存节点（N 为节点总数）。节点增减时，映射关系全局失效，需重新分配数据，导致大规模缓存穿透
一致性哈希算法将哈希值组成虚拟环（0~2^32-1），对每个缓存节点计算哈希值，定位到环上。对数据 key 计算哈希值，在环上顺时针寻找最近节点作为存储位置，增删节点时，仅影响相邻数据（如环上相邻节点），避免全局重新分片
因此，一致性哈希以最小化数据迁移和均衡负载的特性，更适配高并发、需动态扩展的仓储系统

补充知识：
一致性哈希：是一种特殊的哈希技术，专门为解决分布式系统中动态伸缩（节点增加或减少） 时所面临的数据大规模迁移问题而设计的。它主要应用于分布式缓存（如 Redis Cluster、Memcached 客户端分片）、负载均衡器、分布式数据库分片等场景

- 传统哈希的问题：
  - 分片的时候，假如分成 3 片，数据存在哪台机器上呢？
  - 直接 Hash(key)/3，余几就放在哪台机器上
  - 但是如果扩展了一台机器 4，再 Hash(key)/4 的话，所有的数据都要跟着重新存放到服务器上，所有的映射都失效了
  - 为解决上述问题引入了一致性哈希

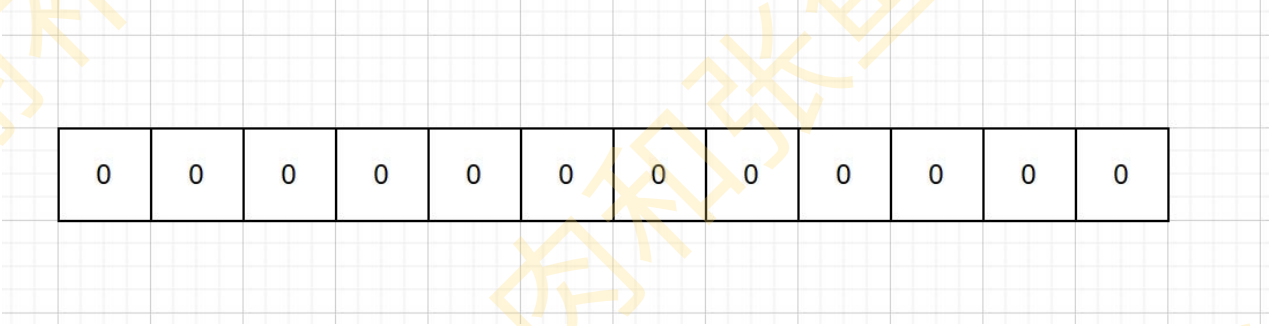
- 一致性哈希原理：
  - Hash(key)/2^{32},把取模的数固定为 2^32，形成一个 0 到 2^32 - 1 的环
  - 比如有 3 个分片，对机器的 ip 或者 hostname 做哈希，根据哈希值把机器安排到环上
  - key1 来了以后会顺时针找最近的落盘，如果增加了 1 台机器，比如 4 号机器落在了 3 和 1 之间，只需要把原来存到 1 号机器面的存到 4 号机器就可以了，其他机器不变

- **增减机器会发生什么？**
  - 加一台机器：只影响新机器和它前面那台机器之间的数据，其他数据完全不用动
  - 减一台机器：只影响挂掉的机器和它前面那台之间的数据，这些数据会顺时针转移到下一台
- **一致性哈希存在问题：**
  - 数据倾斜：节点的哈希位置分布不均匀，导致环上某些区段负责大量数据，而某些节点（落在密集区）负载过高，另一些节点负载过低
  - 节点异质性：节点本身的处理能力和存储空间可能不同
  - 解决方法：通过设置大量虚拟节点可以环节上述问题

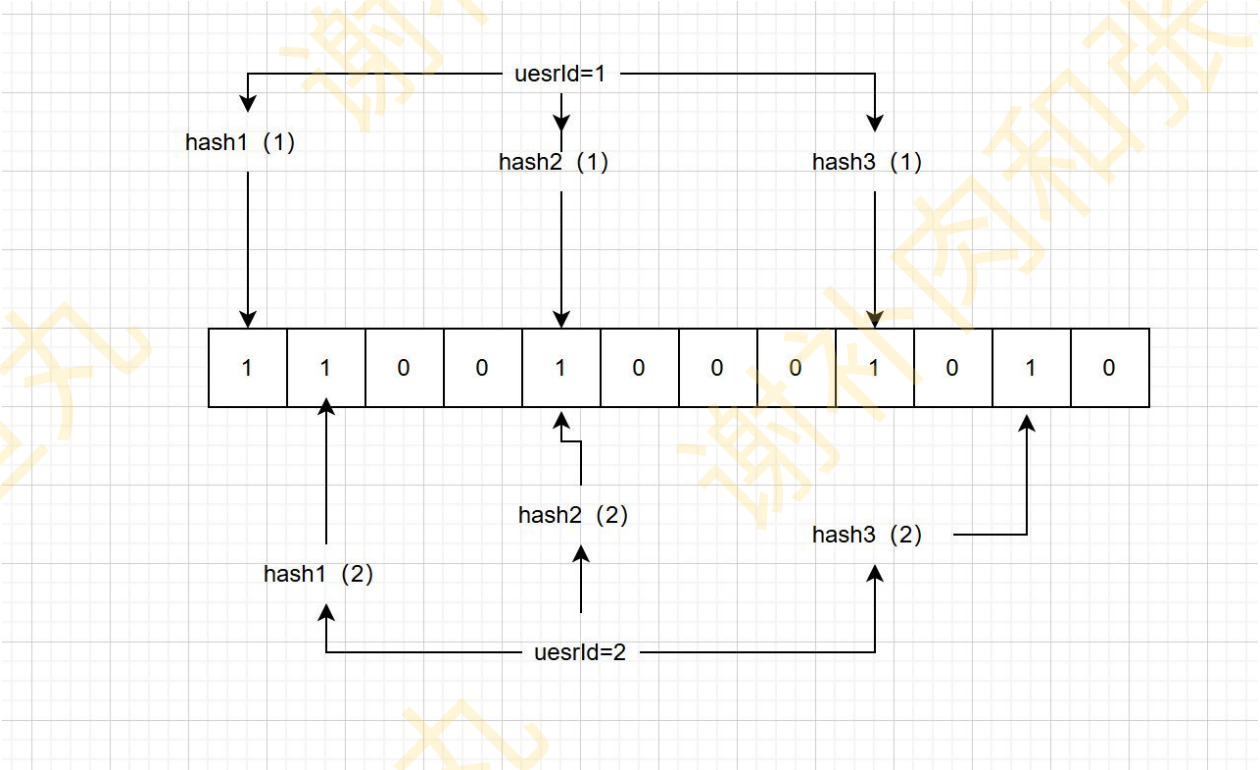
问题 3：全国仓储货物管理系统开发完成，在运营一段时间后，系统维护人员发现大量黑客故意发起非法的商品送达时间查询请求，造成了缓存穿透。张工建议尽快采用布隆过滤器方法解决。请用 200 字以内的文字解释布隆过滤器的工作原理和优缺点。

- **布隆过滤器**：是一种空间高效的概率型数据结构，用于快速判断一个元素是否在某个集合中。核心特点是通过哈希函数和位数组实现低内存占用，但存在一定误判率，而不会漏判。

一开始：每位都是 0 的数组



存储：（我们是要查 userid，userid 作为 key），做三次不同的哈希，每次都对应到布隆过滤器里



判断：userid1 来了，计算哈希函数，如果 3 个全是 1，则存在。如果对应位置不都是 1，则不存在直接返回

但可能会存在哈希碰撞，比如来一个 userid5，5 哈希后在第 1、5、11 位，就出现了哈希碰撞，出现了误判

减少误判方法：

- 增加数组长度
- 设置合适的哈希函数个数

你必须预先估算一个你将要存入的元素数量 n，并设定一个你期望的、能接受的误判率 p

然后，根据这两个值，计算出所需的位数组大小 m 和最优哈希函数个数 k

计算公式：

位数组大小 m 的计算公式：

m = - (n * ln(p)) / (ln(2))^2

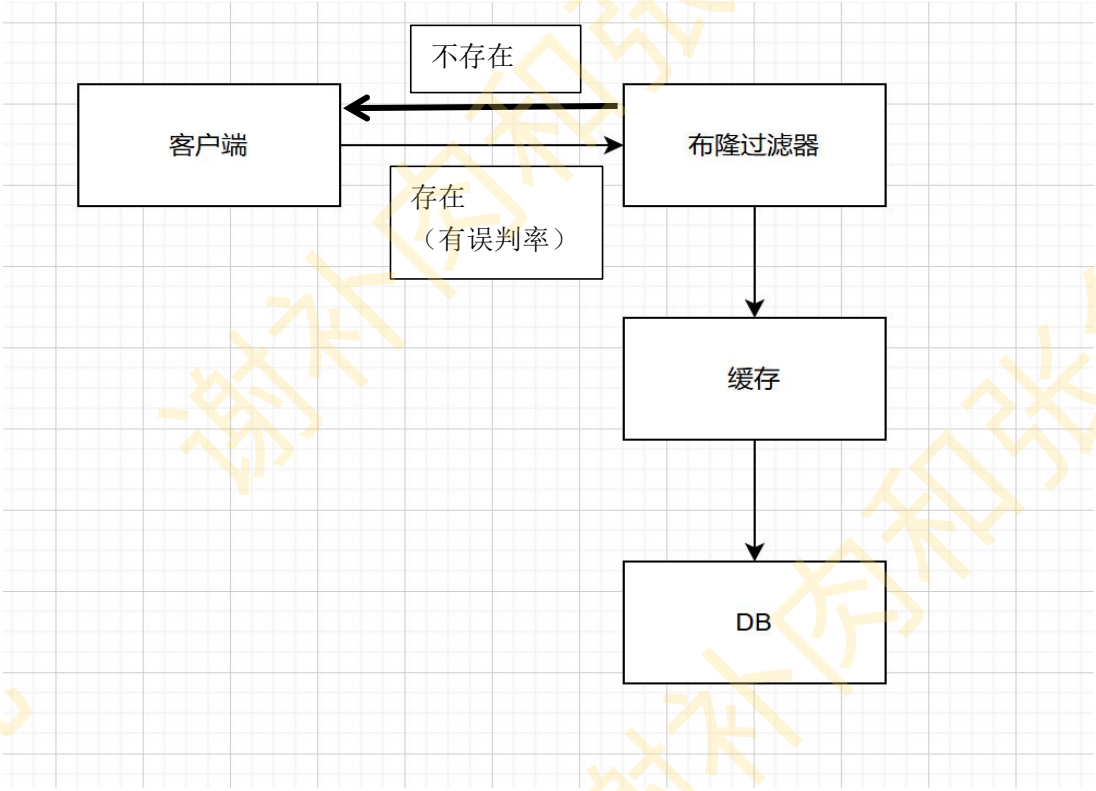
这意味着，在 n 固定时，你期望的误判率 p 越小，所需的位数组 m 就越大（空间开销越大）

最优哈希函数个数 k 的计算公式：

k = (m / n) * ln(2)

计算出 k 后通常四舍五入取整

工作流程：



应用场景：

• 缓存穿透：大量请求查询一个根本不存在的数据（如不存在的用户 ID），导致绕过缓存直接冲击底层数据库。

• 网页爬虫 URL 去重：避免重复爬取相同的 URL。

• 邮件服务垃圾邮件过滤：快速判断一个发件人地址是否在黑名单中。

•数据库查询优化：在查询前快速判断记录是否存在，避免昂贵的数据库查询（尤其对不存在的记录）。

• 分布式系统：快速判断一个文件/数据块是否存在于集群中的某个节点。

• Weak Password Check：快速检查用户设置的密码是否在常用的弱密码名单中。

答案：

布隆过滤器使用一个大型的二进制位数组和多个哈希函数。对数据 key 进行 k 次哈希计算，将位数组中对应位置 1。检查 key 的 k 个哈希位置——若所有位均为 1，则判断可能存在（允许误判）；若有一位为 0，则一定不存在。优点在于以极低内存开销（如亿级数据仅需百 MB）存储海量 key 是否存在；仅需计算 k 次哈希和位检查，时间复杂度 O(k)；拦截非法请求（如不存在 key），避免其穿透缓存直达数据库。缺点是因哈希冲突，可能将不存在的 key 误判为存在（但绝不会反向误判），需调整参数（位数组大小、哈希函数数量）平衡误判率与资源。

基于系统需要拦截大量黑客故意发起非法的商品送达时间查询请求，布隆过滤器是一个好的选择。

（2021 年 11 月）阅读以下关于数据库设计的叙述，在答题纸上回答问题 1 至问题 3。

某医药销售企业因业务发展，需要建立线上药品销售系统，为用户提供便捷的互联网药品销售服务。该系统除了常规药品展示、订单、用户交流与反馈功能外，还需要提供当前热销产品排名、评价分类管理等功能。通过对需求的分析，在数据管理上初步决定采用关系数据库（MySQL）和数据库缓存（Redis）的混合架构实现。

经过规范化设计之后，该系统的部分数据库表结构如下所示。

供应商（供应商 ID，供应商名称，联系方式，供应商地址）；

药品（药品 ID，药品名称，药品型号，药品价格，供应商 ID）；

药品库存（药品 ID，当前库存数量）；

订单（订单号码，药品 ID，供应商 ID，药品数量，订单金额）。

问题 1：在系统初步运行后，发现系统数据访问性能较差。经过分析，刘工认为原来数据库规范化设计后，关系表过于细分，造成了大量的多表关联查询，影响了性能。例如当用户查询商品信息时，需要同时显示该药品的信息、供应商的信息、当前库存等信息。为此，刘工认为可以采用反规范化设计来改造药品关系的结构，以提高查询性能。修改后的药品关系结构为：

药品(药品 ID，药品名称，药品型号，药品价格，供应商 ID，供应商名称，当前库存数量）；

请用 200 字以内的文字说明常见的反规范化设计方法，并说明用户查询商品信息应该采用哪种反规范化设计方法。

补充知识：

规范化：数据库设计的基本原则，目的是通过分解表、消除数据冗余和依赖关系来减少数据异常（插入、更新、删除异常），确保数据的一致性和完整性。它遵循一系列范式（如 1NF, 2NF, 3NF, BCNF 等）

反规范化：在已经规范化的数据库模式基础上，为了提升特定查询的性能，有意地引入一定程度的冗余数据或合并表的过程。这是一种有目的、有策略地牺牲部分规范化规则来换取读取速度的优化技术

常用的反规范化设计方法：

- 添加冗余列：在表中添加一个或多个原本需要通过 JOIN 从其他表获取的列
- 预连接表：将两个或多个经常需要 JOIN 查询的表合并成一个物理表
- 创建汇总表：创建一个新表，预先存储通过复杂聚合查询（如 GROUP BY）计算出的结果
- 使用物化视图：物化视图是存储了实际查询结果的视图（与普通视图只存储查询定义不同）。数据库系统负责维护物化视图数据的更新（全量刷新或增量刷新）
- 水平（垂直）分区：水平是横着切一刀，比如一列是省份，可以横着切一刀把不同省份分在不同表。垂直是竖着砍一刀，一些列放到一个表，另一些列放到另一个表

答案：

1.添加冗余列：在表中添加一个或多个原本需要通过 JOIN 从其他表获取的列

2.预连接表：将两个或多个经常需要 JOIN 查询的表合并成一个物理表

3.创建汇总表：创建一个新表，预先存储通过复杂聚合查询（如 GROUP BY）计算出的结果

用户查询商品信息应该采用：增加冗余列。通过增加供应商名称，当前库存数量冗余列，减少 join 操作，提高查询性能

问题 2：王工认为，反规范化设计可提高查询的性能，但必然会带来数据的不一致性问题。请用 200 字以内的文字说明在反规范化设计中，解决数据不一致性问题的三种常见方法，并说明该系统应该采用哪种方法。

强一致性方案：

保证一致性的关键：确保所有更新操作都能正确传播到冗余副本。

1.事务：将涉及多个表（包括主表和包含冗余数据的表）的更新操作包含在同一个数据库事务中。事务保证 ACID 特性（原子性、一致性、隔离性、持久性）

2.触发器：在数据库层面，为源表（主数据表）的更新（INSERT, UPDATE, DELETE）创建触发器。触发器在执行主操作后自动触发，负责更新相关的冗余数据副本

最终一致性方案：

1.异步批处理：定期（如每分钟、每小时、每天）运行批处理作业或 ETL 流程。作业从主数据源读取变更，并同步到包含冗余数据的各个表或物化视图/汇总表中

2.消息队列：

①应用在修改主数据源后，将数据变更事件发布到消息队列

②一个或多个独立的消费者服务订阅这些消息队列

③消费者接收消息，解析事件内容，并执行更新相关冗余数据的操作（可能是更新其他数据库表、写入缓存、刷新搜索索引等）

答案：

1.事务：将涉及多个表（包括主表和包含冗余数据的表）的更新操作包含在同一个数据库事务中。

2.触发器：在数据库层面，为源表（主数据表）的更新（INSERT, UPDATE, DELETE）创建触发器。触发器在执行主操作后自动触发，负责更新相关的冗余数据副本。

3.消息队列：将更新操作发送给消息队列，异步去更新冗余数据。

分情况考虑对应药品库存扣减这种强一致性要求的场景用事务和触发器较好。对于展示热门商品这种对实时性要求一般的场景可以采用消息队列同步，保证最终一致性即可。

问题 3：该系统采用了 Redis 来实现某些特定功能(如当前热销药品排名等)，同时将药品关系数据放到内存以提高商品查询的性能，但必然会造成 Redis 和 MySQL 的数据实时同步问题。

（1）Redis 的数据类型包括 String、 Hash、 List、 Set 和 ZSet 等，请说明实现当前热销药品排名的功能应该选择使用哪种数据类型。

Zset

（2）请用 200 字以内的文字解释说明解决 Redis 和 MySQL 数据实时同步问题的常见方案。

1.旁路缓存：

读操作：查询 Redis → 未命中则查 MySQL → 数据写入 Redis；

写操作：先更新 MySQL → 同步删除或延迟删除 Redis 相关键

2.MySQL Binlog 同步方案：监测 Binglog 文件进行同步缓存

3.分布式事务双写策略：事务内同步更新 MySQL + Redis

（2020 年 11 月）阅读以下关于数据库缓存的叙述，在答题纸上回答问题 1 至问题 3。

某互联网文化发展公司因业务发展，需要建立网上社区平台，为用户提供一个对网络文化产品(如互联网小说、电影、漫画等)进行评论、交流的平台。该平台的部分功能如下:

(a)用户帖子的评论计数器;

(b)支持粉丝列表功能;

(c)支持标签管理;

(d)支持共同好友功能等;

(e)提供排名功能，如当天最热前 10 名帖子排名、热搜榜前 5 排名等;

(f)用户信息的结构化存储;

(g)提供好友信息的发布/订阅功能。

该系统在性能上需要考虑高性能、高并发，以支持大量用户的同时访问。开发团队经过综合考虑，在数据管理上决定采用 Redis+数据库(缓存+数据库)的解决方案。

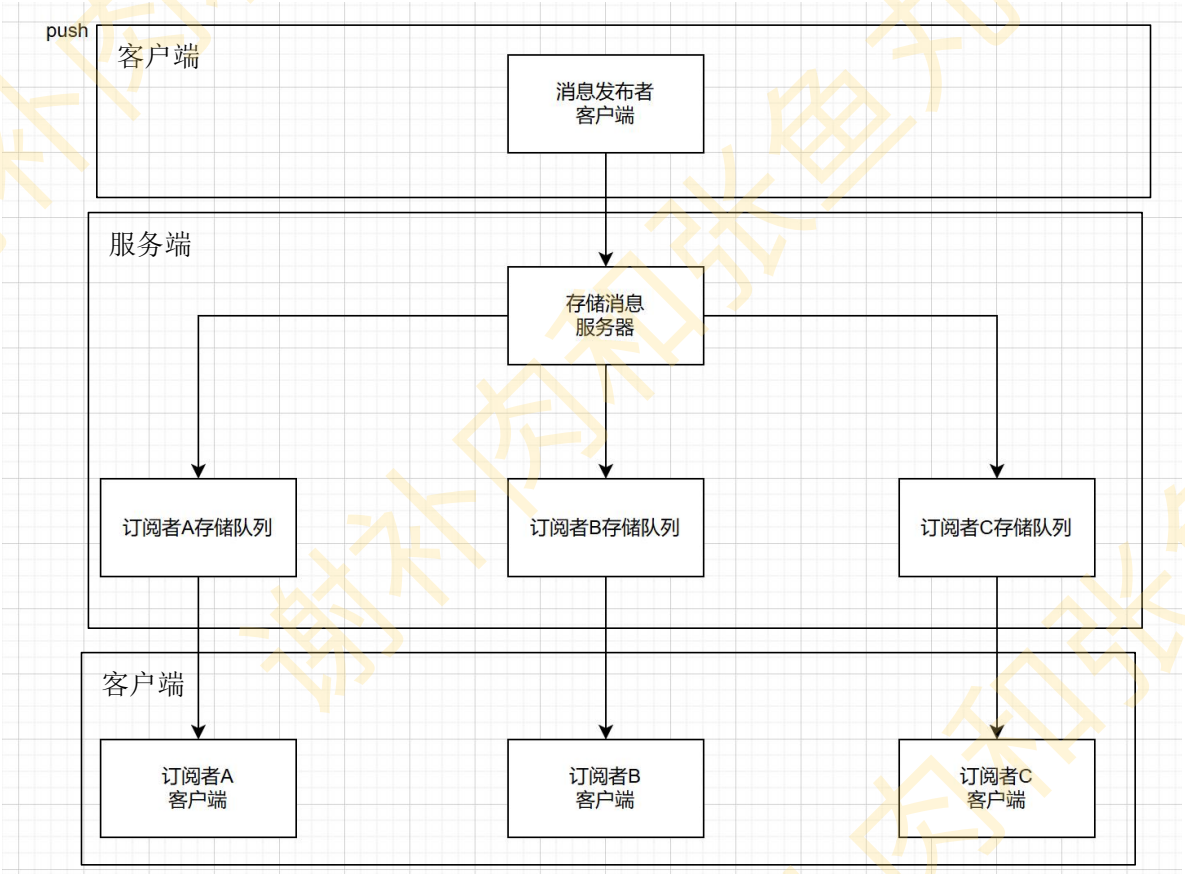
问题 1:Redis 支持丰富的数据类型,并能够提供--些常见功能需求的解决方案。请选择题干描述的(a)~(g)功能选项，填入表 4-1 中(1)~ (5)的空白处

表 4-1 Redis 数据类型与业务功能对照表

数据类型	存储的值	可实现的业务功能
STRING	字符串、整数或浮点数	(1)
LIST	列表	(2)
SET	无序集合	(3)
HASH	包括键值对的无序散列表	(4)
ZSET	有序集合	(5)

知识补充：

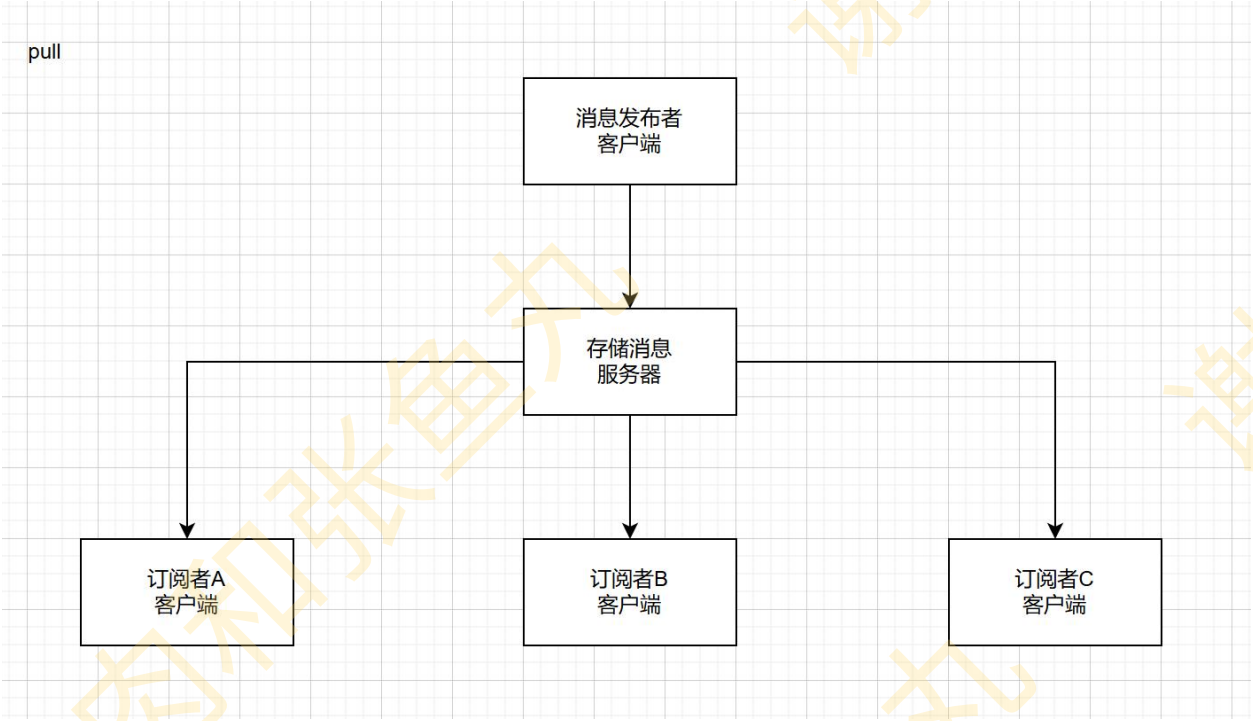
好友信息订阅发布原理-push，推送：



优点：实时性好

缺点：如果发布方有很多订阅者，会出现写扩散问题

好友信息订阅发布原理-pull，拉取：客户端触发，定期拉取或主动触发拉取



优点：不存在写操作

缺点：实时性差，很多订阅者都在同一时刻拉取数据，会出现读扩散

解决方案：

push、pull 结合方式，活跃度高 push，活跃度低 pull

问题 2：该网上社区平台需要为用户提供 7×24 小时的不间断服务。同时在系统出现宕机等故障时，能在最短时间内通过重启等方式重新建立服务。为此，开发团队选择了 Redis 持久化支持。Redis 有两种持久化方式，分别是 RDB(Redis DataBase)持久化方式和 AOF (Append OnlyFile)持久化方式。开发团队最终选择了 RDB 方式。请用 200 字以内的文字，从磁盘更新频率、数据安全、数据一致性、 重启性能和数据文件大小五个方面比较两种方式，并简要说明开发团队选择 RDB 的原因。

1.从磁盘更新频率来说，RDB 更新频率低指定时间间隔生成 RDB 文件才写一次磁盘，AOF 更新频率高每个写命令都要进行写磁盘。

2.从数据安全角度，RDB 两次生成快照之间的时间段内存在丢失数据的风险，AOF 采用追加日志的方式记录每次写操作数据安全性更高。

3.数据一致性方面，AOF 通过记录每次写操作数据一致性更好，RDB 由于生成快照操作过重，不便高频执行，数据一致性较弱。

4.重启性能：RDB 文件直接可以写进 Redis 性能高，AOF 需要按顺序依次执行命令重建数据性能低。

5.文件大小方面，RDB 是二进制文件占用空间小，AOF 记录的是 Redis 命令，虽然有重写操作，还是比 RDB 文件大。

因为该系统有最短时间内通过重启的要求所以选 RDB 方式持久化。

问题 3：缓存中存储当前的热点数据，Redis 为每个 KEY 值都设置了过期时间，以提高缓存命中率。为了清除非热点数据，Redis 选择"定期删除+惰性删除"策略。如果该策略失效，Redis 内存使用率会越来越高，一般应采用内存淘汰机制来解决。请用 100 字以内的文字简要描述该策略的失效场景，并给出三种内存淘汰机制。

不淘汰策略

- noeviction：默认策略，内存不足时拒绝所有写操作，但允许读操作全体键

淘汰策略

- allkeys-lru：从所有键中淘汰最近最少使用的键（LRU 算法）
- allkeys-lfu：从所有键中淘汰使用频率最低的键（LFU 算法）
- allkeys-random：从所有键中随机淘汰键

过期键淘汰策略

- volatile-lru：从设置了过期时间的键中淘汰最近最少使用的键。
- volatile-lfu：从设置了过期时间的键中淘汰使用频率最低的键 volatile-ttl：从设置了过期时间的键中淘汰剩余存活时间最短的键
- volatile-random：从设置了过期时间的键中随机淘汰

NoSql：

NoSQL 是"Not Only SQL" 的缩写。它代表了一类非关系型数据库管理系统。与传统的关系型数据库（如 MySQL, Oracle, SQL Server）不同，NoSQL 数据库在设计上放弃了关系模型、固定的表结构（Schema）、SQL 查询语言以及通常对 ACID 事务的严格保证，以换取其他方面的优势，特别是在可扩展性、灵活性、高性能和处理特定数据类型上。

NoSQL 诞生的背景：

1. 互联网规模的数据量： Web2.0 和大型互联网应用（如社交网络、搜索引擎、电子商务）产生了前所未有的海量数据（Big Data），传统 RDBMS 在单机或主从架构下难以有效水平扩展（因为有多表的关联）

2. 数据类型的多样化：应用需要存储和处理的数据不再仅仅是规整的行列数据，还包括 JSON 文档、键值对、图结构、时间序列、地理位置信息等半结构化和非结构化数据（有些数据没法拆成行和列）

3. 灵活性与敏捷开发： 快速迭代的敏捷开发要求数据库模式（Schema）能够灵活变更，而 RDBMS 的固定模式修改成本较高

4. 高并发与低延迟：某些场景（如实时分析、游戏、广告竞价）需要极高的读写吞吐量和极低的响应延迟

5. 高可用性需求：分布式架构下，需要系统在部分节点或网络分区故障时仍能继续提供服务

NoSQL 数据库的核心特点：

灵活的数据模型 (Flexible Schema / Schema-less / Schema-on-Read):

- **不需要预先严格定义表结构**（字段、数据类型、关系）
- 可以**存储各种结构、半结构化和非结构化数据**（JSON, XML, 键值对, 图节点/边, 列族等）

- **同一集合/表中的不同记录可以有不同的字段**（虽然实践中通常保持一定一致性）
- 模式在**写入时很灵活，读取时可能需要应用逻辑来处理不同结构**（Schema-on-Read）

水平可扩展性 (Horizontal Scalability):

- 核心优势！主要通过在**分布式集群中添加更多廉价的普通服务器**（节点）来**扩展**性能和存储容量
- 通常采用分片/分区技术，将数据分散到多个节点上，实现负载均衡
- 易于应对数据量和访问量的快速增长

高性能：

- 针对特定的数据模型和访问模式进行了优化（如键值访问非常快）
- 简化的数据模型和去中心化架构减少了开销（如避免复杂 JOIN）
- 通常将数据保存在内存中或使用高效的存储引擎

高可用性与容错性：

- 通过数据复制（Replication）将数据副本存储在不同节点上
- 当某个节点故障时，其他拥有副本的节点可以继续提供服务
- 符合 CAP 定理

最终一致性 (Eventual Consistency - BASE)：

- 许多 NoSQL 数据库（特别是分布式键值、文档、列族库）默认采用最终一致性模型（属于 BASE 原则 - Basically Available, Soft state, Eventual consistency）
- 在写入后，数据副本可能暂时不一致，但系统保证在没有新的写入操作后，经过一段时间，所有副本最终会达到一致状态
- 这牺牲了强一致性（Strong Consistency - ACID 中的 C），换取了更高的可用性和分区容忍性。当然，部分 NoSQL 也提供可配置的一致性级别（如读/写法定人数）

NoSQL 数据库的优缺点

优点：

- 极高的可扩展性：轻松水平扩展应对大数据量和高并发
- 灵活的模式：适应快速变化的需求和半结构化/非结构化数据
- 高性能：针对特定模型优化，读写速度通常很快
- 高可用性：分布式架构和复制机制保障服务持续性
- 成本效益：可使用廉价硬件构建集群（相较于大型高端 RDBMS 服务器）
- 适合特定数据模型：为键值、文档、列、图等特定结构提供最优支持

缺点：

- 弱一致性（多数）：默认最终一致性模型可能带来数据短暂不一致问题，需要应用层处理。强一致性选项通常有性能代价
- 有限的查询能力：通常不支持复杂的 JOIN、子查询等关系型操作。查询能力受限于其数据模型和接口
- 无标准接口：缺乏像 SQL 那样的统一查询语言，不同产品 API 差异大，学习和迁移成本较高
- 事务支持有限：传统上对多文档/多行/多图的 ACID 事务支持较弱或不支持
- 成熟度与工具链：部分 NoSQL 数据库相对年轻，管理工具、监控工具、BI 工具支持可能不如成熟的 RDBMS 丰富

• 数据建模挑战：灵活的模式也意味着需要更仔细地根据访问模式设计数据模型，否则性能可能很差。关系型设计的经验不能完全套用

类型	数据模型	核心特点	典型应用	代表产品
键值存储 key-value	key：唯一标识符 value:可以从简单数据到复杂 BLOB 的任何数据	性能极高，读写速度快 数据模型最简单，查询方式单一 极易水平扩展 值对数据库是不透明的（数据库不解析值的内容）	缓存 用户配置、购物车 存储元数据、临时数据 分布式锁 排行榜、计数器	redis memcached
文档数据库	类似 JSON 的文档 文档是自包含的数据单元，包含键值对、嵌套文档和数组 每个文档有唯一 ID	模式灵活，易于迭代开发 数据模型最自然的映射到面向对象编程 支持对文档内部字段进行索引和查询 通常提供丰富的查询表达式	内容管理系统 用户档案、产品目录 移动应用后端 实时分析 日志记录	MongoDb
列族存储	按行键、列族和列限定符组织 行键：主键 列族：列的容器，预先定义 列限定符：列族内的具体列，可动态添加 结构是稀疏的	超大规模数据集 极高写入吞吐量 按行键范围查询和按列查询 水平扩展能力强	时序数据 事件日志记录 推荐系统 内容大数据存储 大量写入的应用	HBase Cassandra
图数据库	使用节点、关系和属性来存储数据 节点：表示实体（人、地点等） 关系：连接节点，有方向和类型 属性：节点和关系的键值对属性	处理深度、复杂关系 遍历关系速度快，性能不受关系深度影响 能够揭示数据中隐藏的模式和连接 使用专门的图查询语言	社交网络 欺诈检测 推荐引擎 知识图谱 网络基础设施管理 权限关系管理	Neo4j ArangoDb

NoSQL 数据库对比及知识点：

Memcached 和 Redis：

为高并发场景提供纯内存高速 KV 缓存，减轻数据库压力，提升系统响应速度与吞吐能力

Memcached：

- 数据类型：仅 key-value（字符串）
- 纯内存、不持久化，重启数据丢失
- 多线程，高并发小数据缓存性能极好
- 仅支持字符串，value 最大一般 1MB
- 典型场景：页面缓存、对象缓存、减轻 DB 压力

对比维度	Memcached	Redis
数据类型	仅支持简单 key-value（字符串）	支持多种：string、hash、list、set
持久化	不支持，纯内存，重启数据丢失	支持 RDB 快照+AOF 日志持久化
线程模型	多线程，并发性能好	单线程命令处理+6.0+多线程 IO
value 大小	一般限制 1MB	支持更大，默认可到 512MB
分布式支持	依赖客户端分片，无原生集群	支持主从、哨兵、Redis Cluster 集群
高可用	弱，需业务层保证	强，哨兵自动故障切换
事务支持	无	支持简单事务（MULTI/EXEC）
适用场景	纯缓存、高并发、简单 KV 场景	缓存、持久化 KV、业务数据结构场景

MongoDB：

面向文档存储，模式灵活、易扩展，适合半 / 非结构化、嵌套结构多、需求迭代快的业务场景

- 文档型 NoSQL 数据库
- 数据格式：BSON（二进制 JSON）
- 特点：模式灵活（schemaless）、易扩展、适合海量半结构化数据

表：集合（Collection）

行：文档（Document）

列：字段（Field）

主键：_id 字段

BSON 存储数据的核心逻辑：

- 固定头部 + 键值对 + 结束符

[文档总长度(4 字节)] + [键值对 1] + [键值对 2] + ... + [结束符(1 字节，0x00)]

MongoDB 对 BSON 文档的查询分为 4 个核心步骤：

1. 解析查询条件，生成 “BSON 格式查询指令”

你写的 MongoDB 查询语句（比如 db.col.find({age: 25})），客户端会先把查询条件转换成 BSON 二进制格式，再发给服务端

- 举例：查询条件 {age: 25} 会被转成 BSON 二进制（包含 “类型标识 0x10（整数）+ 键名 age + 值 25 的二进制”）

2. 匹配索引

MongoDB 会先检查查询字段是否有索引：

① 有索引：直接查索引的 BSON 结构（索引也是以 BSON 格式存储的，只存 “键 + 指针”），通过索引快速定位到目标文档的物理存储位置；

② 无索引：走 “全表扫描（COLLSCAN）”— 逐个读取文档的 BSON 二进制数据，解析后匹配条件

3. 解析 BSON 文档，精准匹配字段

找到目标文档的物理位置后，服务端会对 BSON 二进制做 “按需解析”（不是全解析）：

① 先读 BSON 文档的 “总长度”（4 字节），确定解析范围

② 按 “类型标识 + 键名 + 值” 的结构，只解析查询涉及的字段（比如查 age 就只解析 age 字段，不解析 name/address 等）

③ 二进制值直接对比：比如查询 age:25，就把文档中 age 字段的二进制值（0x00 0x00 0x00 0x19）和查询条件的二进制值直接对比，无需类型转换

4. 返回结果（可选：投影 + 序列化）

- 如果指定了投影（比如 db.col.find({age:25}, {name:1})），只解析并返回指定字段的 BSON 数据
- 服务端把匹配到的 BSON 数据序列化成 JSON/BSON（按需），返回给客户端

对比项	JSON	BSON
编码格式	文本字符串	二进制格式
数据类型	少（字符串、数字、布尔、数组、对象）	更丰富（日期、二进制、Decimal、地理位置、向量等）
读写效率	解析慢、体积大	解析快、存储紧凑、IO 效率更高
遍历性能	无序，需全文扫描	有序，支持快速定位字段
适用场景	数据传输、接口交互	数据库存储、索引构建、高效查询
特殊能力	无	支持高维向量、地理位置等专用类型

HBase 和 Cassandra：

面向海量结构化 / 半结构化数据的分布式列存数据库，支持超高写入、线性扩展，适合大数据存储与离线分析

HBase：

- 基于 HDFS 存储（Hadoop 生态组件），面向海量结构化 / 半结构化数据
- 数据格式：KeyValue 格式 + 列式存储结构
- 特点：稀疏、分布式、可扩展、多版本、强一致性
- 适合：PB 级数据、高写入、离线分析、时序 / 日志 / 画像

数据模型：

1. 行键 Row Key
   - 每一行的唯一 ID
   - 按字典序排序
   - 你以理解成：主键
2. 列族 Column Family

- 一组列的集合
- 建表时必须定好，不能随便改
- 物理上存在一起，所以查询更快
- 可以理解成：文件夹

3. 列 Column
   - 在列族下面
   - 随时可以加，不用提前定义
   - 格式：列族:列名，如：info:name、info:age
   - 可以理解成：文件夹里的文件
4. 版本 Version / TimeStamp
   - 一个格子可以存多个版本
   - 用时间戳区分
   - 可以理解成：文件的修改历史
5. 单元格 Cell
   - 行键 + 列族 + 列 + 版本 定位一个格子
   - 存的都是字节数组

数据查询过程：

先通过 Row Key 定位到数据在哪，再去取数据

HBase 只有 3 种查询，没有复杂 SQL，没有 JOIN

1. 根据 Row Key 精确查询（最快！）
- get 'table', 'rowkey1'
- 毫秒级，性能最高

2. 根据 Row Key 范围查询（很快）
- scan 'table', {STARTROW => 'a', ENDROW => 'b'}
- 因为 Row Key 是有序的，所以范围扫描非常强

3. 全表扫描（最慢，尽量别用）
- 不加 Row Key 直接扫全表
- 考试里一出现就是：性能差、要优化

Cassandra：
- 设计思想：高可用、去中心化、线性扩展
- CAP 中属于：AP 系统（高可用 + 分区容忍）
- 一致性：可调一致性（可以自己指定强一致 / 最终一致）

特点：

1. 完全去中心化：没有 Master，没有中心节点，所有节点平等
2. Gossip 协议：节点之间互相通信、同步状态
3. 无单点故障：任何一台挂了，集群照样用

4. 写入性能极高：底层是 LSM 树
5. 跨数据中心（多机房）支持特别强
6. SSTable 存储

数据模型：先按 Partition Key 分片，再按 Clustering Key 排序
读写机制：

- 写：先写 CommitLog → 再写 MemTable → 刷成 SSTable
- 读：先读 MemTable → 再读 BlockCache → 再读 SSTable
- 合并：后台 Compaction 合并小文件

对比维度	HBase	Cassandra
数据模型	列族数据库（Column Family）行键 + 列族 + 列 + 版本	列族数据库（Column Family）分布式 KV，支持复合键
一致性模型	强一致性（CP 系统）	最终一致性（可调一致性，AP 为主）
CAP 定位	C + P	A + P
底层依赖	强依赖 Hadoop + HDFS + Zookeeper	无外部依赖，去中心化
集群管理	中心化：Master + RegionServer + ZK	去中心化：Gossip 协议，无中心节点
故障处理	Master 单点，依赖 ZK 切换	无单点，自动容错
写入性能	高（LSM Tree）	极高，多数据中心友好
读取性能	点查快，范围扫描极强	点查快，范围扫描一般
适用场景	海量结构化 / 半结构化数据离线计算、时序、日志、画像强一致性要求	高并发、低延迟跨区域、多机房高可用优先
典型架构	Hadoop 生态一套	独立分布式集群

LSM 树：写先往内存写，批量刷磁盘；读从内存 + 磁盘合并读取
写入流程：

1. 先写，日志（WAL/CommitLog） 防止宕机丢失
2. 再写，内存（MemStore/MemTable）
3. 内存满了，批量刷到磁盘，生成只读文件（HFile/SSTable）
4. 后台定期 合并（Compaction） 小文件

为什么写入极快？

- 不修改旧数据，只追加
- 磁盘顺序 I/O，不是随机 I/O
- 内存写 + 批量刷盘

HDFS：分布式文件系统，用来存海量大文件

- 大文件、高吞吐
- 多副本（默认 3 副本）
- 高可靠、可扩展
- 一次写入，多次读取，不支持修改

HDFS 允许将不活跃的数据分配到比较便宜的存储上，用于归档或冷存储。HDFS 可以通过设置存储策

略，将较旧的数据从昂贵的高性能存储上转移到较便宜的存储设备上。对冷数据，选择使用容量大、读写性能不高的存储介质，如机械硬盘；对于热数据，可使用 SSD 硬盘存储。这两种存储在读写效率上差距大。异构特性允许我们对不同文件选择不同的存储介质进行保存，以实现性能的最大化。

“热” 数据：一般新产生的数据被应用程序大量使用，应该采用 SSD 存储

“温” 数据：随着时间的推移，数据访问频率逐渐降低，则可以采用 SAS 盘存储

“冷” 数据：当数据使用率下降得更多，应采用容量大而便宜的 SATA 盘存储

Neo4j ：

专门处理实体间复杂关联关系，高效实现图遍历与深度关联查询，适合社交、风控、知识图谱等场景

核心概念：

- 节点（Node）：实体，比如用户、商品、订单
- 关系（Relationship）：节点之间的连接，有方向
- 属性（Property）：节点 / 关系都可以带键值对
- 标签（Label）：给节点分类，如 :User、:Product

节点 = 实体，关系 = 连接，属性 = 信息，标签 = 分类

存储结构：

- 专门为关系深度遍历优化
- 节点和关系都有物理指针直接指向对方
- 不像关系库要 JOIN，图库遍历极快

数据存储：

- 节点和关系，都在磁盘上用「指针」直接互相指向对方

1. 节点（Node）
- 存：ID、标签、属性
- 最关键：存了指向它所有关系的指针

2. 关系（Relationship）
- 存：ID、类型、属性
- 最关键：指向起点节点、指向终点节点
- 关系是有方向、有类型的

3. 属性（Property）
- 键值对，存在节点或关系上

查询：从一个点出发，沿着关系指针 “顺着线走”，直接找到关联节点，不用 JOIN、不用查表

以查 “张三的朋友的朋友” 举例：

1. 找到起点节点（定位）
- 先通过索引或 ID 找到起始节点：张三

• 这一步和普通数据库差不多
2. 沿着关系 “走过去”（关键！）
• 张三节点里直接存着指向他所有关系的指针
• 系统直接取出：朋友关系
• 每个朋友关系又直接指向朋友节点
→ 这就找到了：张三的朋友
3. 继续沿着关系走（深度遍历）
• 再从 “朋友节点” 出发
• 再走一遍朋友关系
→ 就找到了：朋友的朋友
4. 返回结果
全程没有 JOIN、没有查表、没有比对 ID 只做一件事：顺着指针遍历 → 读取 → 再遍历

Neo4j 查询靠什么快？
1. 节点和关系物理存储相连
2. 用指针直接关联，不需要计算匹配
3. 遍历 = 顺着线走，不是去 “查找”
4. 深度查询性能远远优于关系型数据库

（2023 年 11 月）
阅读下列说明，回答问题 1 至问题 3，将解答填入答题纸的对应栏内。
某国际大型企业拟向企业开发一个在线社交网络服务(SNS)系统，让企业内部的员工能够通过类似社交网络的方式进行工作和自我管理以实现企业内部员工高效、透明、便捷的沟通与协作。其基本的工序包括:
企业内部部门、员工等基本信息的维护和管理
提供常见的 SNS 服务，如 Blog，WiKi,数钱，RSS，即时通信和邮件等
提供社交网络，推荐系统等基本功能提高业务协作除了上述功能性的需求外，还需要满足系统的可拓展、灵活部署以及高性能和高可用性需求企业设计系统小组

问题 1：李工建议采用 SNS 中常见的 NOSQL 数据库技术，原因是传统的关系数据库不能满足实际需求，请用 200 字内说明该系统不能采用传统关系数据库的原因，并给出 NOSQL 数据的基本特点.

传统关系数据库不能满足该 SNS 系统需求，因为 SNS 系统处理的数据类型多样（如非结构化的 Blog 内容、图关系数据），传统关系数据库的固定 Schema 难以支持灵活模式变化，且在高并发场景（如即时通信）下性能瓶颈（如 JOIN 操作复杂）、可扩展性差（水平扩展困难），无法满足高可用和弹性部署需求

NoSQL 基本特点：NoSQL 数据库提供高可扩展性（支持分布式架构）、灵活数据模型（如 Key-Value、文档、列式、图结构）、高性能（优化读写速度），大数据量场景。无 Schema 或弱 Schema（支持半结构化数据）、高可用性（通过复制机制）、易于集群部署，牺牲了 ACID 事务特性（BASE 模型优先）

问题 2：针对李工的建议，王工认为单纯使用 NOSQL 数据库无法完全满足系统要求，考虑到系统功能所涉及到数据类型建议使用混合数据库模型来实现，请用 100 字以内文字说明。混合数据库式如何组织?

针对结构化数据（如员工信息）使用关系数据库（如 MySQL）确保事务一致性
针对半结构化数据（如 Blog 内容）采用文档型 NoSQL 数据库（如 MongoDB）
针对图形数据（如推荐系统）使用图数据库（如 Neo4j）
系统通过 API 层或微服务架构协调，实现数据交互，例如推荐模块访问图数据库，基础服务访问关系数据库

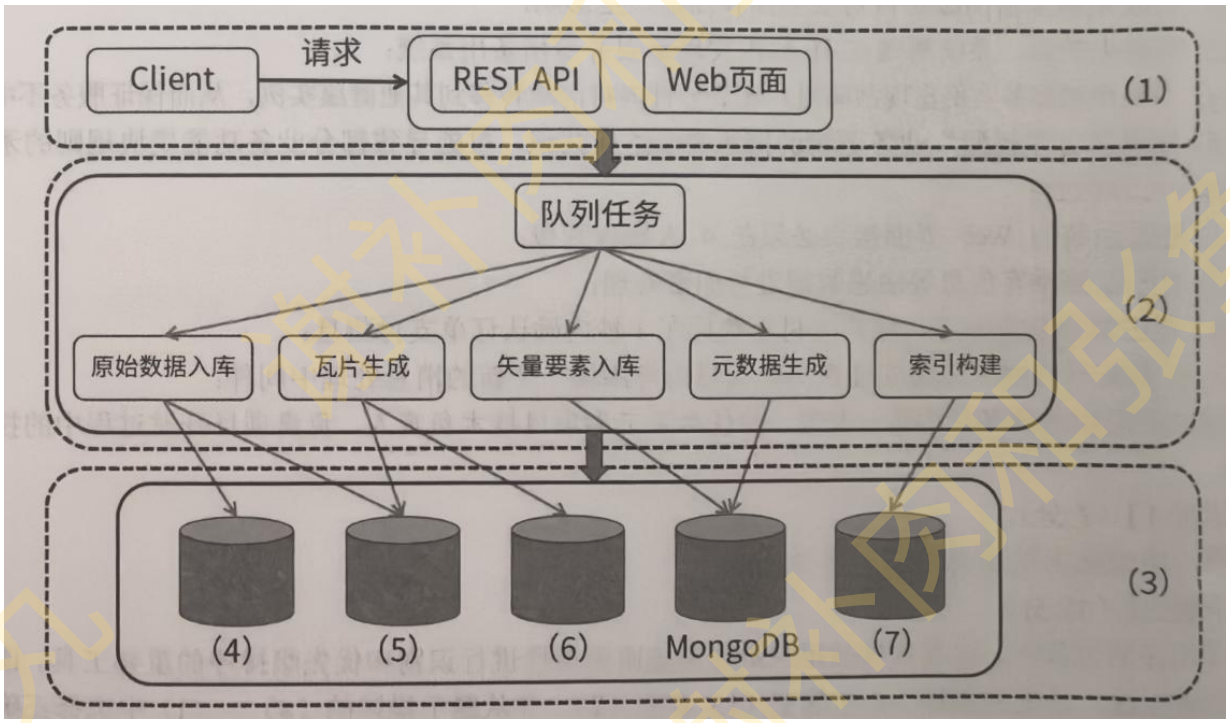
（2024 年 5 月）

【说明】 某公司计划构建一个综合性的地理信息系统 GIS 平台，以支持某市城市空间数据的采集、存储、处理、分析和展示。通过整合卫星遥感影像、无人机航拍数据、地面测量数据等多源信息，项目将建立起高精度的城市三维模型，为城市规划、交通管理、环境保护等领域提供决策支持。同时，GIS 平台还将具备强大的空间分析功能，如地理编码、缓冲区分析、网络分析等，以满足不同用户的专业需求。此外，项目还将注重数据的实时更新与共享，确保 GIS 平台的数据始终保持最新状态，为城市的可持续发展提供有力保障。

【问题 1】（7 分）

补充架构图 5-1，从下列备选项（A）～（G）中选择答案填入图（1）到（7）中。

备选项：(A)处理层；(B)接口层；(C)存储层；(D)HBase；(E)HDFS；(F)ES；(G)MySQL



（1）B（2）A（3）C（4）G（5）E（6）D（7）F

【问题 2】（10 分）

简要说明 MongoDB 是如何实现空间矢量存储的，以及 MongoDB 对于存储非关系型数据的优势是什么。

MongoDB 作为一个文档型数据库，用于将非结构性数据以文档的形式进行存储。MongoDB 可以通过 GeoJSON 格式来存储空间矢量数据，这是一种基于 JSON 的地理空间数据交换格式。在 MongoDB 中，

可以使用 geometry 字段来存储几何形状，使用 coordinates 字段来存储几何形状的坐标。
MongoDB 对于存储非关系型数据的优势：①MongoDB 采用分布式文件系统和空间索引，提高了矢量空间数据的存储和处理效率；②MongoDB 支持多种空间分析和查询操作，可对数据进行深度分析和挖掘；③MongoDB 扩展性好，可以根据业务需要增加字段，可以表达丰富的地理信息。

【问题 3】（8 分）
请说明 HDFS 使用热数据、温数据和冷数据存储的原因。
HDFS 允许将不活跃的数据分配到比较便宜的存储上，用于归档或冷存储。HDFS 可以通过设置存储策略，将较旧的数据从昂贵的高性能存储上转移到较便宜的存储设备上。对冷数据，选择使用容量大、读写性能不高的存储介质，如机械硬盘；对于热数据，可使用 SSD 硬盘存储。这两种存储在读写效率上差距大。异构特性允许我们对不同文件选择不同的存储介质进行保存，以实现性能的最大化。
“热” 数据：一般新产生的数据被应用程序大量使用，应该采用 SSD 存储
“温” 数据：随着时间的推移，数据访问频率逐渐降低，则可以采用 SAS 盘存储
“冷” 数据：当数据使用率下降得更多，应采用容量大而便宜的 SATA 盘存储。

知识点补充：

消息队列 MQ：分布式系统中的异步通信中间件，用来在服务 / 应用之间传递消息，实现异步、解耦、削峰、流量控制

核心作用：
- 异步处理：不需要同步等待下游执行完成，提高响应速度
- 应用解耦：上游不关心下游是谁、有多少、是否正常，只发消息
- 流量削峰：高并发流量先进入队列，下游按能力消费，防止压垮数据库

核心角色：
- 生产者 Producer：发消息
- Broker：MQ 服务器，存储转发
- 消费者 Consumer：收消息、处理
- 主题 Topic / 队列 Queue：消息分类

消费模式：
- 点对点模式（Queue）：一条消息只被一个消费者消费
- 发布 / 订阅模式（Topic）：一条消息可以被多个订阅者消费

存在问题：
1. 消息重复消费
原因：
- 网络超时
- 消费者异常重启

- 消息重投
- 结果：同一条消息被处理多次 → 数据错误

解决方案：幂等性

- 唯一 ID + 幂等表 — 消费前先查幂等表：是否已处理
- 数据库唯一约束 — 订单、流水号等字段建唯一索引，重复消息插入时会报重复键错误
- 状态机判断 — 数据带状态：待处理 → 处理中 → 已完成，只有状态为 “待处理” 才允许执行

2. 消息丢失

可能丢失的三个位置：

1. 生产者 → MQ 丢失
2. MQ 自己丢失
3. MQ → 消费者丢失

解决方案：

① 生产者

- 同步 / 异步确认（ack）
- 事务消息

② MQ 服务端

- 持久化（磁盘）
- 主从、集群

③ 消费者

- 先处理业务，再确认 ack
- 禁止自动确认

3. 消息顺序性

场景：订单创建 → 支付 → 发货，必须按顺序

问题：多线程 / 多队列 → 乱序

解决：

- 同一业务 ID（如订单 ID）哈希路由到同一个队列
- 单线程消费该队列
- 允许局部有序，不追求全局有序

4. 消息堆积

原因：

- 消费太慢
- 消费者宕机
- 生产者流量暴增

危害：

- 数据不实时
- 队列占满，新消息进不来

解决：

- 增加消费者数量
- 优化消费逻辑（异步、批量）

- 清理无效消息
- 降级、限流

Elasticsearch：基于全文检索技术的分布式搜索与分析引擎，处理大规模文本数据支持自然语言查询

分词原理流程：

1. 字符过滤器（Character Filters）：

这一环节会对原始文本做预处理，比如去掉或替换特殊字符，或者把 HTML 实体转换成可读文字。这一步是可选的，但能让输入更干净

2. 分词器（Tokenizer）：

这是分词的核心步骤，它会按照特定规则把文本切割成一个个词元（Token）。例如，标准分词器会依据空格、标点来分割，同时保留这些切分出来的词

3. 标记过滤器（Token Filters）：

分词结果会进一步被加工，比如全部转成小写，去掉停用词（如“the”、“and”），或做词干提取等，让结果更适合搜索

分词器（Tokenizer）：

把原始文本切割成一个个最小的词元。词元才是被 Elasticsearch 真正拿来索引、存储和搜索的基本单位

分词器	输入示例	输出分词结果	特点说明
standard（标准分词器）	Hello,thisisElasticsearch!	["hello", "this", "is", "elasticsearch"]	去掉标点，统一转小写，适合自然语言处理
whitespace（空白分词器）	HelloWorld, 你好 Elasticsearch	["Hello", "World,", "你好", "Elasticsearch"]	仅按空格切分，标点保留
keyword（关键词分词器）	OrderID-12345	["OrderID-12345"]	不切分，整个输入作为一个词元，常用于 ID、手机号等精确匹配
uax_url_email（URL/Email 分词器）	请联系 support@example.com 或访问 https://cheko.cc	["请", "联系", "support@example.com", "或", "访问", "https://cheko.cc"]	能正确识别邮箱和 URL，把它们当作整体

倒排索引：首先对文档数据按照 id 进行索引存储，然后对文档中的数据分词，记录对词条进行索引，并记录词条在文档中出现的位置

- 假设我们有以下三个文档：

文档 1：苹果是一种水果

文档 2：香蕉是一种热带水果

文档 3：苹果和香蕉都是水果

• 倒排索引：苹果：[1，3]，香蕉：[2，3]

这个倒排索引中，如果我们搜索“苹果”，索引列表就会告诉我们“苹果”包含在文档 1 和文档 3 内

相关性算法：相关性得分决定搜索结果排序的关键指标，它衡量文档与查询条件的匹配程度。简单来说，得分越高，说明文档越符合用户的查询需求，常用的算法有 TF-IDF 和 BM25

因素	含义	影响方式	举例说明
查询类型	不同的查询方式决定相关性计算规则	match 会计算相关性；term 更偏向精确匹配，不参与复杂评分	搜索“手机”：match 会考虑词频和相关性，term 只找完全一致的“手机”
词频（TF）	一个词在文档中出现的次数	出现越频繁，得分越高	文档 A 出现 10 次“手机”，文档 B 出现 1 次→A 得分更高
逆文档频率（IDF）	一个词在所有文档中出现的稀有程度	越稀有的词贡献得分越高	“iPhone17” 比 “手机” 更稀有，因此区分度更强
文档长度归一化（Norms）	考虑关键词在短文档 vs 长文档中的代表性	短文档中出现一次的关键词更有代表性	标题 “iPhone17 发布” 中出现一次 “iPhone”，比长评测文档更突出
字段权重（Boost）	手动调整字段的重要性	被 Boost 的字段匹配会得到更高分	给 title 设置 boost=2，比 description 更优先

这里的苹果：[1，3]，香蕉：[2，3]有些类似向量的概念，所以引入向量数据库

向量数据库：

• 向量是什么？

文本、图片、音频、视频通过嵌入模型（Embedding）转为一串数字数组

语义越接近，向量空间距离越近

向量 = 把文字、图片、声音、视频变成一串数字

比如：

“苹果” → 变成 [0.12, 0.56, 0.89, 0.23, ...]

一张猫的图片 → 变成 [0.33, 0.76, 0.11, ...]

• 向量库解决什么问题？

传统数据库无法高效做高维相似度检索

暴力比对太慢 O (n)，向量库用索引做到 O (logn)

• 这个向量是怎么确定的呢？是由模型确定的

比如它内置了很多隐藏的维度（人不用管）：

是不是水果？

能不能吃？

是不是交通工具？

大不大？

...（总共几百上千个维度）

它给每个词打分，就变成了向量：

苹果

是水果：1.0

能吃：0.9

是交通工具：0.0

→ 向量：[1.0, 0.9, 0.0, 0.2, ...]

• 向量数据库怎么查？

1. 你输入一句话 → 转成向量
2. 数据库计算距离
3. 返回距离最近的结果

• 距离怎么算？

1. 欧氏距离（L2）
2. 余弦相似度
3. 点积（内积）

ES 与向量数据库：

维度	Elasticsearch（ES）	向量数据库
靠什么查	关键词、分词、倒排索引	向量、距离计算、相似性
理解什么	字面文字	语义、意思、内容
匹配方式	精确 / 模糊匹配	语义相似匹配
擅长	全文检索、日志、统计	AI 检索、推荐、以图搜图、RAG
数据类型	文本为主	文本、图片、音频、视频都可以
典型场景	百度、电商关键词搜索	智能问答、AI 找资料、相似推荐

最本质的区别：

ES 是「关键词检索」

认识字，不理解意思

你写什么，它查什么

向量数据库是「语义检索」

不看字，理解内容

你问什么意思，它查什么意思

拓展知识，Newsql：

NewSQL 是一类融合了传统关系型数据库（SQL）的强事务一致性（ACID）与 NoSQL 分布式水平扩展能力的新型数据库技术，专为高并发、海量数据的 OLTP（联机事务处理）场景设计

- 核心目标：在分布式环境下，同时满足 ACID 事务、标准 SQL 兼容、水平线性扩展、高可用
- 解决痛点：
  1. 传统 SQL：单机 / 主从，扩展性差、难扛高并发
  2. NoSQL：分布式，但弱事务、不支持完整 SQL / 复杂查询
  3. NewSQL：两者优势结合，弥补短板
- 核心特性：
  1. 分布式架构（Shared-Nothing）
     数据自动分片（Sharding）、多节点集群
     水平扩容：加机器即可提升性能，线性扩展
     无单点瓶颈
  2. 强事务 ACID（跨节点）
     支持分布式事务、跨分片事务一致性
     常用协议：Raft / 2PC / MVCC
  3. 完整 SQL 兼容
     支持 ANSI SQL、JOIN、子查询、索引、事务
     兼容 MySQL/PostgreSQL 协议，低迁移成本
  4. 高可用 & 容灾
     多副本、自动故障切换（Failover）
     RPO=0、RTO 秒级
  5. 面向 OLTP
     高并发读写、低延迟、强一致

特性	NewSQL	传统 SQL	NoSQL
数据模型	关系型	关系型	非关系
扩展方式	水平（分布式）	垂直（升级硬件）	水平（分布式）
事务	ACID（跨节点）	ACID（单机/主从）	最终一致性（BASE）
SQL	完整兼容	完整兼容	弱支持/不支持
典型场景	金融交易、电商核心、支付	企业 ERP、中小应用	日志、用户行为、缓存、大数据
代表产品	TiDB、OceanBase、Spanner、CockroachDB	MySQL、PostgreSQL	MongoDB、Redis、Cassandra

- 如何实现分布式事务 / 跨分片事务一致性

全局时间戳 + 两阶段提交（2PC） + MVCC + 一致性协议（Raft）= 跨分片 ACID

1. 先拿到全局唯一时间戳（TSO）
   - 集群里有一个全局授时服务（TiDB 的 PD、OceanBase 的 RootService）
   - 所有事务统一获取单调递增的时间戳：保证事务顺序、实现 MVCC 多版本并发控制、避免分布式环境下时钟不一致问题

2. 两阶段提交 2PC

跨分片事务会涉及多个节点，必须分两步提交：

1）Prepare 阶段（表决）

- 协调者向所有分片节点发请求
- 各节点执行语句、锁定资源、写 redo log
- 全部回复 “OK” 才能进入下一阶段

2）Commit 阶段（提交）

- 协调者通知所有节点提交
- 所有节点一起提交，事务生效
- 任意节点 Prepare 失败 → 全部回滚

3. MVCC（通过给数据生成多个历史版本，实现 “读不加锁、写不阻塞读、读不阻塞写”，大幅提升并发性能，同时保证事务隔离）避免锁冲突

- 多版本存储，读写不阻塞
- 用时间戳判断可见性
- 保证隔离级别（通常是 RC / SI）

4. 底层一致性协议兜底，每个分片本身是 Raft 组：

- 数据写入必须多数副本确认
- 单机挂了不影响分片数据一致性
- 最终做到：分片内强一致 + 分片间原子提交

• 如何实现自动故障切换

1. 故障感知（心跳机制）

- 节点之间互相发心跳
- 节点 / 分片 / 网络异常会被 ** 管理节点（PD/RootService）** 立即发现
- 判定超时后进入故障流程

2. Leader 自动选举，每个分片是一个 Raft/Paxos 组：

- 原 Leader 挂了
- 剩余 Follower 自动选举新 Leader
- 选举过程秒级完成
- 满足：大多数节点存活即可选举成功

3. 元数据自动更新

- 选举出新 Leader 后
- 管理节点更新路由表
- 告诉 Proxy / 客户端新的访问地址

4. 流量自动切换

5. 数据不丢失（RPO=0）

- 写入必须同步到多数副本
- 主机挂掉，副本已有完整数据
- 不会丢事务，不会丢数据

数据流图 DFD：

- 数据流：具有名字和流向的数据，
- 加工（处理）：对数据流的变换， 
- 数据存储：可访问的存储信息， 
- 外部实体：信息生产者或消费者， 

平衡原则：

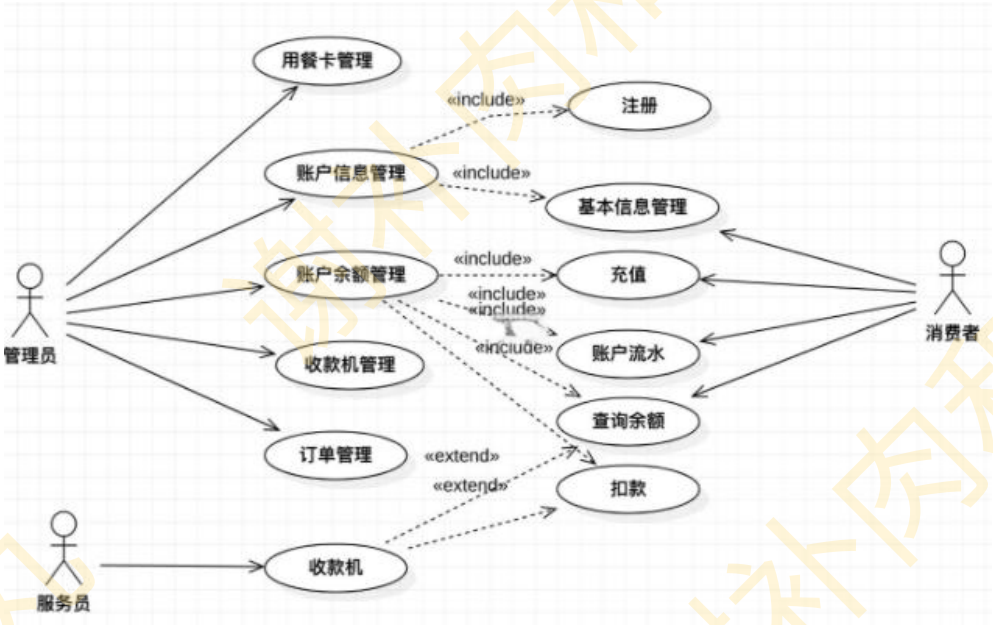
父图（上层数据流图）与子图（下层数据流图）平衡

- 个数一致：两层数据流图中的数据流个数一致
- 方向一致：两层数据流图中的数据流方向一致

不平衡情况：

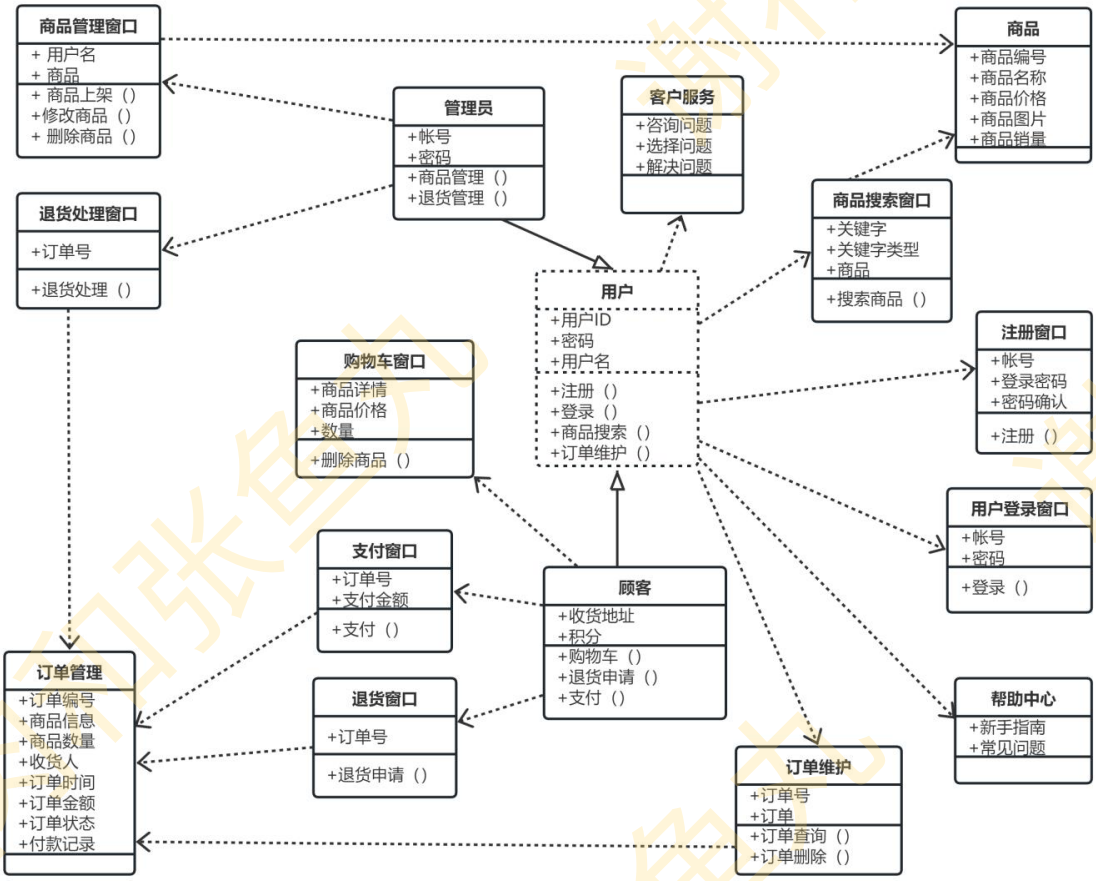
- 黑洞：加工只有输入没有输出
- 奇迹：加工只有输出没有输入
- 灰洞：加工不足以产生输出

UML - 用例图：



- 包含<<include>>：一个用例必须使用另一个用例的功能。结账用例必须包含验证支付信息用例。箭头指向必须的
- 扩展<<extend>>：一定条件下扩展用例会被添加到基础用例中。购物车的基础功能可通过“使用优惠券”用例扩展（仅在满足使用优惠券条件时触发）。箭头指向基础的
- 泛化（空心三角）：子用例继承父用例的行为并可以覆盖或扩展其功能。“线上支付”泛化为“信用卡支付”和“支付宝支付”。三角指向父类

UML - 类图：

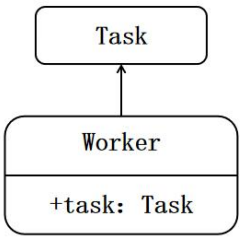


- 类关系：关联强度，强到弱

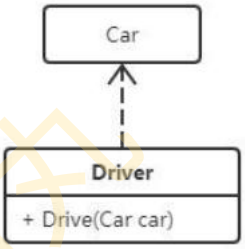
  饭食组局关羽

  泛化=实现>组合>聚合>关联>依赖

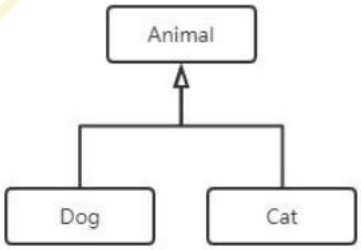
- 关联关系：两个事物间存在明确、稳定的语义联系



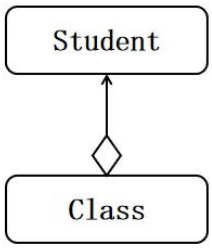
- 依赖关系：两个类 A 和 B，若 B 的变化可能引起 A 的变化，A 依赖于 B



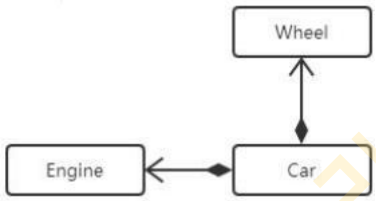
- 泛化关系：父类与子类的关系



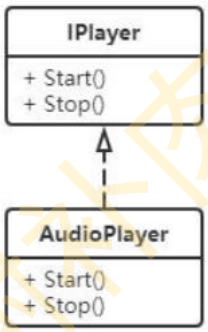
- 聚合关系：“部分”与“整体”生命周期可不同



- 组合关系：“部分”与“整体”生命周期相同



- 实现关系：类实现接口操作，箭头指向接口



各种图对比：

图类型	主要关注点	典型用途
用例图	系统外部用户（参与者）与系统之间的交互	捕获需求、确定系统边界、展示用户价值
类图	系统内部类、属性、方法及类之间的关系	定义系统的结构设计、支撑代码实现
数据流图	数据的加工、流动、存储过程	需求分析阶段，建模系统数据处理逻辑
活动图	工作流、活动之间的顺序、分支、并行	表达业务流程或算法步骤
流程图	操作步骤及逻辑判断	表达算法逻辑、简单过程
状态图	对象的状态变化及触发条件	建模复杂对象的生命周期
顺序图	对象间消息的时间顺序	展示场景交互流程，强调消息调用先后
通信图	对象之间的连接关系及消息传递	表达协作结构和对象间关系

面向对象设计原则：

- 单一职责原则（S）：类应该只有一个设计目的
- 开闭原则（O）：对扩展开放，对修改关闭
- 里氏替换原则（L）：所有引用基类（父类）的地方，必须能够透明地使用其子类的对象，而程序的行为不发生改变
- 接口隔离原则（I）：客户端不应该被强迫依赖它不需要的接口。一个类对另一个类的依赖应该建立在最小的接口上
- 依赖倒置原则（D）：抽象不应该依赖于细节，细节应该依赖于抽象
- 组合/聚合复用原则：通过组合或聚合关系复用已有对象，比继承更灵活，耦合度低
- 最少知识原则（迪米特法则）：软件实体应尽量少与其他实体相互作用

污水池反模式：

分层架构中最典型、最常见的设计缺陷，核心是分层空转、请求穿透、逻辑空洞

典型场景（四层架构为例）

标准四层：表现层 → 业务层 → 持久层 → 数据层

- 正常流程（有逻辑）

表现层 → 业务层（做校验、计算、规则、组合）→ 持久层 → 数据层

- 污水池流程（空转）

表现层 → 业务层（仅调用持久层，无逻辑）→ 持久层 → 数据层

解决方案：

1. 合并空层（最直接）

业务层与持久层合并，减少不必要的中间层

小系统 / 简单场景：采用三层架构（表现层 → 业务持久层 → 数据层）

2. 开放层访问（允许跨层调用）

对简单查询类请求，允许表现层直接调用持久层

前提：做好权限与边界控制，避免滥用

3. 逻辑下沉 / 上移

把简单逻辑下沉到持久层（如简单过滤、排序）

或上移到表现层（如简单展示逻辑）

4. 服务合并与聚合

对频繁穿透的场景，提供聚合服务，减少层间跳转

例如：getUserWithOrder() 一次返回多表数据

5. 架构重构（根治）

重新梳理职责，让每层真正承担对应逻辑

引入领域驱动（DDD），明确业务边界与逻辑归属

领域驱动（DDD）：

- 以业务领域为核心，通过领域专家与开发人员协作，统一业务语言
- 把复杂业务拆解成清晰的领域模型，指导软件设计与架构
- 解决传统数据驱动、过程驱动导致的业务逻辑混乱、难以维护问题

核心概念：

1. 通用语言：
- 业务、产品、开发、测试用同一套术语
- 减少沟通误差，保证模型与业务一致

2. 限界上下文：
- DDD 最关键的拆分依据
- 一个业务边界内，模型语义唯一
- 多个上下文之间通过上下文映射交互

3. 实体：
- 有唯一标识，状态会变
- 例如：订单、用户、商品

4. 值对象：
- 无唯一标识，只描述属性
- 不可变，例如：地址、金额、坐标

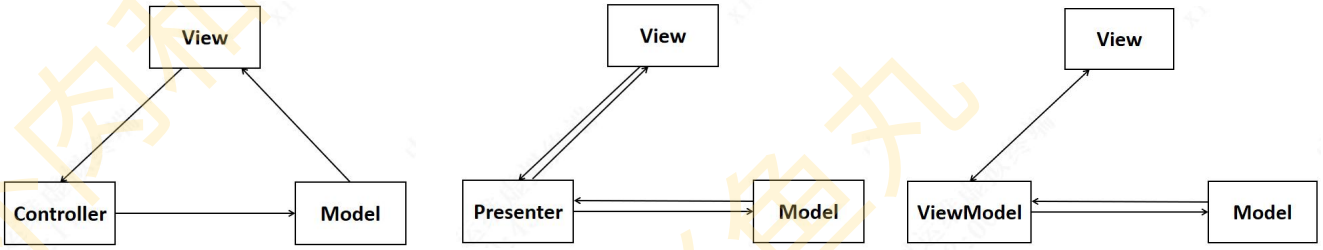
5. 聚合 & 聚合根：
- 聚合：一组相关对象的集合，是数据修改的单元
- 聚合根：聚合对外的唯一入口，控制事务与一致性
- 作用：保证数据一致性，降低复杂对象关联复杂度

6. 领域服务：
- 不属于某个实体 / 值对象，但属于业务规则
- 协调多个领域对象完成业务逻辑

7. 领域事件：
- 领域内发生的一件事
- 用于解耦、最终一致性、异步流程
- 非常适合：订单创建、支付完成、库存扣减等场景

表现层层次架构：

维度	MVC （模型 - 视图 - 控制器）	MVP （模型 - 视图 - 呈现器）	MVVM （模型 - 视图 - 视图模型）
核心组件	模型（Model） 视图（View） 控制器（Controller）	模型（Model） 视图（View） 呈现器（Presenter）	模型（Model） 视图（View） 视图模型（ViewModel）
概念	模型：数据存储与业务逻辑 视图：用户界面展示 控制器：处理用户输入，协调模型与视图	模型：数据存储与业务逻辑 视图：用户界面交互接口 呈现器：处理逻辑，控制视图更新	模型：数据存储与业务逻辑 视图：绑定数据的 UI 视图模型：数据转换双向绑定
数据流	单向：视图→控制器→模型→视图更新（通过控制器）	双向：视图↔ 呈现器↔ 模型（呈现器主动更新视图）	双向：视图↔ 视图模型↔ 模型（自动同步，基于绑定）
数据交互	视图可直接调用控制器，可能访问模型	视图通过接口与呈现器通信，不直接访问模型	视图通过数据绑定与视图模型交互，无直接逻辑调用

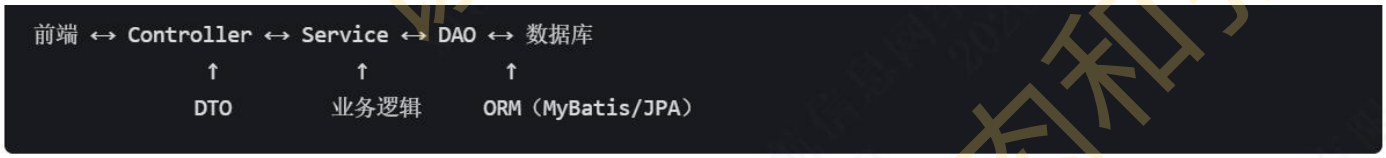


1. MVC 流程（单向驱动，控制器是调度者）
   - 用户交互：用户在 View（视图）上进行操作（如点击按钮）
   - 请求转发：View 将用户操作传递给 Controller（控制器）
   - 业务处理：Controller 调用 Model（模型）执行相应的业务逻辑或数据操作
   - 数据更新：Model 完成处理后，通知 View 数据已更新
   - 视图刷新：View 拉取 Model 数据，将最新数据展示给用户
   - 特点：View 与 Model 存在耦合，Controller 是核心调度者，数据流为单向
2. MVP 流程（双向交互，Presenter 是核心）
   - 用户交互：用户在 View 上操作，View 不直接处理逻辑，而是将事件传递给 Presenter（呈现器）
   - 业务处理：Presenter 调用 Model 执行业务逻辑
   - 数据返回：Model 处理完成后，将结果返回给 Presenter
   - 视图更新：Presenter 主动调用 View 的接口方法，更新界面显示
   - 特点：View 与 Model 完全解耦，View 是被动的，所有逻辑由 Presenter 控制，便于单元测试
3. MVVM 流程（双向绑定，自动同步）
   - 数据绑定：View（UI）与 ViewModel（视图模型）通过数据绑定建立关联
   - 用户交互：用户操作 View，触发数据变化，自动同步到 ViewModel
   - 业务处理：ViewModel 调用 Model 执行业务逻辑
   - 自动更新：Model 数据变化后，ViewModel 自动更新，再通过绑定机制同步到 View

数据访问层设计：

模式名称	核心思想	特点
在线访问模式	应用程序与数据库保持持续连接，实时执行数据操作	实时性强，直接操作数据库，但依赖网络和连接稳定性
DAO （数据访问对象）	将数据访问逻辑与业务逻辑分离，通过接口封装数据库操作	解耦数据层与业务层，支持不同数据库切换，提高代码复用性
DTO （数据传输对象）	用于跨进程 / 网络传输数据的轻量级容器，无业务逻辑	减少网络传输次数，可序列化，独立于数据源结构
离线数据模式	数据从数据源获取后，按预定义结构存储在本地，与数据源断开连接	离线处理数据，支持 XML/JSON 格式转换，不依赖数据库连接
ORM (对象/关系映射)	ORM 在关系型数据库和对象之间建立映射，应用程序可以像操作对象一样操作数据库，它可以大幅减少数据访问层的开发工作，专注于实现业务逻辑	面向对象编程，自动生成 SQL，支持复杂查询优化

模式/技术	核心作用	优点	缺点	适用场景
DAO 数据访问对象	封装数据访问，分离业务与 DB	1. 解耦业务与数据层 2. 易于切换数据源 3. 便于测试、维护	1. 代码量略大 2. 需编写接口+实现	企业应用、标准分层架构、需要统一数据访问
DTO 数据传输对象	层间/远程数据传输，减少调用	1. 减少网络交互次数 2. 不暴露实体，安全 3. 按需传输字段	1. 类数量增多 2. 维护成本略升	分布式、微服务、前后端分离、远程调用
ORM 对象关系映射	对象、表自动映射	1. 面向对象操作 DB 2. 开发效率高 3. 屏蔽数据库差异	1. 复杂查询性能一般 2. 多表联查不灵活	中小型项目、快速开发、业务逻辑复杂
在线访问模式	实时访问数据库，即用即查	1. 数据实时一致 2. 无数据同步问题 3. 架构简单	1. 并发压力大 2. 网络依赖强	实时性要求高、数据强一致、小并发
离线数据模式	先缓存/同步，本地使用	1. 减轻 DB 压力 2. 断网可用、响应快 3. 高并发友好	1. 数据存在延迟 2. 需要同步机制	高并发、移动端、报表、离线操作



- **DTO**：Controller ↔ Service 之间传数据
- **DAO**：专门负责数据访问
- **ORM**：是 DAO 层的**实现技术**

云原生架构：

云原生架构原则：

原则	描述
服务化原则	项目规模大时拆分服务，提升复用性与流量治理能力
弹性原则	系统自动伸缩，适配业务需求
可观测性原则	通过日志、链路跟踪等掌握系统状态
韧性原则	抵御软硬件异常，保障系统稳定
全流程自动化原则	自动化交付提升效率、降低风险
零信任原则	基于身份访问控制，替代传统网络边界安全
持续演进原则	架构适应技术与业务变化，渐进式升级

云原生架构模式：

架构模式	核心定义
服务化架构模式（SOA / 微服务）	按业务域或功能模块拆分应用，每个模块作为独立服务运行，通过接口契约（API / 协议）进行交互常结合 DDD（领域驱动设计）进行服务边界划分，用 TDD 提升质量，并通过容器化与 CI/CD 管理快速迭代优点是灵活扩展、独立部署、降低耦合；缺点是分布式环境下治理成本高典型场景：电商订单、支付、库存拆分为微服务
Mesh 化架构模式（ServiceMesh）	将中间件能力（RPC 通信、负载均衡、限流熔断、安全认证等）从业务进程中剥离，由独立的 Sidecar/Proxy 组件（Mesh 进程）统一处理这样业务代码只需专注逻辑实现，而网络通信、服务治理、可观测性由 Mesh 负责优点是统一治理、解耦复杂逻辑，缺点是引入额外性能开销典型实现：Istio、Linkerd
Serverless 模式	应用无需管理底层服务器，开发者只需编写业务逻辑代码，由云平台自动完成部署、运行、弹性伸缩和关闭应用一般通过事件或流量触发执行，按调用次数或运行时间计费优势是极大降低运维成本、按需伸缩，劣势是冷启动延迟和厂商锁定风险典型场景：图片处理、日志分析、IoT 数据采集
存储计算分离模式	将计算层与存储层解耦：计算节点保持无状态，数据全部存储在分布式存储 / 云存储中有状态服务通过写前日志（WAL）+ 快照（Snapshot）机制实现恢复优点是弹性伸缩、成本优化；缺点是计算与存储分离带来延迟问题典型应用：Snowflake、TiDB、云数仓
分布式事务模式	在跨数据库 / 跨服务的事务中保持数据一致性，常见模式有：XA 模式：两阶段提交，强一致，但性能差消息最终一致性：通过消息队列异步保证，性能好但一致性延迟 TCC 模式：应用层显式 Try/Confirm/Cancel，灵活但侵入性强 SAGA 模式：将长事务分解为本地事务 + 补偿操作，适合复杂业务流程 SEATA AT 模式：通过代理数据库操作实现自动回滚，性能较好，但受限于数据库类型
可观测架构	通过三大核心能力实现对系统运行状态的全面感知：Logging：多级日志，记录系统行为和错误 Tracing：分布式调用链路追踪，快速定位性能瓶颈 Metrics：系统指标采集与量化（QPS、响应时间、并发度等）结合 SLO/SLA 定义服务目标，用于监控、报警和运维优化典型工具：Prometheus、Grafana、ELK、Jaeger
事件驱动架构（EDA）	系统以事件为核心进行交互，事件作为独立数据单元，包含 Schema，可被生产者发布、消费者订阅具备 QoS 保证和失败补偿机制优点是服务解耦、异步处理、可扩展；适合高并发、异步业务场景，如订单完成触发发货、数据库变更触发缓存更新常用技术：Kafka、Pulsar、EventBridge

容器技术：

容器 vs 虚拟机：

对比维度	容器	虚拟机
运行原理	基于操作系统层面的虚拟化，共享宿主机内核，直接运行在操作系统之上	通过硬件虚拟化技术，模拟出完整的硬件环境，运行独立的操作系统
资源隔离方式	利用 cgroups 进行资源限制（如 CPU、内存、磁盘 I/O 等），通过 namespace 实现命名空间隔离（如进程、网络、文件系统等），不同容器共享内核但彼此隔离	依靠硬件虚拟化技术，在不同虚拟机实例间实现完全的资源隔离，每个虚拟机有独立的内核和硬件资源
启动速度	启动非常快，一般在秒级。因为无需启动完整操作系统，仅需加载应用及其依赖	启动相对较慢，通常需要数十秒到数分钟，要加载整个操作系统
资源占用	占用资源少，仅需运行应用所需的资源，因为共享宿主机内核	资源占用多，每个虚拟机都有独立的操作系统和完整硬件资源模拟，开销大
应用移植性	移植性好，将应用及其依赖打包在镜像中，环境一致性高，可在不同支持容器的环境中快速部署	移植有一定难度，受虚拟硬件环境和操作系统影响，迁移时需考虑兼容性
隔离性	隔离性相对较弱，虽能实现进程、网络等隔离，但共享内核，存在一定安全风险（如内核漏洞可能影响多个容器）	隔离性强，每个虚拟机相互独立，一个虚拟机出现问题基本不影响其他虚拟机
管理复杂度	相对简单，主要管理容器镜像、容器生命周期（创建、启动、停止、删除等），通过编排工具可实现集群管理	管理复杂，需管理多个虚拟机的操作系统安装、配置、补丁更新等，以及虚拟硬件资源的分配和管理

docker vs k8s：

对比维度	容器	容器编排技术
定义	一种将应用及其依赖项打包成独立单元的技术，可在不同环境中运行，共享宿主机内核	用于自动化部署、管理和扩展容器化应用程序的技术，能管理多个容器组成的集群
核心功能	隔离应用运行环境，实现应用的快速部署和迁移	容器集群管理、资源调度、服务发现、负载均衡、自动伸缩、故障恢复等
代表工具	Docker	Kubernetes（k8s）
部署难度	部署单个容器相对简单，通过拉取镜像并启动容器即可	部署和配置容器编排系统较为复杂，涉及多个组件的安装、配置和集群初始化
适用场景	适合部署单一、简单的应用，或作为微服务架构中单个服务的运行载体	适用于大规模、复杂的容器化应用集群管理，如大型互联网应用、企业级分布式系统，这些系统需要高效的资源调度、自动伸缩和服务治理等功能
管理粒度	主要管理单个容器的生命周期，如创建、启动、停止、删除，以及容器内应用的运行状态	从集群层面管理多个容器，包括容器的分组、调度到不同节点、监控容器的健康状态、动态调整容器数量等

常用指令：

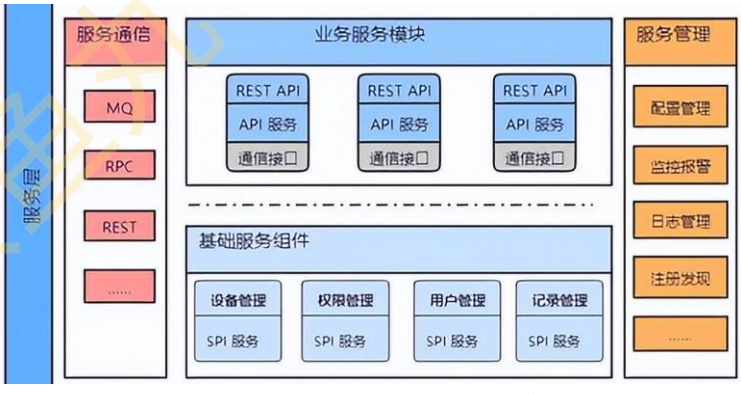
技术	常用指令	指令说明
Docker	docker pull 镜像名:版本	拉取镜像（如 docker pull nginx:latest）
	docker images	查看本地所有镜像
	docker build -t 镜像名:版本 路径	基于 Dockerfile 构建镜像
	docker rmi 镜像 ID/镜像名	删除指定镜像
	docker push 镜像名:版本	推送镜像到仓库（需先登录）
	docker ps / docker ps -a	查看运行中容器 / 查看所有容器（含停止的）
	docker start/stop/restart 容器 ID/容器名	启动 / 停止 / 重启容器
	docker exec -it 容器 ID/容器名 /bin/bash	进入容器交互终端
	docker rm 容器 ID/容器名	删除指定容器（需先停止）
	docker logs -f 容器 ID/容器名	实时查看容器日志
k8s	kubeadm init	初始化 k8s 主节点
	kubectl get pods	查看所有 Pod 状态
	kubectl get services	查看所有服务（Service）
	kubectl get deployments	查看所有部署（Deployment）
	kubectl get nodes	查看集群所有节点
	kubectl describe pod Pod 名称	查看 Pod 详细信息（排障常用）
	kubectl delete pod/deploy/svc 名称	删除指定资源
	kubectl scale deploy 部署名 --replicas=数量	扩缩容 Pod 数量
	kubectl exec -it Pod 名称 -- /bin/bash	进入 Pod 内容器终端
	kubectl logs Pod 名称	查看 Pod 日志
	kubectl get events	查看集群事件（排障关键）

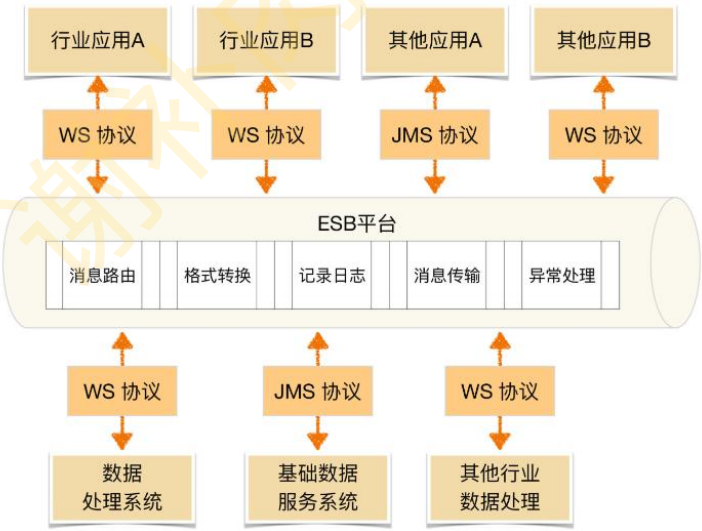
微服务与 SOA：

微服务：一种架构模式，将单一应用程序开发为一组小型服务，每个服务运行在其独立的进程中，服务间通过轻量级机制（通常是 HTTP/REST API）通信，并且独立部署

SOA（面向服务架构）：一种企业级的架构风格，将应用程序的不同功能单元（服务）通过中立的标准接口（通常基于 SOAP/WSDL）进行定义和发布，使得这些服务可以以统一的方式进行交互

特性	微服务	SOA（面向服务的架构）
服务粒度	服务粒度极为精细，每个微服务通常仅负责单一的业务功能，例如在电商系统中，商品展示、订单处理、支付等功能可分别由独立的微服务实现。这种细粒度的划分使得每个服务专注于特定任务，便于开发、维护和独立扩展。	服务粒度相对较大，一个服务可能涵盖多个业务功能。比如在企业资源规划（ERP）系统中，一个服务可能同时包含采购管理和库存管理的相关功能，其整合程度更高。
通信方式	主要使用 HTTP/RESTful API 进行通信，这种方式具有良好的语言和平台无关性，不同语言开发的微服务之间可以轻松交互，并且 RESTful 风格简单易用，便于与各种客户端进行集成。	通信方式更为多样化，除了 HTTP 外，还常使用 SOAP（简单对象访问协议）等协议。SOAP 具有严格的消息格式和标准，适用于对数据交互规范性和安全性要求较高的企业级应用场景。
数据管理	每个微服务通常拥有自己独立的数据库，形成数据源唯一的特性，确保数据的独立性和安全性。不同微服务的数据库可以根据自身业务需求选择合适的数据库类型，如关系型数据库、非关系型数据库等。	数据管理方式较为灵活，可能采用多个服务共享同一个数据库的方式，或者通过企业服务总线（ESB）进行数据交换和整合，实现不同服务间的数据交互和协调。
架构风格	遵循去中心化的架构风格，不存在中心化的服务总线。各个微服务独立部署、运行和管理，通过轻量级的通信机制相互协作，具有较高的灵活性和可扩展性。	可能包含中心化的企业服务总线（ESB），ESB 充当服务间通信和数据交换的枢纽，负责处理服务的注册、发现、路由以及消息转换等功能，在一定程度上增强了服务间的集成和管理能力。
复杂性	由于服务的独立性和去中心化特性，微服务架构的系统复杂性相对较低。每个微服务可以独立开发、测试和部署，降低了整体系统的耦合度。但同时，微服务数量众多也会带来服务治理、监控和运维等方面的挑战。	由于服务间可能存在较强的耦合关系，并且中心化的 ESB 增加了架构的复杂性，使得 SOA 架构的系统在设计、开发和维护上相对复杂，对团队的技术能力和管理水平要求较高。

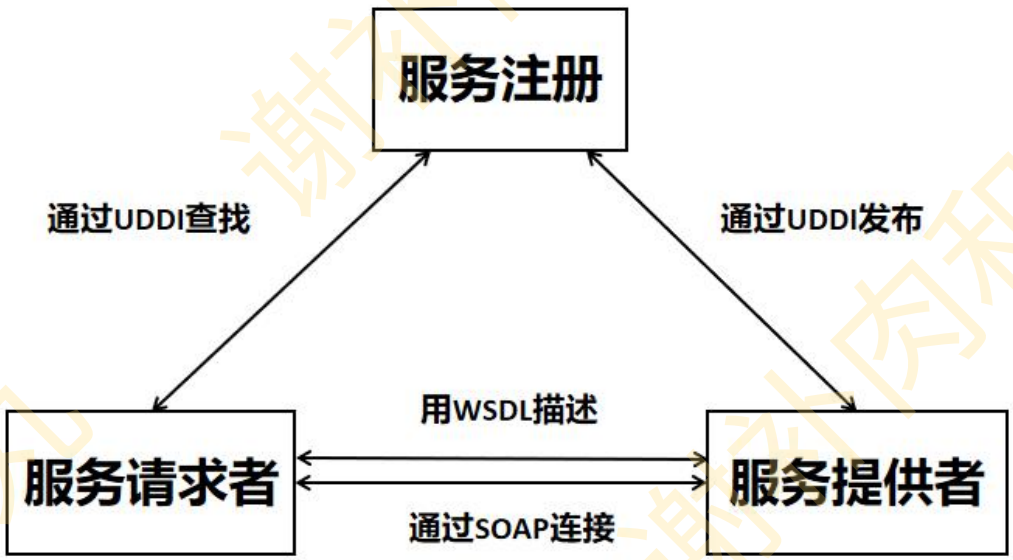




特性	SOA	单体架构
服务粒度	将业务功能分解为多个相对独立、粒度较细的服务，每个服务专注于特定的业务功能，可被不同应用复用，提高了业务灵活性和可扩展性。	整个应用是一个紧密耦合的整体，所有功能模块都集成在一起，难以单独拆分和复用，新增或修改功能可能影响整体。
通信方式	采用多种通信协议，如 SOAP、HTTP 等，服务之间通过标准的接口进行通信，实现不同服务间的交互和数据共享，支持分布式部署。	内部通过方法调用的方式实现模块间交互，无需复杂的外部通信协议，主要在同一应用内部完成。
数据管理	数据管理较为灵活，可以是多个服务共享数据库，也可通过企业服务总线（ESB）进行数据交换和整合，实现不同服务间的数据交互和协调。	通常使用单一数据库，所有功能模块都直接访问和操作同一数据源，数据的一致性和管理相对集中。
架构风格	属于分布式架构，各个服务可以独立部署和运行，可能存在中心化的 ESB，用于协调服务间的通信和数据交换，实现服务的集成和管理。	是集中式架构，所有功能集中在一个应用程序中，运行在同一环境下，易于开发和部署，但后期维护和扩展较困难。
故障影响	某个服务出现故障时，一般不会对其他服务产生直接影响，仅会影响依赖该服务的特定功能，故障隔离性较好。	一旦应用中的某个部分出现故障，可能会导致整个应用无法正常运行，故障影响范围大。

服务注册模式：

角色	操作内容
服务提供者	使用 UDDI 将 Web 服务发布到服务注册中心，通过 WSDL 描述服务
服务注册中心	接收并存储服务提供者通过 UDDI 发布的服务信息，供服务请求者查找
服务请求者	借助 UDDI 在服务注册中心查找 Web 服务，依据 WSDL 了解调用方式，通过 SOAP 与服务提供者建立连接并使用服务



名称	定义	核心作用
UDDI 通用发现和集成服务	是一个用于发现和集成 Web 服务的协议标准，基于 XML 的跨平台的描述规范	提供统一的方式发布和搜索 Web 服务信息。服务提供者可在注册表注册服务，供使用者查找和连接
WSDL Web 服务描述语言	用于描述 Web 服务接口的 XML 格式语言	定义 Web 服务的功能、输入输出参数、绑定协议和访问端点等信息，让服务使用者了解如何调用和使用 Web 服务
SOAP 简单对象访问协议	基于 XML 的消息传输协议，用于在 Web 服务之间进行通信	定义消息格式、编码规则和 RPC 约定，用于在应用程序之间交换结构化信息，可与多种因特网协议和格式结合使用，如 HTTP、SMTP 等
BPEL 业务流程执行语言	基于 XML 的业务流程定义语言，用于描述 Web 服务之间的业务流程和交互逻辑	用于编排多个 Web 服务，将它们组合成一个完整的业务流程，定义服务间的调用顺序、条件分支、异常处理和事务管理等，实现复杂业务场景的自动化执行

RESTful 架构:

主题	内容要点
资源与 URI	资源是任何有被引用必要的事物，包括实体和抽象概念，URI 是资源唯一标识，可看作资源地址或名称
统一资源接口	遵循统一接口原则，使用标准 HTTP 方法（GET、PUT、POST、DELETE 等）访问资源
资源的表述	客户端获取的是资源的表述，资源表述形式多样，如文本可采用 html、xml、json 等格式
资源的链接	REST 需利用超媒体概念，在表述格式中加入链接引导客户端，增强资源连通性
状态的转移	服务端不保存客户端状态，客户端与服务端交互无状态，每次请求包含所需信息，从而可推进状态转移

HTTP 方法：

- GET：从服务器获取资源（查询操作）
- POST：在服务器创建新资源（提交操作）
- PUT：更新服务器上的完整资源（替换操作）
- PATCH：对服务器上的资源进行部分更新
- DELETE：从服务器删除指定资源
- HEAD：与 GET 类似，但只返回响应头，不返回响应体
- OPTIONS：查询服务器支持的 HTTP 方法

企业服务总线（ESB）：

采用“总线”模式管理简化应用集成拓扑，ESB 提供位置透明性的消息路由和寻址服务、服务注册和命名的管理功能，支持多种消息传递范型（如请求/响应、发布/订阅等）、多种可以广泛使用的传输协议、多种数据格式及其相互转换，还提供日志和监控功能

SOA 设计原则：

特性	简要概括
无状态	避免服务请求者依赖服务提供者状态
单一实例	防止功能冗余
明确定义的接口	由 WSDL 定义，其中 WS-Policy 描述规约，XML 模式定义消息格式等
自包含和模块化	封装组件，实现功能实体独立部署、管理
粗粒度	服务数量适度，靠消息交互
松耦合性	使用者仅见接口，服务位置、实现等对其不可见
重用能力	服务具备可重用性
互操作性、兼容和策略声明	确保不同的系统、应用程序、设备或组件之间能够相互通信

系统架构评估：

- 敏感点：一个或多个构件（和/或构件之间的关系）的特性
- 权衡点：影响多个质量属性的特性，是多个敏感点的综合
- 利益相关人：系统架构涉及到的利益相关者，会试图影响架构决策以实现自己目标
- 场景：从风险承担者的角度对于系统的交互的简短描述，架构评估中一般采用刺激、环境和响应三方面来描述
- 风险点：可能导致系统架构出现问题或无法满足质量属性要求的特性、决策或情况。这些因素一旦发生，会给系统带来负面的影响，如降低性能、损害安全性等
- 非风险点：不会对系统架构的质量属性造成负面影响，或者对系统架构的影响在可接受范围内的特性、决策或情况

评估方法	描述	优点	缺点
基于调查问卷	通过设计问卷或检查表，利用相关人员的经验和知识进行评估	能够充分利用评估人员的经验和知识	在很大程度上依赖于评估人员的主观推断
基于场景的评估	分析软件架构对场景的支持程度，判断架构对质量需求的满足程度 SAAM、ATAM 都在其中	能够系统地评估架构对特定场景的支持	需要详细定义和分析场景，可能耗时
基于度量的评估	建立质量属性和度量之间的映射原则，从架构文档中获取信息，分析推导出系统的质量属性	客观性强，可以量化评估结果	需要精确的度量方法和映射原则，可能需要专业知识

信息安全相关：

- 对称加密：使用同一个密钥来加密和解密数据

加密过程：

1. 发送方有：明文 + 对称密钥
2. 用对称密钥 + 加密算法对明文处理
3. 输出：密文

解密过程：

1. 接收方有：密文 + 同一把对称密钥
2. 用同一把对称密钥 + 解密算法处理密文
3. 输出：明文

• 非对称加密：使用一对密钥，即公钥和私钥。公钥可以公开给任何人，用于加密数据；而私钥则由用户自己保密，用于解密数据

加密过程（别人发给你）：

1. 接收方提前生成：公钥、私钥
2. 接收方把公钥发给发送方
3. 发送方用接收方公钥加密明文 → 密文

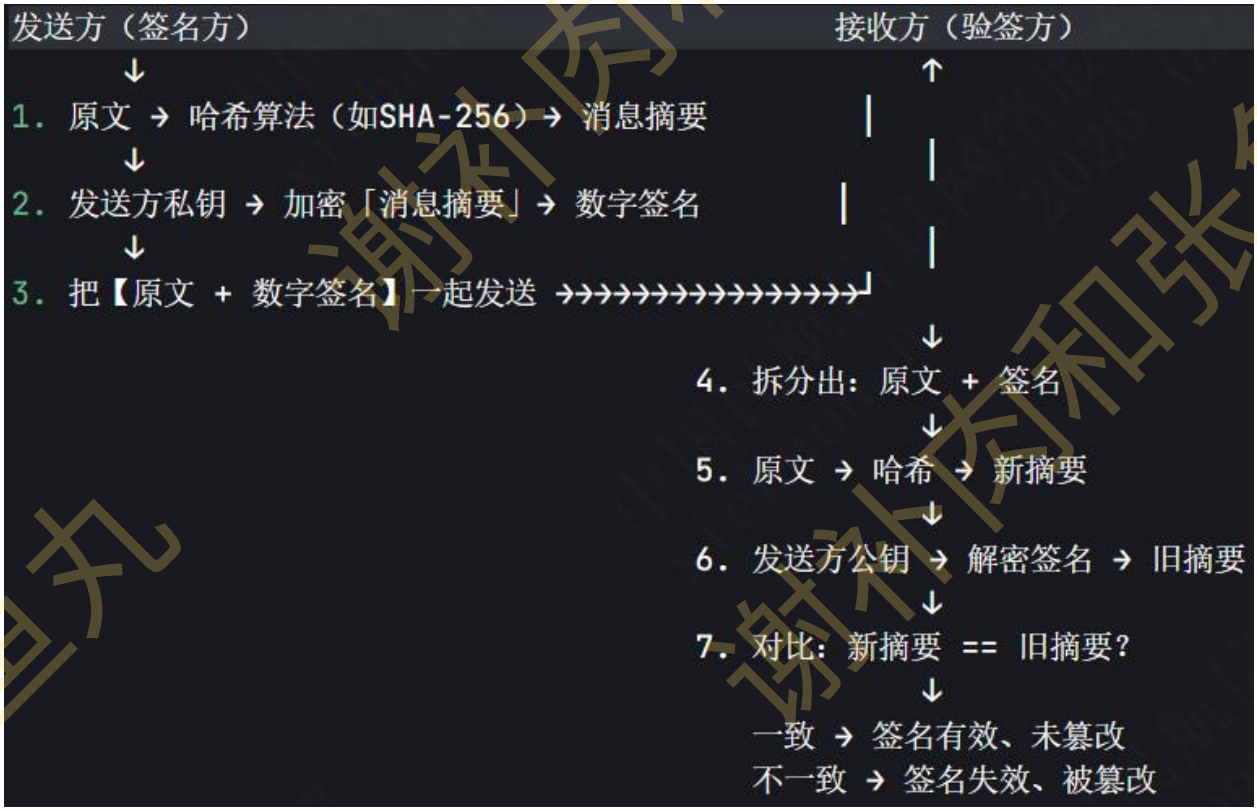
解密过程:

1. 接收方收到密文
2. 用自己的私钥解密密文 → 明文

对比项	对称加密	非对称加密
密钥数量	单一密钥	密钥对（公钥 + 私钥）
加密速度	快（适合大数据）	慢（适合小数据）
密钥管理	需安全共享	公钥公开，私钥保密
主要用途	数据加密	密钥交换、数字签名、身份验证
数学基础	替代 / 置换操作	数论难题（分解质因数、离散对数等）
典型算法	AES、DES、3DES	RSA、ECC、DH

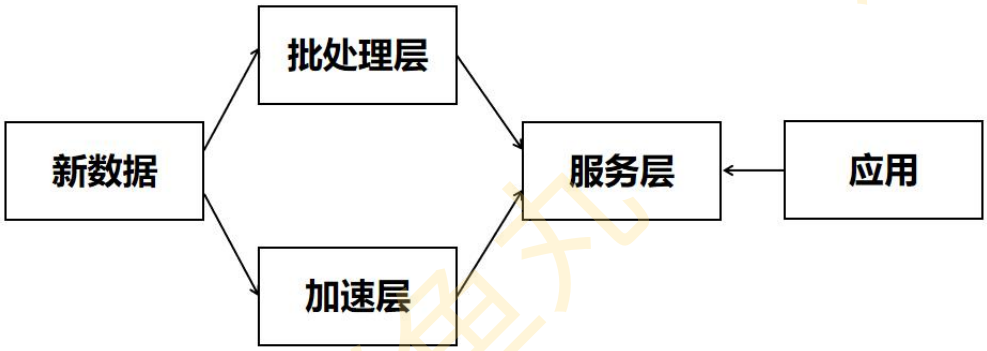
• 认证技术：身份确认、数据完整性、防伪造和防抵赖

• 数字签名：附加在数据单元上的数据或密码变换，确认数据来源、完整性和防止伪造；基于非对称加密算法，有普通和特殊数字签名算法；主要功能是保证信息传输完整性、发送者身份认证和防止抵赖



大数据架构：

Lambda 架构：三层，即批处理层、加速层和服务层



- 批处理层：负责所有批处理操作，存储整个数据集并计算批处理视图
  技术：Hive、Spark-SQL、MapReduce
- 加速层：使用流式计算技术实时处理当前数据，弥补批处理视图计算高延迟
  技术：Storm、SparkStreaming、Flink
- 服务层：以批处理层和加速层结果数据为基础，对外提供低延时数据查询和即席查询服务
  技术：传统数据库、Kylin、Presto、Impala、Druid

- 批处理层和加速层分开设计的原因：
  容错性：加速层处理的数据持续写入批处理层。当批处理层重新计算的数据集涵盖了加速层处理的数据集后，可以修正加速层的一些错误
  复杂性隔离：加速层运用增量算法处理实时数据，且其复杂性远高于批处理层，所以要进行隔离
  横向扩容：当系统面临的数据量或负载增大时，分层的设计可通过增加更多的机器资源来维持性能

Lambda 架构优缺点：

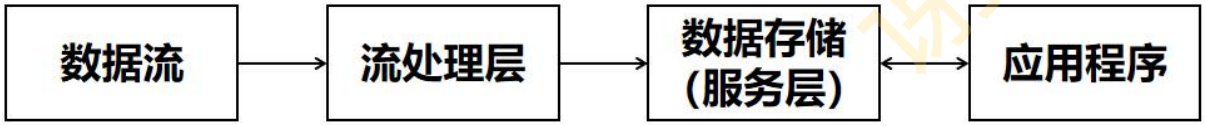
优点：

- 容错性好：
  Lambda 架构为大数据系统提供了更友好的容错能力，一旦发生错误，可以修复算法或从头开始重新计算视图
- 查询灵活度高：
  数据存储多样性：批、实时处理结果存于不同存储系统，满足不同查询需求
  多种查询接口支持：服务层提供 SQL、RESTfulAPI 等接口，结合可视化工具展示结果
- 易伸缩：
  分布式计算与存储：批、实时处理框架基于分布式架构，可按需添加节点
  弹性资源调度：框架配备调度器，自动分配管理资源，依据任务需求动态调整
- 易扩展：
  分层架构解耦性：分层设计，各层职责明确，扩展功能不影响其他层
  插件化与接口化设计：各层组件采用插件、接口设计，便于集成新组件

缺点：

- 技术栈复杂，开发维护成本高，数据迁移有一致性、速度等挑战

Kappa 架构：消息传输层、流处理层、数据存储



- 消息传输层：使用 Kafka 等消息队列实现高吞吐、低延迟的数据传输，支持数据持久化和重放
- 流处理层：采用 Flink、Storm 等框架进行实时计算，生成低延迟结果并输出至服务层
- 数据存储：处理后的数据存储于 HBase、Cassandra 等高性能存储系统，支持快速查询

Kappa 架构优缺点：

优点：

- 将实时和离线代码统一，方便维护，统一数据口径：

使用 Kappa 架构处理订单数据，开发人员无需分别维护实时和离线两套代码，能更高效地进行功能迭代和问题排查，避免因数据不一致导致的分析偏差

缺点：

- 消息中间件缓存的数据量和回溯数据有性能瓶颈：

当需要回溯过去几个月的消息数据进行分析时，由于消息中间件缓存数据量过大，可能导致回溯操作缓慢甚至超时，影响数据分析的效率

- 实时数据处理时，大量不同实时流关联易因数据顺序问题导致数据丢失：

在交通流量监控系统中，有车辆位置流、交通信号灯状态流等多个实时流数据。当关联这些数据计算车辆在不同信号灯状态下的通行情况时，若数据到达顺序混乱，可能会遗漏某些车辆在特定信号灯状态下的记录

- 抛弃离线数据处理模块，同时抛弃离线计算更稳定可靠的特点：

采用 Kappa 架构后，过度依赖实时计算，在网络波动或实时计算系统出现短暂故障时，可能无法及时、准确地识别风险

对比内容	Lambda 架构	Kappa 架构
复杂度与开发、维护成本	需要维护两套系统（引擎），复杂度高，开发、维护成本高	只需要维护一套系统（引擎），复杂度低，开发、维护成本低
计算开销	需要一直运行批处理和实时计算，计算开销大	必要时进行全量计算，计算开销相对较小
实时性	满足实时性	满足实时性
历史数据处理能力	批式全量处理，吞吐量巨大，历史数据处理能力强	流式全量处理，吞吐量相对较低，历史数据处理能力相对较弱

分布式系统

CAP 理论：

一致性（Consistency）、可用性（Availability）和分区容错性（Partition Tolerance）

CAP 理论认为，分布式系统最多只能同时满足其中的两个特性，无法同时满足全部三个特性

- 数据一致性：系统中写请求后各相关节点数据均变化，读请求能获取变化后数据
- 可用性：系统内节点接收读写请求后，都要处理并给出响应结果，即使数据可能有问题
- 分区容错性：系统中节点通信出现问题产生分区，系统仍需继续运行，各分区可独立对外服务

- 一致性和分区容忍性（CP）：在网络分区期间，某些操作可能不可用，牺牲了可用性
- 可用性和分区容忍性（AP）：在网络分区期间，不同节点可能有不同的数据副本，导致数据不一致
- 一致性和可用性（CA）：无法在分布式环境下实现

BASE 理论：

通过业务，牺牲强一致性而获得可用性，允许数据在一段时间内是不一致的，但是最终达到一致性状态

- 基本可用：分布式系统遇不可预知故障时，允许损失部分可用性
- 软状态：允许系统存在中间状态，且该状态不影响整体可用性，不同节点数据副本同步可存在延时
- 最终一致性：系统中所有数据副本经一段时间同步后，最终达到一致状态，无需实时强一致性

刚性事务 2PC：

特点	内容
一致性	强一致（刚性事务）
优点	实现简单，保证所有节点要么全提交，要么全回滚
缺点	1. 同步阻塞，性能差 2. 协调者单点故障风险高 3. 极端情况下可能数据不一致
适用	对一致性要求极高、并发不高的场景

角色：

- 协调者（Coordinator）：事务管理者
- 参与者（Participant）：各个节点 / 数据库

第一阶段：准备阶段（投票阶段）

- 事务询问：协调者向所有参与者发送事务内容，询问是否可以执行提交操作，并开始等待各参与者进行响应
- 执行事务：各参与者节点，执行事务操作，并将 Undo（没提交就崩了，撤销回去）和 Redo（提交了没写完，重做一遍）操作计入本机事务日志
- 各参与者向协调者反馈事务问询的响应。成功执行返回 Yes，否则返回 No

阶段二：提交阶段（执行阶段）

- 协调者在阶段二决定是否最终执行事务提交操作，包含两种情形执行事务提交和中断事务

• 执行事务提交：所有参与者 ReplyYes，那么执行事务提交。发送提交请求，协调者向所有参与者发送 Commit 请求，事务提交。参与者收到 Commit 请求后，会正式执行事务提交操作，并在完成提交操作之后，释放在整个事务执行期间占用的资源，反馈事务提交结果。参与者在完成事务提交后，向协调者发送 ACK 消息确认，完成事务。协调者在收到所有参与者的 ACK 后，完成事务

• 中断事务：当存在某一参与者向协调者发送 No 响应，或者等待超时。协调者只要无法收到所有参与者的 Yes 响应，就会中断事务。发送回滚请求。协调者向所有参与者发送 Rollback 请求，回滚。参与者收到请求后，利用本机 undo 信息，执行 Rollback 操作。并在回滚结束后释放该事务所占用的系统资源，反馈回滚结果。参与者在完成回滚操作后，向协调者发送 ACK 消息，中断事务。协调者收到所有参与者的回滚 ACK 消息后，完成事务中断

柔性事务 TCC：

对比维度	具体内容
一致性	最终一致性，通过 Try-Confirm-Cancel 三段式操作保证分布式事务最终一致
优点	1.不依赖数据库底层事务，支持跨库、跨服务分布式场景 2.资源预留机制可有效避免并发冲突 3.容错性强，Confirm/Cancel 支持幂等重试
缺点	1.侵入性高，需为每个业务定制 Try、Confirm、Cancel 逻辑 2.开发成本高，对技术人员业务理解要求高 3.Try 阶段资源锁定时间较长，高并发场景下影响吞吐量
适用场景	不要求强一致性，并发大的场景

• Try 阶段：尝试执行，完成所有业务检查（一致性），预留必需业务资源（准隔离性），如果全部成功则进入 Confirm 阶段，如果有失败则进入 Cancel 阶段

• Confirm 阶段：确认执行真正执行业务，不做任何业务检查，只使用 Try 阶段预留的业务资源。要求具备幂等设计，Confirm 失败后需要不断重试，直到成功

• Cancel 阶段：取消执行，释放 Try 阶段预留的业务资源。Cancel 操作满足幂等性 Cancel 阶段的异常和 Confirm 阶段异常处理方案基本上一致，也要求支持重试、保证最终成功

对比维度	2PC（两阶段提交）	TCC（Try-Confirm-Cancel）
一致性	强一致性，事务执行过程中所有节点要么全提交，要么全回滚	最终一致性，允许短时间数据不一致，通过补偿操作保证最终一致
优点	1.实现简单，原理易懂 2.强一致特性满足高一致性业务需求 3.对业务代码无侵入	1.不依赖数据库底层事务，支持跨库、跨服务场景 2.资源预留机制避免并发冲突 3.容错性强，支持失败重试
缺点	1.同步阻塞，参与者全程锁定资源，性能差 2.协调者单点故障风险高 3.极端情况（如协调者宕机）可能导致数据不一致	1.侵入性极高，需为每个业务定制 Try/Confirm/Cancel 逻辑 2.开发成本高，对技术人员业务理解要求高 3.Try 阶段资源锁定时间较长，影响高并发场景吞吐量
适用场景	1.对一致性要求极高、并发量低的场景 2.中心化的小型分布式系统（如传统单体系统拆分的小集群） 3.数据库原生支持的分布式事务场景	1.高并发、分布式核心业务场景（如机票预订、电商下单） 2.跨多个独立服务/数据库的事务场景 3.可接受短时间数据不一致的业务

数据库系统：

索引的建立与选择遵循准则：

• 建立索引时，应选用经常作为查询，而不常更新的属性。避免对一个经常被更新的属性建立索引，因为这样会严重影响性能

• 一个关系上的索引过多会影响 UPDATE、INSERT 和 DELETE 的性能，因为关系一旦进行更新，所有的索引都必须跟着做相应的调整

• 尽量分析出每个重要查询的使用频度，这样，可以找出使用最多的索引，然后可以先对这些索引进行适当的优化

• 对于数据量非常小的关系不必建立索引，因为对于小关系而言，关系扫描往往更快，而且消耗的系统资源更少

数据分片：

把数据库横向扩展（ScaleOut）到多个物理节点上的一种有效的方式，其主要目的是为突破单节点数据库服务器的 I/O 能力限制，解决数据库扩展性问题

通俗说，数据分片 = 把一个大数据库，按规则拆成多个小数据库（分片），分散存储、分散查询

1. 垂直分片（按列 / 按业务拆分）

把一张大表按字段 / 业务拆成多张表，通常拆库也拆表

典型场景：

• 用户库拆成：用户基本信息库、用户订单库、用户日志库

• 把冷热字段分离：经常查询的放一起，大字段 / 不常用的单独分片

优点：

• 业务清晰，隔离性好

• 单表变小，查询更快

缺点：

• 跨业务查询需要分布式事务 / 跨库 join

• 扩容不灵活，仍可能单表过大

2. 水平分片：

同一张表，按行数据分到不同库 / 表，结构完全一样，只是数据不同。

典型场景：

• 订单表、日志表、用户表超千万 / 亿数据

水平分片常见策略

① 范围分片

• 按 ID 范围、时间范围

• 例：user_id 1~100 万放分片 1，100 万～200 万放分片 2

• 优点：扩容简单

• 缺点：容易数据倾斜（热点集中）

② 哈希分片（最常用）

• 对分片键 hash 后取模

• 例：hash (user_id) % 8

• 优点：数据分布均匀
• 缺点：扩分片时要重新哈希（rehash），数据迁移量大
③ 一致性哈希
• 解决普通哈希扩分片时全量迁移问题
• 只影响少量节点
④ 地理 / 业务分片
• 按地区、商户、业务线分
• 适合多地域部署、合规要求

数据分片带来的问题：
1. 跨分片分页、排序、聚合困难
• count、sum、order by、limit 不能直接用
2. 分布式事务复杂
• 跨分片事务一致性难保证
3. 跨分片 JOIN 性能差
4. 数据倾斜
• 部分分片数据量大，成为瓶颈
5. 扩容、数据迁移成本高
6. 全局唯一 ID
• 不能用自增主键，需雪花算法、UUID 等

数据分区：
通俗说，同一个数据库实例、同一个表空间内，把一张大表，按某种规则物理上分成多个部分，但对外逻辑上还是一张表。提升查询性能，只扫描对应分区，不扫全表；快速清理历史数据；分散 IO 压力—不同分区放不同文件 / 磁盘；维护方便—单独备份、恢复某个分区
1. 范围分区（RANGE）
按连续区间分区：时间、ID、数值
• 按年 / 月 / 日
• 按 ID 段
优点：
• 时序数据最适合
• 删历史数据极快
典型场景：订单表、日志表、操作记录
2. 列表分区（LIST）
按枚举值分区：固定离散值
• 省份、城市、状态、类型
例：
• 华东地区放一个分区
• 华南地区放一个分区

适合：值固定、不经常变

3. 哈希分区（HASH）

对字段做 hash，均匀分布

- 解决数据倾斜
- 没有明显范围或列表时用

优点：数据分布均匀缺点：不能按范围快速删数据

4. 组合分区（复合分区）

先按一种分区，再分区

- RANGE-HASH（先时间，再哈希）
- RANGE-LIST

分库分表：

分库，解决单机承载能力有限的问题，把请求分散到多台服务器上。分表，解决单表数据量过大的问题，把数据拆开，让查询在更小的范围内完成

1. 垂直分库分表（按列 / 业务拆）

- 垂直分库：按业务模块拆库

用户库、订单库、商品库、支付库

- 垂直分表：把大表字段拆开

常用字段一张表，大字段 / 冷数据一张表

优点：

- 业务清晰、隔离性好
- 单表变小，查询更快

缺点：

- 无法解决单表数据量过大
- 会出现跨库关联、分布式事务

2. 水平分库分表（按行拆）

同一张表，按行拆分到多个库 / 表，表结构完全一样。

常见分片策略：

① 范围分片（按 ID / 时间）

易扩容，但容易数据倾斜

② 哈希分片

分布均匀，但扩容要 rehash

③ 一致性哈希

解决扩容数据迁移问题

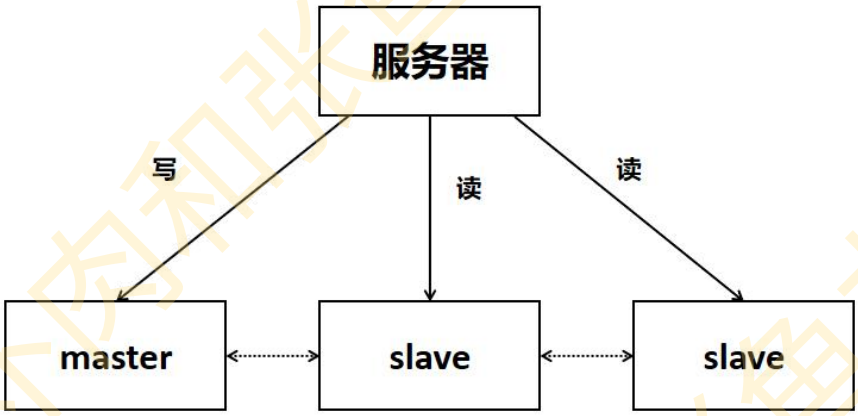
④ 业务分片（用户 ID、商户 ID）

水平分片是分库分表的核心实现方式，通常所说的分库分表，主要指水平分片

对比项	数据分区 Partition	数据分片 Sharding 分库分表
所在位置	单库、单实例、单机	跨库、跨实例、跨机器
对应用透明	完全透明	一般需要中间件
解决问题	单表太大、查询慢、清理难	并发上限、容量上限、性能瓶颈
分布式	不是分布式	分布式架构
复杂度	低	高
适用阶段	单库还能扛，表太大	单库扛不住，必须分布式

数据库高可用模式：

读写分离：



主从复制：

（1）主库将数据库中数据的变化写入到二进制日志

（2）从库连接主库，并请求复制

（3）从库会创建一个 I/O 线程向主库请求更新的二进制日志

（4）主库会创建一个二进制日志 dump 线程来发送二进制日志，从库中的 I/O 线程负责接收

（5）从库的 I/O 线程将接收的二进制日志写入到中继日志中

（6）从库的 SQL 线程从中继日志中读取事件并应用到本地数据库中

带来的优点：

- 数据冗余：提供数据副本，防止主库故障导致数据丢失
- 负载均衡：将读操作分发到从库，减轻主库负载
- 高可用性：主库故障时，能快速切换到从库，保障系统运行

带来的问题：

- 一致性问题

解决方法：

同步方式	原理	优点	缺点
半同步复制	主库提交事务后，等至少一个从库接收并写入事务 binlog 到本地中继日志，收到从库 ACK 后主库才正式提交事务并返回客户端	利用数据库原生功能，比较简单	主库写请求时延增长，吞吐量降低

数据库中间件	所有读写走中间件，写请求路由到主库，读请求路由到从库。记录写库 key，主从同步时间窗口内读请求路由到主库，时间过了再路由到从库，如 Canal、Otter	能保证绝对一致	数据库中间件成本较高
缓存记录	写操作时记录 key 到 cache 并设超时时间（如 500ms）再修改主数据库；读时先查 cache，命中则路由到主库，未命中则路由到从库	相对数据库中间件成本较低	为保证一致性引入 cache 组件，读写数据库时多了缓存操作

大模型应用：

检索增强生成（RAG）：在大模型生成回答之前，先从外部知识库 / 数据库中检索相关知识，把检索到的内容作为上下文一起送给大模型，让模型基于真实、准确、最新的资料生成答案

RAG 标准流程：

1. 知识库构建

• 文档上传、清洗、分段（Chunk）

① 清洗文档：去掉多余的格式信息，如 HTML 标签、PDF 页眉页脚等

② 分段处理：长文档不可能直接输入模型，因此要切分成较小的文本块。这里有 3 种策略

语义切分：确保每个块是语义独立的（如问答对、完整段落）

结构切分：按标题层级切分，保留文档的逻辑结构

递归切分：当文本过长时，递归按字符数拆分，既保持连贯性又控制长度

2. 向量化与存储

• 文本转成向量（Embedding）

• 存入向量数据库（如：Milvus、Chroma、Pinecone）

完成分块后，每个文本块需要通过 嵌入模型（Embedding Model） 转换为向量。常见的模型有 BERT、Sentence-BERT、BGE 等。向量本质上是一个高维坐标点，用来表示文本的语义信息。比如“苹果手机”和“iPhone”会被映射到相近的向量空间位置，从而实现语义层面的检索

3. 用户问题检索

• 用户问题 → 向量化 → 向量库检索

• 召回最相关的若干段知识

4. 提示词 + 检索内容 + 大模型生成

• 把资料塞进上下文

• 模型基于资料作答

模型上下文协议（MCP）：旨在为大语言模型（LLM）与外部世界之间提供一个统一的“通用接口”

MCP 工作流程：

1. 初始化与能力协商

• Host 创建 Client，Client 与 Server 建立连接，完成身份验证、权限协商、能力（支持的原语）交换

2. 原语发现

• Client 向 Server 发送*/list 请求，获取可用的 Tools、Resources、Prompts 列表

3. 请求处理（核心）

• 用户提问 → Host 解析 → Client 封装为 MCP 请求 → 发送至 Server

• Server 根据请求类型：调用 Tool 执行操作 / 获取 Resource 返回数据 / 返回 Prompt 模板

• Server 将结果封装为 MCP 响应 → 返回 Client

4. 生成与响应

• Client 将 Server 返回的上下文 / 结果传递给 LLM → LLM 结合自身能力生成最终回答 → Host 返回给用户

智能体（AGENT）：

能感知环境 → 自主决策 → 规划步骤 → 调用工具 → 执行动作 → 达成目标的智能实体

通俗说，不只是问答，而是能自己思考、自己干活的 AI

经典智能体架构：

1. 感知（Perception）

• 获取用户需求、环境信息、外部数据

2. 记忆（Memory）

• 短期记忆（上下文）、长期记忆（历史经验、知识库）

3. 规划（Planning）

• 拆解任务、制定步骤、判断顺序

4. 决策与行动（Action）

• 调用工具、执行逻辑、访问系统、生成内容

5. 反馈迭代（Reflection）

• 反思结果对不对，错了就修正、重试

Agent 工作流程：

1. 用户提出目标
2. Agent 理解目标
3. 拆解任务：大任务 → 小步骤
4. 调用工具 / 知识（搜索、RAG、代码、数据库）
5. 执行，并记录结果
6. 反思是否达到目标
7. 没达到 → 重新规划、修正
8. 达到目标 → 输出最终结果

Agent 关键技术：

1. 大模型作为大脑

• 负责理解、推理、规划

2. 工具调用（Function Call）

• 让 AI 能使用外部能力

3. RAG 检索增强

• 提供准确、实时知识

4. 反思机制（Reflection）

• 减少错误，提高可靠性

5. 多智能体协作（Multi-Agent）

• 多个 Agent 分工合作（产品、开发、测试）

对比项	传统大模型 LLM	AI 智能体 Agent
核心能力	问答、生成文本	自主思考、规划、执行、完成任务
交互方式	一问一答	多轮、自主迭代、持续任务
自主性	低，需要人引导	高，自主完成复杂任务
工具使用	简单提示调用	自主规划、自主调用、自主纠错
适用场景	写作、问答、总结	数据分析、自动化流程、开发、决策
复杂度	低	高

MLOps vs LLMOps：

• MLOps：机器学习全生命周期工程化平台

面向传统机器学习模型（分类、回归、预测、风控等）

• LLMOps：大模型时代的 MLOps

面向大语言模型 LLM 全生命周期管理，是 MLOps 的升级版

MLOps：

1. 定义

MLOps = Machine Learning Operations 是一套方法论 + 工具链 + 流程规范，用于实现机器学习模型从数据采集 → 特征工程 → 训练 → 验证 → 部署 → 监控 → 迭代的自动化、可复现、可规模化，本质：把 DevOps 思想用到机器学习里

2. 核心目标：

• 解决模型训练不可复现

• 解决训练与部署环境不一致

• 解决模型上线慢、迭代难

• 实现持续训练（CT）、持续集成（CI）、持续部署（CD）

• 提升模型可观测性、可治理性

3. 典型流程

数据获取 → 数据预处理 → 特征工程 → 模型训练 → 模型评估 → 模型打包 → 模型部署 → 在线推理 → 监控（漂移、性能）→ 迭代更新

4. 关键技术 / 工具

• 版本管理：DVC、Git LFS

• 实验管理：MLflow、Weights & Biases

• 特征平台：Feast

- 模型服务：TensorFlow Serving、TorchServe、KServe
- 调度：Kubeflow、Airflow
- 监控：模型漂移（Concept Drift / Data Drift）

LLMOps：

1. 定义

LLMOps = Large Language Model Operations 针对大语言模型的全生命周期运营工程体系，是 MLOps 在大模型场景下的延伸与扩展

2. 为什么 LLMOps ≠ MLOps

- LLM 与传统 ML 差异巨大，所以需要专门的 LLMOps：

LLM 参数量极大，训练 / 微调成本高、硬件要求高

LLM 效果依赖提示词（Prompt），需要 Prompt 管理与优化

存在幻觉、偏见、安全问题，需要治理

支持 RAG、微调、Agent 等复杂架构

推理成本高，需要动态批处理、量化、蒸馏优化

3. LLMOps 核心能力

- 大模型选型与管理（开源 / 闭源、私有部署）
- 提示词工程与版本管理
- 高效微调（LoRA、QLoRA、全参数微调）
- RAG 系统构建与检索优化
- 推理服务加速与调度
- 效果监控：幻觉、一致性、安全性
- 成本监控：Token 消耗、算力成本
- 安全与合规：内容审核、权限、隐私

4. LLMOps 典型工具链

- 管理：LangSmith、LlamaIndex Cloud
- 微调：Transformers、PEFT
- 部署：vLLM、TGI、TensorRT-LLM
- 监控：LangFuse、Arize
- 向量库：Milvus、Qdrant

维度	MLOps	LLMOps
适用模型	传统机器学习（LR、XGBoost、CNN 等）	大语言模型 LLM
核心关注点	特征、模型精度、训练漂移	Prompt、幻觉、推理成本、RAG、安全
训练成本	较低	极高（算力、数据、时间）
部署方式	模型文件上线	推理引擎 + 向量库 + 提示管理
迭代方式	重训练、重部署	Prompt 优化、LoRA 微调、RAG 更新
监控重点	准确率、数据漂移	效果、安全性、Token 成本、时延

Prompt 优化（提示词工程）：

1. 定义

Prompt 工程是在不修改大模型参数的前提下，通过设计、优化、结构化输入提示，让大模型输出更准确、稳定、符合预期的技术。一句话：不微调模型，只微调提问方式

2. 核心目标

- 提升输出准确性，减少幻觉
- 增强输出可控性、一致性
- 提高任务完成度（分类、抽取、总结、推理）
- 降低 Token 消耗，节约成本
- 适配复杂场景：RAG、多轮对话、Agent

3. 经典 Prompt 结构

- 角色定义：你是 XX 领域专家 / 架构师 / 工程师
- 任务说明：请完成 XX 任务
- 约束条件：格式、长度、禁止内容、专业程度
- 输入内容：用户问题 / 上下文 / 文档片段
- 输出格式：JSON、表格、步骤、要点

4. 常用优化技巧

- 少样本学习（Few-shot）：给几个示例
- 思维链（CoT）：让模型 “一步步思考”
- 上下文增强：结合 RAG 检索内容
- 结构化指令：强制输出 JSON/Markdown
- 负面提示：明确禁止幻觉、编造
- 多轮对话管理：历史对话压缩、摘要

优点：

- 无需训练、零代码、低成本
- 迭代极快，实时生效
- 安全风险低，不改动模型

缺点：

- 效果上限有限
- 复杂任务仍需微调
- 过长 Prompt 会占用上下文窗口

LoRA 微调（低秩适配）：

1. 定义

LoRA（Low-Rank Adaptation）是一种大模型高效微调技术，不训练全量模型参数，只在 Transformer 层插入低秩小矩阵进行训练，实现低成本、快速微调。一句话：只改一点点参数，让模型适配你的业务

2. 核心思想

- 大模型权重矩阵具有低秩特性
- 新增两个小矩阵 A、B 训练

- 原模型冻结不动
- 推理时合并小矩阵，不增加延迟

3. 为什么要用 LoRA？
- 全量微调显存爆炸、成本极高
- LoRA 仅训练 0.01%~1% 参数
- 训练速度快、显存占用低
- 可多任务独立存储，随时切换
- 不破坏原模型能力

4. 优点
- 轻量化、低成本、速度快
- 显存需求大幅降低
- 支持多 LoRA 权重热切换
- 不影响推理速度
- 适合企业私有领域微调

5. 常见同类技术
- QLoRA：量化 + LoRA，显存更低
- Adapter：插入小模块
- Prefix Tuning / Prompt Tuning：只优化提示向量

谢补闪和张鱼丸

数据库 vs 数据仓库 vs 数据湖

对比维度	数据库（OLTP 业务库）	数据仓库 DW（OLAP）	数据湖 DataLake
核心用途	日常业务增删改查、交易下单、用户管理	历史数据分析、报表、BI、统计	存放所有原始数据，不限格式，留着随时挖掘
数据类型	结构化数据（表、行、列，固定字段）	结构化、半结构化（规整建模后）	结构化 + 半结构化 + 非结构化日志、音视频、文本、图片、原始日志都能存
处理场景	联机事务处理 OLTP	联机分析处理 OLAP	原始数据存储、大数据计算、AI 训练、探索分析
数据粒度	精细、最新实时业务数据	汇总、聚合、清洗加工后的数据	原始最细粒度，不做预处理、原汁原味
存储格式	关系型表（InnoDB 等）	列式存储、分区表、维度建模	原始文件：Parquet、ORC、JSON、日志、二进制
读写特征	高频写入、更新、删除，查询简单	极少更新、批量导入、海量查询分析	一次写入、多次读取，不随意改原始数据
建模方式	范式建模（三范式），减少冗余	维度建模（星型 / 雪花），方便分析	无固定建模，想用再加工建模
实时性	毫秒 / 秒级，强实时	小时 / 天级，准实时 / 离线	可实时入湖、也可离线批量入湖
成本	存储成本高、扩容贵	中等，按规整数据付费	成本最低，廉价对象存储堆海量数据
一句话定位	干活做生意的业务系统	看报表做分析的决策系统	什么都装的原始数据大水库

云原生底层架构：

OpenStack：

- Horizon：Web 管理界面（控制台）
- Keystone：统一身份认证（账号 / 权限）
- Nova：虚拟机管理（创建 / 调度 / 迁移）
- Glance：镜像仓库（系统模板）
- Neutron：虚拟网络（VPC / 子网 / 路由）
- Cinder：云硬盘（块存储）
- Swift：对象存储（文件 / 备份）
- Trove：PaaS 模块

PVE：

- PVE API/UI：统一入口，管理所有资源
- Corosync：集群通信，保证节点一致性
- Pacemaker：HA 资源管理，故障自动切换
- QEMU/KVM：全虚拟化，跑 Windows/Linux 虚拟机
- LXC：操作系统级容器，轻量高效
- Ceph（可选）：分布式存储，高可用
- Linux Bridge：虚拟网络，连接虚拟机 / 容器

Harvester：

- Harvester UI / Rancher：管理界面，统一管理 VM 和容器
- Kubernetes：核心编排，管理所有节点和 Pod
- KubeVirt：让 K8s 原生运行虚拟机（KVM）
- Longhorn：分布式块存储，自动副本、快照、备份
- Multus CNI：多网络接口支持，VM 可连多个网络
- Prometheus/Grafana：监控告警

架构图技术汇总：

需求图	包含关系 - «contain»
	派生关系 - «derive»
	满足关系 - «satisfy»
	验证关系 - «verify»

移动应用		
移动终端系统	Android	安卓手机操作系统
	IOS	苹果手机操作系统
	Harmony	鸿蒙操作系统
	Symbian	塞班手机系统
	BlackBerry	黑莓手机系统
	LiteOS	华为轻量级物联网 OS
	RT-Thread	国产实时操作系统
	Ubuntu	Linux 桌面/服务器系统
	Tizen	三星物联网/手机系统
移动开发技术	Android SDK	安卓开发工具包
	Kotlin	安卓官方推荐开发语言
	Jetpack	安卓组件库套件
	Swift	iOS 开发语言
	Objective-C	iOS 旧版开发语言
	Xcode	iOS 开发 IDE
	Flutter	谷歌跨端 UI 框架
	Weex	阿里跨端框架
	React Native	脸书跨端框架
移动本地存储	SQLite	嵌入式轻量数据库
	Realm	移动端高性能数据库
	Room	安卓官方数据库框架
	SharedPreferences	安卓轻量存储
	Keychain	iOS 安全存储
	UserDefaults	iOS 轻量存储
	MMKV	微信开源移动端高性能 KV 存储

客户端	Application Client	运行在客户端，能直接访问 ejb 组件
表示层	Applet	嵌入浏览器并在其中执行的 java 程序
	JFC	开发桌面图形界面 GUI 应用
	JSP	简化 servlet 开发的技术，在 html 中嵌入 java 代码最终转换成 servlet
通信层	IIOP	corba 中对象之间进行通信的协议
	RMI	实现远程过程调用的 java api
	Java IDL	跨语言、跨平台分布式对象通信技术
服务层	JNDI	用于在 java 应用中查找和访问命名
业务逻辑层	EJB	服务器组件模型，提供事务处理等企业级业务以实现业务逻辑
ESB 接入协议	JMS	Java 消息服务应用程序接口
持久层	JDBC	是一种可用于执行 SQL 语句的 Java API
	JCA	将 java ee 应用服务器连接到信息系统

模型 表现层	Java Bean	封装数据，提供 getter/setter 方法
视图 表现层	JSP	主要负责展示界面
控制器 表现层	Servlet	接收 HTTP 请求，调用 JavaBean（Model）处理业务逻辑
MVVM 表现层	Vue.js	运行时充当了绑定引擎

web 应用		
表示层 前端	HTML5	前端页面结构标记语言
	CSS3	前端页面样式描述语言
	JavaScript/ES6	前端脚本语言，现代 JS 标准
	Vue、React、Angular	三大主流前端框架，用于构建用户界面
	UniApp、AntDesign	跨端框架/企业级 UI 组件库
	AJAX、Axios	前端异步请求工具
	CDN	内容分发网络，加速静态资源
Web 服务容器 部署运行 Web 应用	Tomcat	开源 Java Servlet 容器
	Jetty	轻量级 Java Web 容器
	Undertow	高性能 Web 服务器，红帽开发
	WebLogic	Oracle 商业 Java 应用服务器
	WebSphere	IBM 商业 Java 应用服务器
	JBoss/WildFly	Java EE 应用服务器/JBoss 新一代应用服务器
	Nginx	高性能 HTTP 和反向代理服务器，也用作负载均衡和静态资源服务器
	ApacheHttpd	经典 Web 服务器
后端 Web 框架 处理请求业务	SpringMVC	基于 Spring 的 MVC 框架，用于构建 Java Web 应用程序
	SpringBoot	简化 Spring 应用开发的框架，支持快速构建独立应用
	SpringCloud	微服务开发框架，提供服务注册、发现、配置管理等功能
	Struts2	早期 Java MVC Web 框架
	JSF	Java 官方企业级 Web 框架
	Django	Python Web 框架
	Flask、FastAPI	轻量级 Python Web 框架/高性能 Web 框架
	Express（Node.js）	Node.js 后端 Web 框架
	Gin、Beego	Go 语言 Web 框架
	ASP.NETCore	微软跨平台 Web 框架
接口 API 技术 前后端交互	RESTfulAPI	基于 REST 架构风格的 API 设计，使用 HTTP 方法进行资源操作
	JSON、XML	数据交换格式，用于前后端数据传输
	WebService（SOAP）	Web 服务间的 XML 格式数据交换
	GraphQL	API 查询语言，客户端精确获取所需数据
	BFF 层	后端适配前端的聚合层
	Swagger/OpenAPI	API 文档与接口调试工具
业务逻辑 服务治理	Dubbo、gRPC	微服务远程调用框架
	SSO、OAuth2、JWT	统一身份认证与授权标准
	SpringSecurity、Shiro	安全框架，提供认证、授权、加密和会话管理功能
	Activiti、Flowable	工作流引擎，用于业务流程编排
	Drools	规则引擎，实现复杂业务规则

数据访问层 操作数据库	MyBatis、MyBatis-Plus	Java 持久层框架，支持 SQL 映射和存储过程
	Hibernate、JPA	Java 持久层框架增强版，简化数据库操作
	JDBC	Java 数据库连接标准接口
	Druid、HikariCP	数据库连接池，高性能数据库连接管理
负载均衡 接入层	Nginx、LVS、F5	高性能负载均衡与接入层解决方案
运维 监控 Web 应用	Log4j、SLF4J	Java 日志框架，用于日志记录与输出
	SkyWalking、Zipkin	分布式链路追踪系统，用于微服务监控与问题排查

谢补闪和张鱼丸

微服务 分布式框架	SpringBoot	简化 Spring 应用开发的框架，支持快速构建独立应用
	Dubbo	阿里开源的 RPC 框架，用于微服务之间的高性能远程调用
	gRPC	基于 HTTP/2 和 Protocol Buffers 的高性能 RPC 框架
	SpringCloudAlibaba	微服务开发框架，提供服务注册、发现、配置管理等功能
	Quarkus	红帽云原生 Java 微服务框架
	Micronaut	轻量级云原生微服务框架
	Istio	服务网格，治理微服务流量、安全、可观测
	Linkerd	轻量级服务网格
权限认证 统一登录	OAuth2.0	开放授权标准，允许第三方应用获取用户资源访问权限
	JWT	JSON Web Token，用于在各方之间安全传输信息的紧凑令牌
	CAS	集中式认证服务，用于多系统单点登录(SSO)解决方案
	Shiro	Java 安全框架，提供认证、授权、加密和会话管理功能
	SpringSecurity	基于 Spring 的安全框架，提供全面的身份认证和授权支持
	SSO 单点登录服务	统一身份认证，一次登录多系统通行
Web 框架 处理业务逻辑 响应前端请求	SpringMVC	基于 Spring 的 MVC 框架，用于构建 Java Web 应用程序
	SpringBoot	简化 Spring 应用开发的框架，支持快速构建独立应用
	SpringCloud	微服务开发框架，提供服务注册、发现、配置管理等功能
	Struts2	早期 Java MVC Web 框架
	Struts	早期 Java MVC Web 框架
	JSF	Java 官方企业级 Web 框架
	ASP.NETMVC	微软.NET MVC Web 框架
	Django	Python Web 框架，提供完整的 MVC 架构和丰富的内置功能
	Flask	轻量级 Python Web 框架，适合小型应用和 API 开发
	FastAPI	Python 高性能 Web 框架
	Nginx	高性能 HTTP 和反向代理服务器，也用作负载均衡和静态资源服务器
	Express	Node.js 后端 Web 框架

API 网关 接入服务	Nginx	高性能 HTTP 和反向代理服务器，也用作负载均衡和静态资源服务器
	OpenResty	Nginx+Lua 高性能网关
	Kong	云原生 API 网关，支持插件扩展
	APISIX	国产云原生 API 网关
	Zuul	Spring Cloud 微服务网关
	SpringCloudGateway	Spring 官方反应式网关
	F5	硬件负载均衡器
	LVS	Linux 虚拟服务器，四层负载均衡
Web 应用容器 Web 框架 处理业务逻辑 响应前端请求	Tomcat	开源 Java Servlet 容器，用于运行 Java Web 应用程序
	Jetty	轻量级 Java Web 容器
	Undertow	高性能 Web 服务器，红帽开发
	JBoss	Java EE 应用服务器
	WildFly	JBoss 新一代应用服务器
	WebLogic	Oracle 商业 Java 应用服务器
	WebSphere	IBM 商业 Java 应用服务器
	Resin	高性能 Java Web 容器
	GlassFish	Oracle 开源 Java 应用服务器
	SpringMVC	基于 Spring 的 MVC 框架，用于构建 Java Web 应用程序
	SpringBoot	简化 Spring 应用开发的框架，支持快速构建独立应用
	SpringCloud	微服务开发框架，提供服务注册、发现、配置管理等功能
	Struts2	早期 Java MVC Web 框架
	Struts	早期 Java MVC Web 框架
	JSF	Java 官方企业级 Web 框架
	ASP.NETMVC	微软.NET MVC Web 框架
	Django	Python Web 框架，提供完整的 MVC 架构和丰富的内置功能
	Flask	轻量级 Python Web 框架，适合小型应用和 API 开发
	FastAPI	Python 高性能 Web 框架，用于快速构建 API
	Nginx	高性能 HTTP 和反向代理服务器，也用作负载均衡和静态资源服务器
	Express	Node.js 后端 Web 框架
	Javaquery	前端 JS 库，用于简化 DOM 操作与交互
	View	前端视图层组件

大数据应用		
实时数据集成 实时数据采集	Kafka	分布式流处理平台，用于高吞吐、低延迟的实时数据发布与订阅
	Zookeeper	分布式协调服务，用于管理集群配置、命名服务和分布式锁
	Flume	分布式日志收集系统，用于高效收集、聚合和移动大量日志数据
	Nginx	高性能 HTTP 和反向代理服务器，也用作负载均衡和静态资源服务器
	Canal	基于 MySQL binlog 的增量数据同步工具，用于数据实时同步
	Filebeat	轻量级日志采集器，用于收集服务器日志并发送到 Elasticsearch 等
	Logstash	数据处理管道工具，用于收集、转换和发送各种来源的数据
	DataX	离线数据同步工具，支持多种数据源之间的高效数据传输
	埋点 SDK	用于在应用中嵌入代码，采集用户行为、系统性能等数据的开发工具
离线数据集成 离线数据采集	Sqoop	用于在 Hadoop 和关系型数据库之间高效传输大批量数据的工具
	ETL	数据抽取、转换、加载的过程，用于数据集成与清洗
	DataX	离线数据同步工具，支持多种数据源之间的高效数据传输
	Kettle	开源 ETL 工具，用于数据集成、转换和加载，支持可视化开发
	Talend	企业级数据集成平台，提供 ETL、数据质量管理等全方位数据解决方案
	Informatica	企业级数据集成软件，提供数据治理、ETL、数据质量等功能
	DMETL	达梦数据集成工具，用于异构数据源之间的数据抽取、转换和加载
消息中间件 实时缓冲	Kafka	分布式流处理平台，用于高吞吐、低延迟的实时数据发布与订阅
	RabbitMQ	基于 AMQP 协议的消息队列，用于应用间异步通信和解耦
	RocketMQ	阿里开源的分布式消息中间件，支持高吞吐、高可用的消息传递
	ActiveMQ	开源消息中间件，支持多种消息协议，用于企业级消息通信
	Pulsar	云原生分布式消息流平台，高可用、可扩展

实时计算	SparkStreaming	Spark 生态的准实时流计算组件
	Storm	分布式实时计算系统，用于处理无限流数据的实时分析
	Flink	分布式流处理框架，支持高吞吐、低延迟的实时数据计算
	KafkaStreams	Kafka 自带轻量级实时流处理引擎
	ApacheSamza	基于 Kafka 的分布式流处理框架
	ApacheHeron	Twitter 开源实时计算引擎，替代 Storm
	Tajo	分布式 SQL 查询引擎，用于 Hadoop 数据分析
	Apex	企业级分布式实时流处理引擎
离线计算 批量处理引擎	Impala	实时 SQL 查询引擎，用于 Hadoop 生态系统的交互式分析
	Hive	基于 Hadoop 的数据仓库工具，支持类 SQL 查询分析大规模数据
	Spark	快速通用的计算引擎，支持批处理、流处理和机器学习
	Mapreduce	Hadoop 经典分布式批处理计算模型
	Airflow	工作流调度工具，编排 ETL 与数据任务
	Presto	分布式 SQL 查询引擎，多数据源联邦查询
	Tez	基于 Hadoop YARN 的数据流处理框架，优化 MapReduce 作业
实时数仓 实时 OLAP 计算实时分析	ClickHouse	列式存储数据库，用于实时分析大规模数据的 OLAP 系统
	Doris	新一代 MPP 实时 OLAP 数据库，极速分析
	StarRocks	新一代 MPP 实时 OLAP 数据库，极速分析
	TiDB	分布式 NewSQL 数据库，支持水平扩展和强一致性，兼容 MySQL
	Presto	分布式 SQL 查询引擎，多数据源联邦查询
	Impala	实时 SQL 查询引擎，用于 Hadoop 生态系统的交互式分析
离线数据湖 湖仓一体离线	Hudi	用于构建流数据湖的框架，支持增量数据处理和数据版本管理
	Iceberg	开源表格式，用于管理大规模数据集，支持 ACID 事务和 Schema 演进
	DeltaLake	数据湖存储框架，支持 ACID 与版本管理
实时存储层 计算结果落地	KV/缓存	键值型内存存储，用于加速读写
	Redis	高性能内存数据库，用于缓存、会话存储和实时数据处理
	Tair	阿里分布式高性能 KV 缓存存储
	Memcached	分布式内存缓存系统，用于减轻数据库负载，提高访问速度
	InfluxDB	开源时序数据库，专存监控指标

	Prometheus	开源监控系统，用于收集和分析时间序列数据
	OpenTSDB	基于 HBase 的分布式时序数据库
	TimescaleDB	时序优化型 PostgreSQL 数据库
	HBase	分布式列式数据库，用于存储和管理大规模非结构化数据
	Hudi	用于构建流数据湖的框架，支持增量数据处理和数据版本管理
	Iceberg	开源表格式，用于管理大规模数据集，支持 ACID 事务和 Schema 演进
	DeltaLake	数据湖存储框架，支持 ACID 与版本管理
	MySQL	开源关系型数据库，广泛用于 Web 应用和企业系统
	PostgreSQL	开源关系型数据库，支持复杂查询和 JSON 数据类型
	MemSQL	内存关系型数据库，高并发低延迟
	Elasticsearch	分布式搜索引擎，用于全文检索和数据分析
	Cassandra	分布式 NoSQL 数据库，提供高可用性和可扩展性的数据存储
	HDFS	Hadoop 分布式文件系统，用于存储大规模数据集的分布式存储系统
	CDH	Cloudera Hadoop 发行版，大数据全套套件
	Elaticsearch	分布式搜索引擎，用于全文检索和数据分析
	Opentsdb	时序监控数据库
	Hive	基于 Hadoop 的数据仓库工具，支持类 SQL 查询分析大规模数据
	Oceanus	腾讯云实时计算平台
	Greenplum	MPP 架构企业级数据仓库
	Teradata	传统企业级数据仓库一体机